Automatic Text Summarizer

Dr. Annapurna P Patil, Shivam Dalmia, Syed Abu Ayub Ansari, Tanay Aul, Varun Bhatnagar
annapurnap2@msrit.edu shivam.dalmia@gmail.com ayub1993@gmail.com
Department of Computer Science
M. S. Ramaiah Institute of Technology
Bangalore

***Abstract*—In today's fast-growing information age we have an abundance of text, especially on the web. New information is constantly being generated. Often due to time constraints we are not able to consume all the data available. It is therefore essential to be able to summarize the text so that it becomes easier to ingest, while maintaining the essence and understandability of the information. We aim to design an algorithm that can summarize a document by extracting key text and attempting to modify this extraction using a thesaurus. Our main goal is to reduce a given body of text to a fraction of its size, maintaining coherence and semantics.**

Keywords-Automatic Summarization, Extraction, Graph-based, Synonyms, Abstraction.

I. INTRODUCTION

Automatic summarization is the process of condensing textual content into a concise form for easy digestion by humans, using a computer program. Summaries can be produced from a single document or multiple documents; they should be short and preserve important information. Summarization can be extractive or abstractive: extractive summarization aims to extract important and relevant parts of the given content and fuse them coherently. Abstractive summarization aims at paraphrasing the source document, similar to manual summarization. Automatic text summarization is a useful tool when there is a lot of textual information to be analyzed manually. Automatic summarization is used to condense the large amounts of textual data. This achieves the following benefits:

- Firstly, several redundancies can be removed. The user does not waste time reading repetitive data.
- Secondly, summarization allows you to remove data that is not essential to the understanding of the document.

II. GENERAL DESCRIPTION

There are many methods to proceed with automatic text summarization. In our model we use an extractive technique to obtain the summary from the given text. This summary is then improved further by replacing a few parts of it using an abstractive technique. The extraction of sentences from the document is done keeping coherence in mind and therefore the summary maintains the essence of the original document. The sentences are then ranked using a text- ranking algorithm, namely TextRank[1] and the final cluster or summary is formed.

The important functions of the summarizer are:

• Reducing a single document to a user-defined fraction of its original size while maintaining coherence.

• Choosing the most relevant and important sentences from the text.

• Improving the abstraction and/or length of the summary by using a thesaurus to replace semantically related units.

In effect, we aim to extractively summarize a single English document, not more than 300 sentences long, to a fraction of its original size, while maintaining cohesion, and then use a lexical database to abstract the generated summary.

The software uses the external tool WordNet[4] to abstract the generated summary. WordNet is a lexical database that groups words by semantic relations. The Natural Language Toolkit (NLTK) for Python[6] is used to access the database through the program. ROUGE[5] is used for evaluating the summarization.

978-1-4799-3080-7/14/$31.00 ©2014 IEEE

Authorized licensed use limited to: Jain University. Downloaded on March 03,2025 at 04:04:47 UTC from IEEE Xplore. Restrictions apply.

III. Related Work

The extractive summary is generated completely using the principles outlined in the TextRank extraction technique, described in [1].

References [2] and [3] describe graph based approaches to replacing words with synonyms. In [2] the concept of using the context and possible synonyms of a word to form a graph has been explored. We use a similar method, however the similarity scores are generated using the WordNet[4] similarity function, and then the scores of vertices of the graph are calculated using Equation[2]. This results in a procedure similar to that of TextRank[1], but operating on words, and hence informally referred to as WordRank.

IV. Design

A. Architectural Design

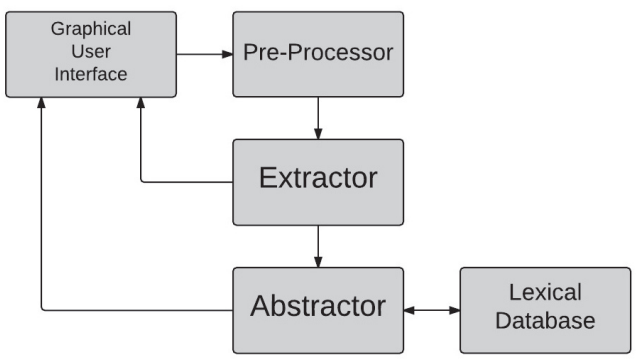


Figure [1]

The architectural design of the algorithm is as illustrated by the above diagram (Figure [1]). The User Interface is the front-end of the algorithm which is used for the upload of the text files and to provide the results of the summarizer. The pre-processor performs certain preprocessing tasks on the document, such as splitting into sentences, removal of stop-words and stemming. The Extractor provides an extractive summary, which contains the top r% of the important sentences of the document (where r is entered by the user). The Abstractor uses the Lexical Database which includes the WordNet thesaurus for the paraphrasing of the summary provided by the Extractor.

B. Design Rationale

In this architectural design, The Extractive Summarizer merely copies the information scored highest by the system to the summary. This process is implemented using [1]. However, the process of abstraction of the summary based on the semantics is the main focus of this architecture for the algorithm. The next step involves altering the text using the lexical database which in this case is the WordNet thesaurus.

V. Implementation

The preprocessing is done in the following steps:

1. The given text is extracted from the file and split into sentences based on punctuation like periods (.), interrogation marks (?), and exclamation marks (!).
2. The sentences are each split into words on whitespaces.
3. Stop-words such as ‘a’, ‘an’, ‘the’, ‘of’, ‘I’, are removed from the document, as are any trailing punctuation marks.
4. The remaining words are stemmed into their base forms. E.g. ‘replacement’, ‘replacing’, ‘replaced’, ‘replace’ are all stemmed to ‘replace’.
   NOTE: These steps are done only to facilitate the processing. The results contain all the words in their original forms.
5. The resulting words are stored as a list according to their sentences.

The TextRank algorithm is then implemented, as follows:

1. First, each sentence (a node in the graph) is compared with every other sentence. A similarity score is assigned to the pair, i.e., an edge weight based on the following formula:

$$Similarity(S_i, S_j) = \frac{|\{w_k \mid w_k \in S_i \,\&\, w_k \in S_j\}|}{\log|S_i| + \log|S_j|} \tag{1}$$

   This scores the similarity between two sentences in terms of the number of common words between the two sentences, divided by the sum of the logarithms of their lengths.
2. The above generates an adjacency matrix with weights assigned to edges between pair of nodes. From this matrix, we generate scores for each node V_i by running the following formula iteratively:

$$WS(V_i) = (1-d) + d \times \sum_{V_j \in In(V_i)} \frac{w_{ji}}{\sum_{V_k \in Out(V_j)} w_{jk}} WS(V_j) \tag{2}$$

Authorized licensed use limited to: Jain University. Downloaded on March 03,2025 at 04:04:47 UTC from IEEE Xplore. Restrictions apply.

Here $G=(V,E)$ is a directed graph with the set of vertices V and set of edges E. Each edge E_{ij} has weight w_{ij}. For a given vertex $In(V_i)$ is the set of vertices that point to it (predecessors), and $Out(V_i)$ is the set of vertices that vertex Vi points to (successors). In this situation, since the graph is undirected $In(V_i)$ is equal to $Out(V_i)$. The above equation (Equation [2]) gives the score of a vertex V_i, where d is a damping factor that can be set between 0 and 1, which has the role of integrating into the model the probability of jumping from a given vertex to another random vertex in the graph. The factor d is usually set to 0.85 [1].

3. The above formula is run until the weighted score of each node does not change beyond 5 decimal places. Then we have a weighted score for each of the sentences in the document. We merely pick out the sentences with the highest weights and display them as per their initial order in the document.

Finally, the Abstraction works as follows:

1. The given text is extracted from the summary and split into sentences based on punctuation like periods (.), interrogation marks (?), and exclamation marks (!).
2. The sentences are each split into words on whitespaces.
3. Stop-words such as ‘a’, ‘an’, ‘the’, ‘of’, ‘I’, are removed from the document, as are any trailing punctuation marks.
4. The resulting words are stored as a list according to their sentences.
5. Using the NLTK brown corpus, we train a unigram tagger to tag individual words with their parts of speech (noun, verb, adjective, adverb).
6. For words of length greater than five, we do the following:
   a) Get a set of senses for the word using WordNet.
   b) Get the basic sense of each word in the context associated with the given verb.
   c) Using these set of senses, we rank them using a WordRank algorithm, which is similar to TextRank, but with words instead of sentences. This gives us ranks of all the senses of the given word related to its context. Out of these we pick the sense with the highest rank as it is most likely to fit into that context.
   d) From the obtained sense, we obtain a synonym of the given word which is the shortest possible.
   e) If we can obtain a synonym different from the original word, the word is replaced with this synonym.

VI. RESULTS

Qualitatively we have ascertained that important sentences are being selected during the extraction phase and during the abstraction phase, a few words are being replaced by not inappropriate synonyms. However, the abstraction phase performs a very sparse replacement currently.

The current algorithm seems to work well on news articles, technical documents and encyclopaedic entries. However, on essays, fiction and documents with a lot of direct speech the extraction does not seem to yield very coherent summaries.

Below is an analysis of the performance of our system against certain reference summaries. The documents analyzed were news articles, some of which are shown hereafter, along with their individual results. The average precision, recall and f-scores returned by the ROUGE evaluation metric are shown. A high **precision** means that the algorithm returned substantially more relevant results than irrelevant, while high **recall** means that the algorithm returned most of the relevant results. A measure that combines precision and recall is the harmonic mean of precision and recall, the balanced F-score:

$$F = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \tag{3}$$

The results show that the algorithm performed reasonably well, the score being equivalent to TextRank scores. The average score is above the DUC (Document Understanding conference) baseline of 0.4162

```
ROUGE-SU* Average_Recall:
0.55881 (95%-conf.int. 0.44861 - 0.67397)

ROUGE-SU* Average_Precision:
0.37666 (95%-conf.int. 0.22824 - 0.46657)

ROUGE-SU* Average_F_Score:
0.42277 (95%-conf.int. 0.32318 - 0.52257)
```


Authorized licensed use limited to: Jain University. Downloaded on March 03,2025 at 04:04:47 UTC from IEEE Xplore. Restrictions apply.

VII. Test Case

The following sample was taken to test the proposed concept.

Original Document [8]:

Number of Words: 245

Number of Sentences: 19

Number of Characters: 1212

"Thomas Alva Edison was one of the greatest inventors of the 19th century. He is most famous for inventing the light bulb in 1879.

He also developed the world's first electric light-power station in 1882. Edison was born in the village of Milan, Ohio, on February 11, 1847.

His family later moved to Port Huron, Michigan. He went to school for only three months, when he was seven. After that, his mother taught him at home.

Thomas loved to read. At twelve years old, he became a train-boy, selling magazines and candy on the Grand Trunk Railroad.

He spent all his money on books and equipment for his experiments. At the age of fifteen, Edison became manager of a telegraph office.

His first inventions helped improve the telegraph, an early method for sending messages over electric wires.

At twenty-one, Edison produced his first major invention, a stock ticker for printing stock-exchange quotes. He was paid $40,000 for this invention.

He took this money and opened a manufacturing shop and small laboratory in Newark, New Jersey.

Later he gave up manufacturing, and moved his laboratory to Menlo Park, New Jersey. At this laboratory, he directed other inventors.

During the rest of his life he and his laboratory invented the phonograph, film for the movie industry, and the alkaline battery.

By the time he died at West Orange, New Jersey on October 18, 1931, he had created over 1,000 inventions."

Extractive Summary (Ratio -33%):

Number of Words: 116

Number of Sentences: 9

Number of Characters: 590

"Thomas Alva Edison was one of the greatest inventors of the 19th century.

He is most famous for inventing the light bulb in 1879.

His first inventions helped improve the telegraph, an early method for sending messages over electric wires.

At twenty-one, Edison produced his first major invention, a stock ticker for printing stock-exchange quotes.

He was paid $40,000 for this invention.

He took this money and opened a manufacturing shop and small laboratory in Newark, New Jersey.

Later he gave up manufacturing, and moved his laboratory to Menlo Park, New Jersey.

During the rest of his life he and his laboratory invented the phonograph, film for the movie industry, and the alkaline battery.

By the time he died at West Orange, New Jersey on October 18, 1931, he had created over 1,000 inventions."

Authorized licensed use limited to: Jain University. Downloaded on March 03,2025 at 04:04:47 UTC from IEEE Xplore. Restrictions apply.

Abstractive Summary:

Number of Words:116

Number of Sentences: 9

Number of Characters: 559

"Thomas Alva Edison was one of the greatest inventors of the 19th century.

He is most <famed> for inventing the light bulb in 1879.

His first <designs> helped improve the telegraph, an early method for sending messages over electric wires.

At twenty-one, Edison produced his first major <design>, a stock ticker for printing stock-exchange quotes.

He was paid $40,000 for this <design>.

He took this money and opened a manufacturing shop and small <lab> in Newark, New Jersey.

Later he gave up manufacturing, and moved his <lab> to Menlo Park, New Jersey.

During the rest of his life he and his <lab> invented the phonograph, film for the movie industry, and the <alkalic> battery.

By the time he died at West Orange, New Jersey on October 18, 1931, he had created over 1,000 <designs>."

VIII. Conclusion And Future Work

Our algorithm has proved to perform well for most summarization purposes. The current extractive summary is advantageous for certain formats of documents. The abstraction is slight and marginally improves readability and length. However, the abstraction does not strictly generate an abstractive summary in the true sense, as natural language processing techniques are not used. The advantage of this method is that it operates completely algorithmically, and does not require sophisticated techniques. However, often the replacement is not sufficiently appropriate or ideal.

Future work that we aim for is as follows:

1. Using the lexical database for improving cohesion among sentences while extraction. [4] This will possibly improve the extraction of essays, etc.
2. Improving efficiency and speed of extraction and abstraction.
3. Replacing words with better synonyms and including replacement of digrams, trigrams and phrases.
4. Using NLP to improve abstraction.

References

[1] Mihalcea, Rada, and Paul Tarau. "TextRank: Bringing order into texts." Proceedings of EMNLP. Vol. 4. No. 4. 2004.

[2] Ravi Som Sinha and Rada Flavia Mihalcea, "Using centrality algorithms on directed graphs for synonym expansion." FLAIRS Conference, AAAI Press, 2011.

[3] Blondel, Vincent D., and Pierre P. Senellart. "Automatic extraction of synonyms in a dictionary." vertex 1 (2011): x1.

[4] Sankar, K., and L. Sobha. "An approach to text summarization." Proceedings of the Third International Workshop on Cross Lingual Information Access: Addressing the Information Need of Multilingual Societies. Association for Computational Linguistics, 2009.

[5] George A. Miller (1995). "WordNet: A Lexical Database for English." Communications of the ACM Vol. 38, No. 11: 39-41.

Christiane Fellbaum (1998, ed.) "WordNet: An Electronic Lexical Database." Cambridge, MA: MIT Press.

[6] Lin, C. Y. (2004, July). "Rouge: A package for automatic evaluation of summaries." In Text Summarization Branches Out: Proceedings of the ACL-04 Workshop (pp. 74-81)

[7] Bird, Steven, Edward Loper and Ewan Klein (2009), "Natural Language Processing with Python." O'Reilly Media Inc.

[8] Sample Text Source: Grolier Electronic Publishing, Inc., 1995

Authorized licensed use limited to: Jain University. Downloaded on March 03,2025 at 04:04:47 UTC from IEEE Xplore. Restrictions apply.

Automatic Text Summarization Methods: A Comprehensive Review

Divakar Yadav[1], Jalpa Desai, Arun Kumar Yadav[2]

20mcs105@nith.ac.in, dsy99@rediffmail.com, ayadav@nith.ac.in

ORCID: [1]https://orcid.org/0000-0001-6051-479X , [2]https://orcid.org/0000-0001-9774-7917

Abstract:

One of the most pressing issues that have arisen due to the rapid growth of the Internet is known as information overloading. Simplifying the relevant information in the form of a summary will assist many people because the material on any topic is plentiful on the Internet. Manually summarising massive amounts of text is quite challenging for humans. So, it has increased the need for more complex and powerful summarizers. Researchers have been trying to improve approaches for creating summaries since the 1950s, such that the machine-generated summary matches the human-created summary. This study provides a detailed state-of-the-art analysis of text summarization concepts such as summarization approaches, techniques used, standard datasets, evaluation metrics and future scopes for research. The most commonly accepted approaches are extractive and abstractive, studied in detail in this work. Evaluating the summary and increasing the development of reusable resources and infrastructure aids in comparing and replicating findings, adding competition to improve the outcomes. Different evaluation methods of generated summaries are also discussed in this study. Finally, at the end of this study, several challenges and research opportunities related to text summarization research are mentioned that may be useful for potential researchers working in this area.

Keyword: Automatic text summarization, Natural Language Processing, Categorization of text summarization system, abstractive text summarization, extractive text summarization, Hybrid Text Summarization, Evaluation of text summarization system

1. Introduction:

The task of compressing a piece of text into a shorter version, minimizing the size of the original text while keeping crucial informational aspects and content meaning, is known as summarization. Fig. 1 shows task of summarization in a simple way. A summary is a reductive transformation of a source text into a summary text by extraction or generation (Radev et al., 2004). According to another definition, "An automatic summary is a text generated by a software that is coherent and contains a significant amount of relevant information from the source text. Its compression rate τ is less than a third of the length of the original document (Hovy & Lin, 1996). The ratio between the length of the summary and the length of the source document is calculated by the compression rate τ as shown below:

$$\tau = \frac{|Summary|}{|Source|}$$

Where | • | indicates the length of the document in characters, words, or Sentences. τ can be expressed as a percentage. In fact, (C. Y. Lin, 1999) study shows that the best performances of automatic summarization systems are found with a compression rate of τ = 15 to 30% of the length of the source document.

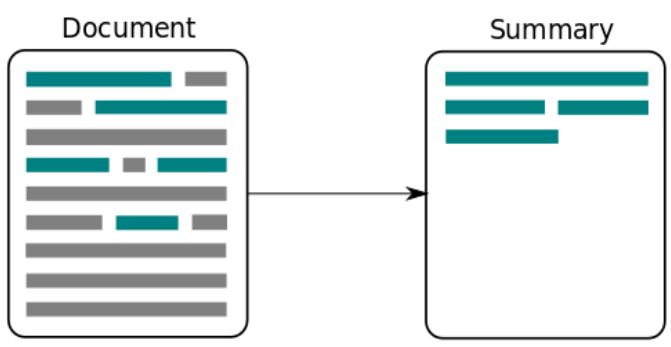


Fig. 1: generating summary from input document

Understanding the source text and creating a brief and abbreviated version of it are two processes in the human generation of summaries. Fig. 2 shows how human produces the summaries of an original text document. The summarizer's linguistic and extra-linguistic abilities and knowledge are required for both understanding the material and producing summaries. Although people can write better summaries (in terms of readability, content, form, and conciseness). Automatic text summarizing is a useful supplement to manual summation rather than a substitute.

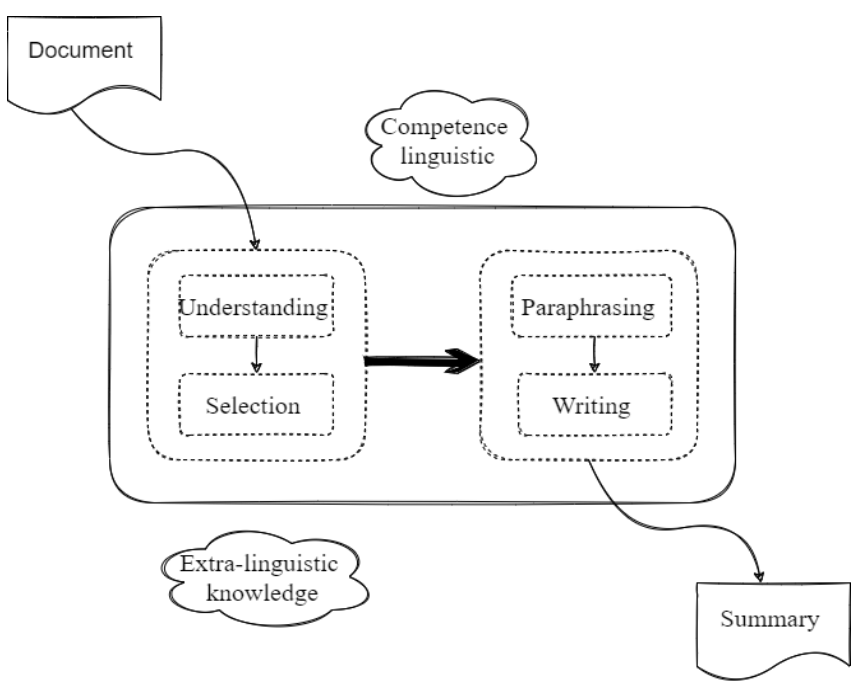


Fig. 2: human's process for generating summaries

1.1 Requirement of Text Summarization

The adage "too much information kills information" is as relevant today as it has ever been. The fact that the Internet is available in various languages only adds to the aforementioned document analysis challenges. Automatic text summarization aids in the effective processing of an ever-increasing volume of data that humans are just unable to handle. Let's look at some eye-opening facts about the world of data provided by Arne von See (2021) as shown in fig. 3. Some facts about it are: in the previous two years, 90 percent of the world's data has been created. The majority of businesses only look at 12% of their data. Each year, bad data costs the United States $3.1 trillion. By 2025, the amount of data created will have surpassed 180 zettabytes. To download all of the material from the internet now, it would take a human around 181 million years.

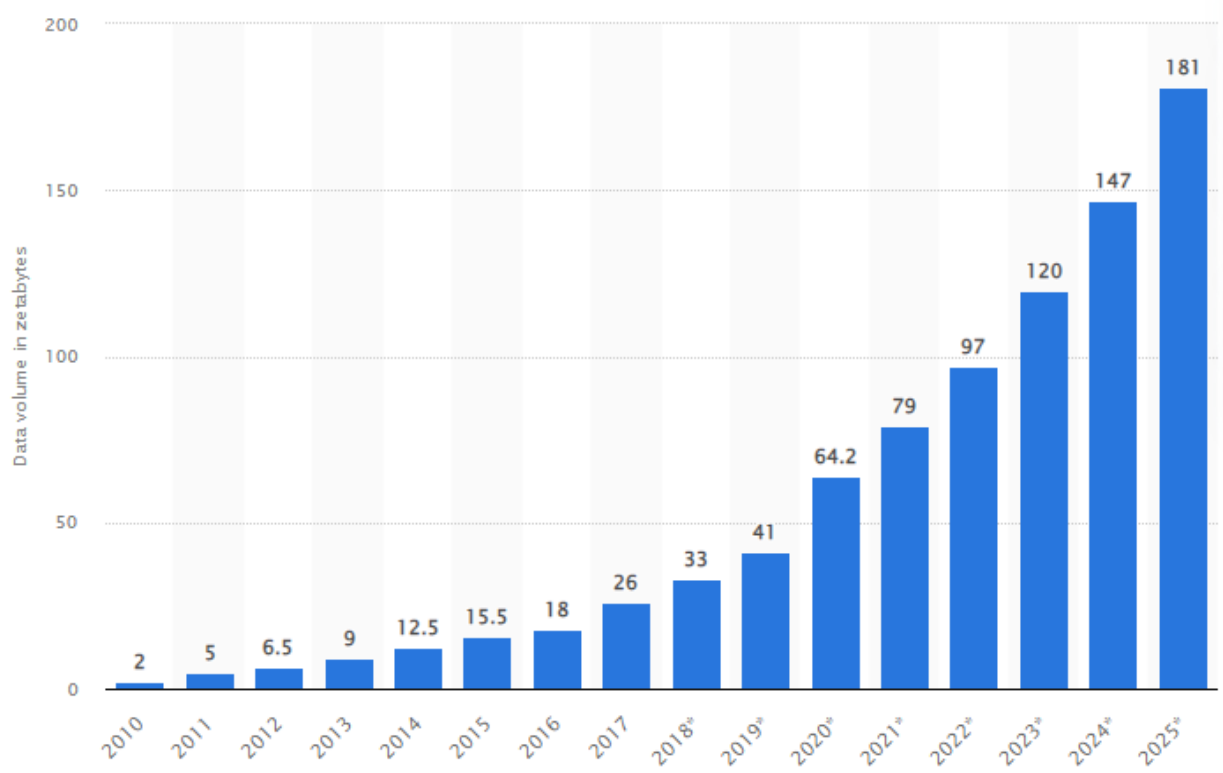


Fig. 3. Volume of data/information created, captured, copied, and consumed worldwide from 2010 to 2025 (Arne von See, 2021)

There are several valid reasons in favour of the automatic summarization of documents. Here are listed just a few (Ab & Sunitha, 2013)

i. Summaries saves reading time.
ii. Summaries help in the selection of documents when conducting research.
iii. Indexing is more successful when automatic summarization is used.
iv. When compared to human summarizers, automatic summary systems are less biased.
v. Because they provide personalized information, personalized summaries are important in question-answering systems.
vi. Commercial abstract services can improve the number of texts they can process by using automatic or semi-automatic summarizing techniques.

Automatic Text Summarization (ATS) is a relatively new learning issue that has gotten a lot of interest. As research advances, we hope to see a breakthrough that will help with this by giving a timely technique of summarising big texts. We present an overview of text summarising techniques in this work to highlight their usefulness in dealing with enormous data and to assist researchers in using them to address challenges**.** The fig. 4 shows the number of researcher papers published in domain of text summarization in a particular time interval staring from 1958.

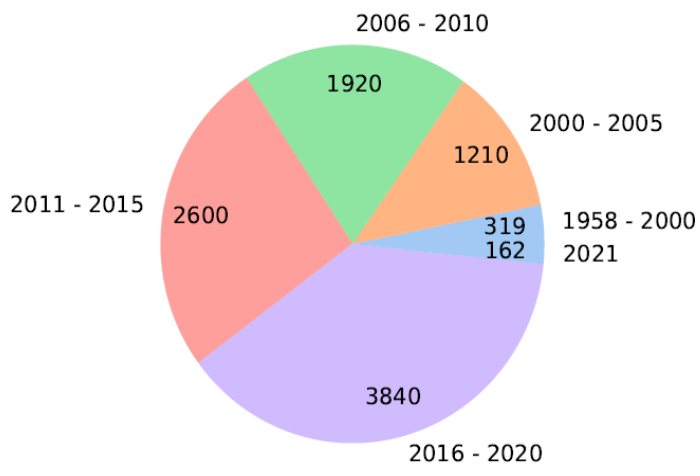


Fig. 4: Number of Research articles published in the domain of ATS in different time interval

1.2 Main Contribution of study

This work provides a concise, current and comprehensible view in the field of text summarization. The major contributions of this study are as under:

a. Starting from ground level, make the reader comfortable with the ATS system and why we need an ATS system. Provided examples of ATS systems presented in the literature for each application and illustrated ATS systems' classifications.
b. Provided a detailed analysis of the three ATS approaches extractive, abstractive, and hybrid. Furthermore, the review table is built on factors like dataset, approach, performance, advantages, and disadvantages.
c. Provided an overview of the standard datasets and provided complete details about evaluation methods available for the ATS system.
d. Detailed analysis of challenges and future scopes for text summarization.

This article is arranged into six sections. Section 1 discusses the introduction of an automatic text summarization system with its requirements and applications. The automatic text summarization is divided into many categories discussed in detail in section 2. Next, section 3 focused on Extractive, Abstractive and Hybrid text summarization. The evaluation methods for summaries generated by the system are discussed in section 4. Later that frequently used datasets for summarization task is listed in section 5. Lastly, the conclusion is given in section 6.

2. Categorization of ATS

There are different classifications for an automatic text summarization (ATS) system based on its input, output, purpose, length, algorithms, domain, and language. There are many other factors that can be considered while discussing the classification of summarization. Different researchers have considered different factors. As per our survey, the detailed categorization of an ATS system is given in fig. 5. A detailed explanation of a particular category are discussed in following sub-sections as under:

2.1 Based on no. of Input documents

Based upon size of input source documents that are used to generate a summary, summarization can be divided in two types:

- **Single Document**: Single document text summarization is automatic summarization of information a single document (Garner, 1982).

- **Multiple Document:** Multi-document text summarization is an automatic summarization of information from multiple document (Ferreira et al., 2014).

Multi-document summarization is important where we must put different types of opinions together, and each idea is written with multiple perspectives within a single document. Single document text summarization is easy to implement, but multi-document summarization is a complex task. Redundancy is one of the biggest problems in summarizing multiple documents. Carbonell & Goldstein (1998) has given MMR (Maximal Marginal Relevance) approach, which helps to reduce redundancy. Another main problem for multi-document summarization is heterogeneity within a large set of documents. It is very complex to summarize multiple documents with extractive methods where there are so many conflicts and biases in the real world. Here for multiple documents, abstractive summarization performs far better. However, multi-document summarization also brings issues like redundancy in output summary while working with a huge number of documents. Single document text summarization is used in a limited field like reading the given comprehension and giving an appropriate title or summary. In contrast, multi-document text summarization can be used in the field of news summarization from different sites, customer's product reviews from different vendors, Q&A systems and many more.

SummCoder (Joshi et al., 2019) is a new methodology for generic extractive single document text summarization. The method creates a summary based on three criteria they developed: sentence content relevance, sentence novelty, and sentence position relevance. The novelty metric is produced by utilizing the similarity among sentences represented as embedding in a distributed semantic space, and the sentence content relevance is assessed using a deep auto-encoder network. The sentence position relevance metric is a custom feature that gives the initial few phrases more weight thanks to a dynamic weight calculation method controlled by the document length. In the extractive multi-document text summarization field, Sanchez-Gomez et al. (2021) shows that all feasible combinations of the most prevalent term-weighting schemes and similarity metrics have been implemented, compared, and assessed. Experiments with DUC datasets were conducted, and the model's performance was evaluated using eight ROUGE indicators and the execution time. The TF-IDF weighting scheme and cosine similarity give the best result of 87.5% ROUGE score as a combination.

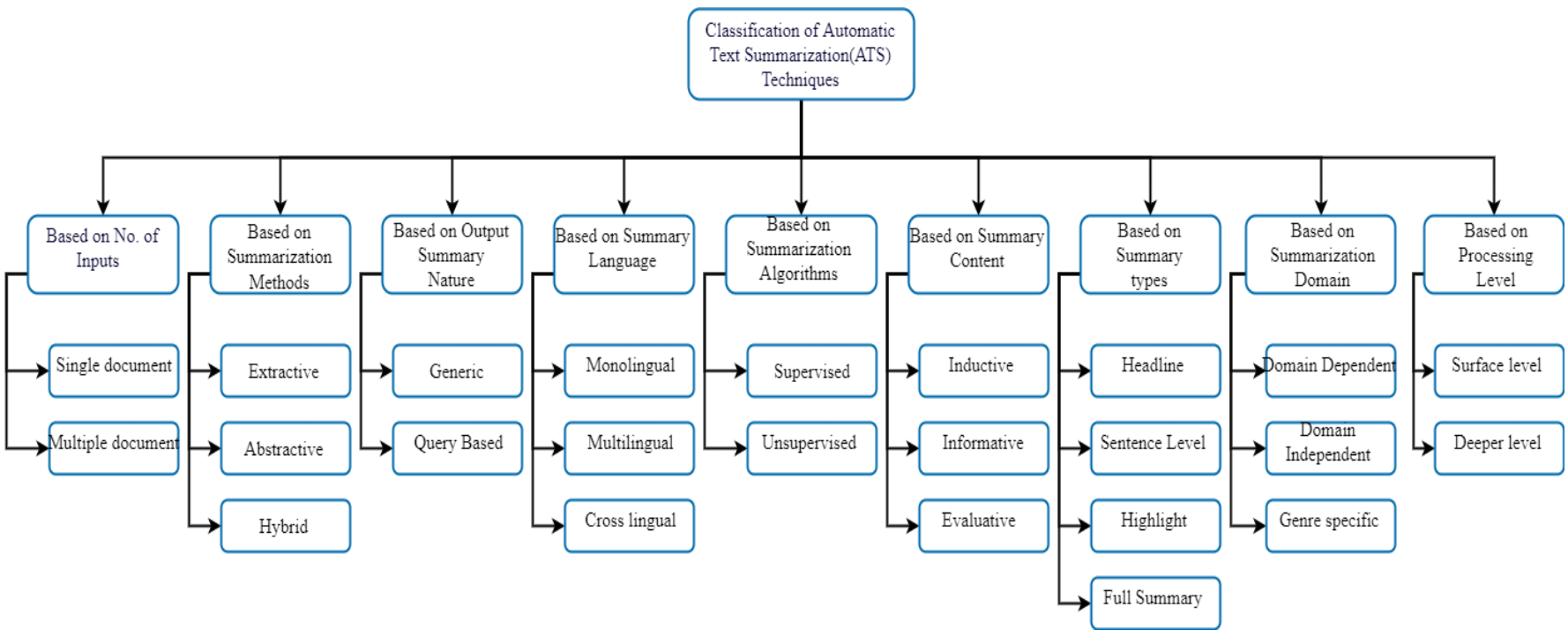


Fig. 5: Detailed Categorization of automatic text summarization system

2.2 Based on Summarization Methods

Based on methods that how can summaries are produced, i.e. Just picking up sentences from the source text or generating new sentences after reading source text or a combination of both, summarization can be divided into three types:

- ***Extractive Automatic Text Summarization:*** Extractive text summarization is the strategy of concatenating on extracting summary from a given corpus (Rau et al., 1989).
- ***Abstractive Automatic Text Summarization:*** Abstractive text summarization involves paraphrasing the given corpus and generating new sentences (Zhang et al., 2019).
- ***Hybrid Automatic Text Summarization:*** It combines both extractive and abstractive methods. It means extracting some sentences and generating a new one from a given corpus (Binwahlan et al., 2010).

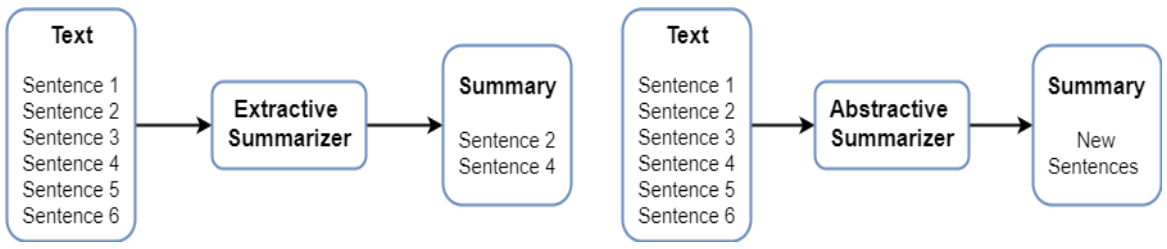


Fig. 6: Extractive text summarizer and Abstractive text summarizer

Think of a highlighter used to point out important sentences in a book. It is an example of extractive text summarization. Now think of the notes we prepare from a book using our own words. It is an example of abstractive text summarization. Extractive text summarization is like copy-pasting some of the important sentences from the source text, while abstractive text summarization selects some meaningful sentences and generates new sentences from previously selected sentences. Refer Fig. 6 for a better understanding of Extractive and Abstractive summarization. Hybrid text summarization combines an approach for producing a summary efficiently. Both Extractive and Abstractive text summarization falls into Machine Learning and NLP domain. Additionally, abstractive text summarization covers NLG. The survey of both approaches is shown in the later section of this article. Critical areas where extractive text summarization is applied are news, medical, book, legal document, abstractive text summarization, customer reviews, blog, tweet summarization, etc.

The plus point of the extractive text summarization model is that the sentences in the summaries must adhere to the syntactic structure's constraints. However, that model's shortcoming is that the summaries' sentences may not be semantically meaningful. This disadvantage arises because adjacent sentences in the summaries are not always contiguous in the original text. Because ATS models learn the collocation between words and construct a sequence of keywords based on the collocation between words after training, they have the advantage of inclusive semantics. The downside of ATS models is that it is challenging to meet the criterion of syntactic structure with this sequence of keywords. Rare words are another major flaw in traditional ATS models. The number of occurrences of a rare word and its collocation will define its importance, but humans will use other elements to assess whether a word is essential. As a result, in some instances, some words that appear infrequently might be deemed unimportant, although a portion of these words is critical for summary construction from a human perspective (Song et al., 2019b).

2.3 Based on Output Summary Nature:

Based on the output summary's characteristics the ATS system can be divided into two types:

- ***Generic:*** Generic text summarizers fetch important information from one or more documents to provide a concise meaning of given document(s) (Aone et al., 1997).

- ***Query-Based:*** A query-based summarizer is built to handle multi-documents and gives a solution as per the user's query (Van Lierde & Chow, 2019). The score of sentences in each document is based on the frequency counts of words or phrases in query-based text summarization. Sentences containing query phrases receive higher marks than sentences containing single query words. The sentences with the highest scores and their structural context are extracted for the output summary (Kiyani & Tas, 2017)..

A query-based sentence extraction algorithm is given as below (Pembe & Güngör, 2007):

i. Rank all the sentences according to their score.
ii. Add the main title of the document to the summary.
iii. Add the first level-1 heading to the summary.
iv. While (summary size limit not exceeded)
v. Add the next highest scored sentence.
vi. Add the structural context of the sentence: (if any and not already included in the summary)
vii. Add the highest-level heading above the extracted text (call this heading h).
viii. Add the heading before h in the same level.
ix. Add the heading after h in the same level.
x. Repeat steps 7, 8 and 9 for the subsequent highest-level headings.
xi. End while

A query is not used in generic summaries. Because they do not comprehensively assess the original document, query-based summaries are biased. They are not ideal for content overview because they solely deal with user queries. Generic summaries are necessary to specify the document's category and to describe the document's essential points. The key subjects of the documents are considered in the best general summary, which strives to minimize redundancy as much as possible (Kiyani & Tas, 2017).

2.4 Based on Summary Language

Based on the language of input and output of the ATS system, it can be divided into the following 3 categories:

- ***Monolingual:*** In a Monolingual text summarizer, the language of the input document and output summary is the same (Kutlu et al., 2010).
- ***Multilingual:*** In a Multilingual text summarizer, input is written in many languages (Hindi, English, and Gujarati), and output summary is generated likewise in these languages (Hovy & Lin, 1996).
- ***Cross- Lingual:*** In a Cross-lingual text summarizer, the input document is in one language (say English), and the output summary is in another language (say Hindi) (Linhares Pontes et al., 2020).

Most of the research papers studied in this article are based on monolingual text summarization. Compared to monolingual, multilingual and cross-lingual is challenging to implement. It takes more effort to train a machine on more than one language structure. SUMMARIST (Hovy & Lin, 1996) is a multilingual text summarization system based on an extraction strategy that generates summaries from English, Indonesian, Spanish, German, Japanese, Korean, and French sources. Cross-Language Text Summarization (CLTS) (Linhares Pontes et al., 2020) generates a summary in a target language from source language materials. It entails a combination of text summarising and machine translation methods. Unfortunately, this combination leads to mistakes, which lowers the quality of summaries. CLTS systems may extract relevant information from both source and destination languages through joint analysis, which improves the development of extractive cross-lingual summaries. Recent methods for CLTS have offered compressive and abstractive approaches; however, these methods rely on frameworks or tools that are only available in a few languages, restricting their applicability to other languages.

2.4 Based on Summarization Algorithms

Based on the actual algorithm that is used to generate the summarises, the ATS system is divided into two types as given below:

- ***Supervised:*** The supervised summarizer needs to train the sample data by labelling the input text document with the help of human efforts (Neto et al., 2002).
- ***Unsupervised:*** In the Unsupervised summarizer training phase is not needed (Alami et al., 2019).

In order to select important content from documents in a supervised system, training data is required. Training data is a large volume of labelled or annotated data is required for learning techniques. These systems are approached as a two-class classification issue at the sentence level, with positive samples being sentences that belong to the summary and negative samples being sentences that do not belong to the summary. On the other hand, unsupervised systems do not require any training data. They create the summary by just looking at the documents they want to look at for summarization. As a result, they can be used with any newly observed data without needing extra adjustments. These systems use heuristic methods to extract relevant sentences and construct a summary. Clustering is used in unsupervised systems (Gambhir & Gupta, 2017).

A single document supervised machine learning-based approach for the Hindi language is given by Nikita (2016). The sentences are divided into four categories: most influential, important, less important, and insignificant. The summarizer is then trained using the SVM supervised machine learning algorithm to extract important sentences based on the feature vector. Sentences are then included in the final summary based on the required compression ratio. The experiment was carried out on news stories from various categories such as Bollywood, politics, and sports, and the results showed 72 percent accuracy at a compression ratio of 50 percent and 60 percent at a compression ratio of 25 percent. Recently, an unsupervised neural network approach has been studied by Meknassi et al. (2021) for Arabic language text summarization. A new approach using documents clustering, topic modelling, and unsupervised neural networks have been proposed to build an efficient document representation model to overcome problems raised with Arabic text documents. The proposed approach is evaluated on Essex Arabic Summaries Corpus and compared against other Arabic text summarization approaches using ROUGE measure.

2.5 Based on Summary Content

Based on the type of the content of the output summaries, the system is categorised into two parts as:

- ***Inductive:*** Indicative summary contains only generic idea about the source document (Bhat et al., 2018).
- ***Informative:*** Informative summary contains all the main topics about the original document (Bhat et al., 2018)**.**
- ***Evaluative or Critical:*** It capture the summary from author's point of view on a given topic (Jezek & Steinberger, 2008)**.**

Inductive summarization is used to indicate what the document is all about, and it aims to give an idea to a user whether to read this original document or not. The length of this summary is approximately 5% of the original content. The informative summarization system summarises the primary text concisely. The helpful summary is around 20% of the whole text length (Kiyani & Tas, 2017). A typical example of evaluative summaries are reviews, but they are pretty out of the scope of nowadays summarizers. It should be emphasized that the three groupings indicated above are not mutually exclusive and are common summaries that have both an informative and an indicative role. Informative summarizers are frequently used as a subset of indicative summarizers (Jezek & Steinberger, 2008).

2.6 Based on Summary types

Based on the length of the generated summaries, the ATS system is divided into four types as given below:

- ***Headline:*** A headline generated from a source document is usually shorter than a sentence (Barzilay & Mckeown, 2005).
- ***Sentence level:*** The sentence-level summarizer produces a single sentence from the original text (Y. H. Hu et al., 2017).
- ***Highlight:*** Highlights are produced in a compressed form of the original text written in bullet points (Tomek, 1998).
- ***Full summary:*** Full summary is generated as per the user's compression rate or user's requirements (Koupaee & Wang, 2018)

Headlines, highlights, and sentence level type of summary are generally used in news database or opinion mining or for social media dataset whereas a full summary is commonly used for all the domains.

2.7 Based on Summarization Domain:

Based on the domain of the input and output of the ATS system, it is divided into following 3 categories:

- ***Genre Specific:*** It accepts only special type of input text format (Hovy & Lin, 1996) .
- ***Domain dependent:*** Domain dependent summarization is specific to one domain (Farzindar & Lapalme, 2004)**.**
- ***Domain independent:*** Domain independent summarization system is independent of source documents' domain.

In genre-specific summarization, there is a restriction on the text template. Newspaper articles, scientific papers, stories, instructions, and other types of templates are available. The summary is generated by the system using the structure of these templates. On the other hand, independent systems have no predefined limitations and can take a variety of text kinds. Furthermore, some techniques only summarise texts whose subject can be characterized in the system's domain; these systems are domain-dependent. These systems impose some restrictions on the topic matter of documents. Such systems know everything there is to know about a specific subject and use that knowledge to summarise. Generally, graph-based techniques are adopted for domain-dependent summarisation as they have sound potential. The authors of (Moradi et al., 2020) have given an efficient solution to deal with the challenges in graph-based methods. To capture the linguistic, semantic, and contextual relationships between the sentences, they trained the model by continuous word representation model. i.e., Word2vec's Skiagrams and Continuous Bag of Words (CBOW) models(Mikolov et al., 2013) and Global Vectors for Word Representation (GloVe) (Mutlu et al., 2020)(Hanson Er, 1971) on a large corpus of biomedical text. To solve the challenge of ranking the most important nodes in a graph, they adopted undirected and weighted graph ranking techniques like the PageRank algorithm (Brin & Page, 2012). Newspaper stories and scientific text have distinct qualities than legal text. In a comprehensive document of the news genre, for example, there is little or no structure. The presence of the same term at different levels of the hierarchy will have distinct effects. The relevance of the words in a judgement is determined by the source of the ruling (whether it is from a District Court, State Court, Supreme Court, or Federal Court). We can generally ignore references/citations when summarizing content; however, this may not be possible in legal writings (Kanapala et al., 2019).

2.8 Based on Processing Level

Based on the processing level of the input document, the system is divided into two types:

- ***Surface-level approaches:*** In this scenario, data is represented by shallow feature ideas and their combinations.

Statistically salient terms, positionally salient terms, cue phrases, domain-specific or a user's query terms are examples of shallow features. The outcomes are in the form of extracts (Ježek et al., 2007).

- ***Deeper-level approaches:*** Extracts or abstracts may be produced through deeper-level techniques. In the latter scenario, synthesis is used to generate natural language. It requires some semantic analysis. For example, entity techniques can construct a representation of text entities (text units) and their relationships to identify salient areas. Entity relationships include thesaural, syntactic, and semantic relationships, among others. They can also use discourse methodologies to represent the text structure, such as hypertext mark-up or rhetorical structure (Ježek et al., 2007).

3 Detailed about ETS, ABS and HTS

Among all the Automatic text summarization system classification, the most commonly accepted or used categories are Extractive, Abstractive and hybrid summarization. Thus, this article focused is on these two approaches mainly. Now lets us see a detailed survey on these approaches:

3.1 Extractive text summarization

Since starting with the era when first-time automatic text summarization came into the picture (Luhn, 1958), the text processing task is performed mainly by using features based on IR (Information Retrieval) measures, i.e., term frequency (TF), inverse term frequency (TF-IDF). Table-1 shows a detailed survey on extractive text summarization with research papers, the dataset used, the system's accuracy, and its pros and cons.Earlier, the efficiency of the summary was prepared by the proportion of no. of judged-important points to total no. of words in the summary (Garner, 1982). The immediate summarization result and relationship to detailed comprehension and recall results were analyzed in that study. The lack of linguistic knowledge is a weak point for extracting helpful information from a large amount of the data. To overcome these two limitations: (i) One mechanism that deals with unknown words and gaps in linguistic information. (ii) To extract linguistic information from text automatically, SCISOR (System for Conceptual Information Summarization, Organization and Retrieval) was developed by Rau et al. (1989). Experiments performed on summarization until 1990 were focused on just extracting (reproduction) the summaries from original text rather than abstracting (newly generated). SUMMRIST system (Hovy & Lin, 1996) was developed with the help of NLP techniques, in which one can create a multi-lingual summarizer by modifying some part of the structure.

The challenges with traditional frequency-based, knowledge-based and discourse-based summarization lead to encountering these challenges with robust NLP techniques like corpus-based statistical NLP (Aone et al., 1997). The summarization system named DimSum consists of a summarization server and summarization client. The features produced from these powerful NLP algorithms were also used to give the user numerous summary views in an innovative way. Evaluating the summaries of humans and systems by four parameters; optimistic evaluation, pessimistic evaluation, intersection evaluation, union evaluation(Salton et al., 1997) and proven that the summaries generated by the two humans are dissimilar for the same article while automatic methods are favourable here. A robust summarization was practically implemented on online news 'New York Times' by Tomek (1998), which gives summaries very quickly with including significantly less portion of original lengthy text. The study of effects of headings on text summarization proven (Lorch et al., 2001) that readers depend heavily on organizational signals to construct a topic structure. The machine learning approach (Neto et al., 2002) considers automatic text summarization as a two-class classification problem, where a sentence is considered 'correct' if it appears in extractive reference summary or otherwise as 'incorrect'. Here they used two famous ML classification approaches, Naïve Bayes and C4.5. Lexicon is a salient part of ant textual data. Focusing on an algorithm (Silber & McCoy, 2002) that efficiently makes lexical chains in linear time is a feasible intermediate representation of text summarization.

As a prior study shows, supervised methods were where human-made summaries helped us find parameters or features of summarization algorithms. Despite that, unsupervised methods (Nomoto & Matsumoto, 2003) with diversity functionality define relevant features without any help from human-made summaries. (Yeh et al., 2005b) proposed a trainable summarizer that generates summaries based on numerous factors such as location, positive keyword, negative keyword, centrality, and resemblance to the title. It uses a genetic algorithm (GA) to train the score function to find a good combination of feature weights. After that, it employs latent semantic analysis (LSA) to derive a document's or corpus' semantic matrix and semantic sentence representation to build a semantic text relationship map. Combining three approaches: a diversity-based method, fuzzy logic, and swarm-based methods (Binwahlan et al., 2010), can generate good summaries. Where diversity-based methods use to figure out similar sentences and get the most diverse sentence and concentrate on reducing redundancy, while swarm-based methods are used to distinguish the most important and less important sentences then use fuzzy logic to tolerate redundancy, approximate values and uncertainty, and this combination concentrates on the scoring techniques of sentences. While comparing two-approach Swarm-fuzzy based methods performs well than diversity-based methods here.

The construction of methods for measuring the efficiency of SAS (Systems of automatic summarization) functioning is an important area in the theory and practice of automatic summarization. Based on a model vocabulary supplied by subjects, four techniques (ESSENCE (ESS), Subject Search Summarizer (SSS), COPERNIC (COP), Open Text Summarizer (OTS)) of automatic text summarization are evaluated by Yatsko & Vishnyakov (2007). The distribution of vocabulary terms in the source text is compared to the distribution of vocabulary terms in summaries of various lengths generated by the systems. (Ye et al. (2007b) contend that the quality of a summary can be judged by how many concepts from the source documents can be retained after summarization. As a result, summary generation can be viewed as an optimization task involving selecting a set of sentences with the least answer loss. The proposed document concept lattice (DCL) is a unique document model that indexes sentences based on their coverage of overlapping concepts. The authors of (Ko & Seo, 2008) suggested method merges two consecutive sentences into a bi-gram pseudo sentence, allowing statistical sentence-extraction tools to use contextual information. The statistical sentence-extraction approaches first choose salient bi-gram pseudo sentences, and then each selected bi-gram pseudo sentence is split into two single sentences. The second sentence-extraction operation for the separated single sentences is completed to create a final text summary.

CN-Summ(Complex Networks-based Summarization) was proposed by (Antiqueira et al., 2009). Nodes relate to sentences in the graph or network representing one piece of text, while edges connect sentences that share common significant nouns. CN-Summ consists of 4 steps: 1) prepossessing (lemmatization). 2) resulting text is mapped to a network representation according to adjutancy and weight metrics of order n*n (n is no. of sentences/nodes) .3) compute different network measurements 4) the first m sentences are selected as summary sentences depending upon compression rate. Alguliev & Aliguliyev (2009) gave a new approach for unsupervised text summarization. That approach is focused on sentence clustering, where clustering is the technique of detecting interesting distributions and patterns within multidimensional data by establishing natural groupings or clusters based on some similarity metric. Here the researchers have proposed a new method to measure similarity named Normalized Google Distance (NGD) and to optimize criterion functions discrete differential evolution algorithm called as MDDE (Modified Discrete Differential Evolution) Algorithm is proposed.

Swarm Intelligence (SI) is the collective intelligence resulting from the collective behaviours of (unsophisticated) individuals interacting locally and with their environment, causing coherent functional global patterns to emerge. The primary computational parts of swarm intelligence are Particle Swarm Optimization (PSO), which is inspired by the social behaviour of bird flocking or fish schooling, and Ant Colony Optimization (ACO), which is inspired by the behaviour of ants. Binwahlan et al. (2009a) suggested a model based on PSO whose primary goal is to score sentences while focusing on dealing equally with text elements depending on their value. Combining three approaches: diversity-based method, fuzzy logic, and swarm-based methods (Binwahlan et al., 2010) can generate good summaries. Where diversity-based methods use to figure out similar sentences and get the most diverse sentence and concentrate on reducing redundancy, while

swarm-based methods are used to distinguish the most important and less important sentences then use fuzzy logic to tolerate redundancy, imprecise values and uncertainty, and this combination concentrates on the scoring techniques of sentences. While comparing two-approach Swarm-fuzzy based methods performs well than diversity-based methods here.

In (Mashechkin et al., 2011), the researchers had used LSA(Latent Semantic Analysis) for text summarization. The original text is reproduced as a matrix of terms and sentences. Text sentences are represented as vectors in the term space, and a matrix column represents each sentence. The resultant matrix is then subjected to latent semantic analysis to construct a representation of text sentences in the topic space, which is performed by applying one of the matrix factorizations (singular value decomposition (SVD)) to the text matrix. (Alguliev et al., 2011b) consider text summarization problem as integer linear programming problem while assuming that summarization is a task of finding a subset of sentences from the original text to represent important detail of the original text. That study focused on three characteristics (relevance, redundancy, and length) and tried to optimize that by particle swarm optimization algorithm (PSO) and branch and bound optimization algorithm. In extractive text summarization, sentence scoring is the most commonly used technique. The study (Ferreira et al., 2013) evaluated 15 algorithms available for sentence scoring based on quantitative and qualitative assessments. In conjunction with a graph-based ranking summarizer, Wikipedia is given by (Sankarasubramaniam et al., 2014a). It has given a unique concept by introducing incremental summarization property, where single and multi-document both can provide additional content in real-time. So, the users can first see the initial summary, and if willing to see other content, they can make a request.

Using a deep auto-encoder (AE) to calculate a feature space from the term-frequency (tf) input, (Yousefi-Azar & Hamey, 2017b) offer approaches for extractive query-oriented single-document summarization. Both local and global vocabularies are considered in experiments. The study shows the effect of adding slight random noise to local TF as the AE's input representation and propose the Ensemble Noisy Auto-Encoder as a collection of such noisy AEs (ENAE). Even though there is a lot of study on domain-based summarising in English and other languages, there is not much in Arabic due to a lack of knowledge bases. A hybrid, single-document text summarization approach is proposed in (Al-Radaideh & Bataineh, 2018a) paper (ASDKGA). The method uses domain expertise, statistical traits, and genetic algorithms to extract essential points from Arabic political documents. For domain or genre-specific summarization (such as for medical reports or specific news articles), feature engineering-based models have shown to be far more successful, as classifiers can be taught to recognize particular forms of information. For general text summary, these algorithms produce poor results. To overcome the issue, an entirely data-driven approach for automatic text summarization is given by (Sinha et al., 2018)

The most challenging difficulties are covering a wide range of topics and providing diversity in summary. New research based on clustering, optimization, and evolutionary algorithms has yielded promising results for text summarization. A two-stage sentences selection model based on clustering and optimization techniques, called COSUM, was proposed by Alguliyev et al. (2019b). The sentence set is clustered using the k - means algorithm in the first stage to discover all subjects in a text. An optimization approach is proposed in the second step for selecting significant sentences from clusters. The most crucial reason for the lack of domain shift approaches could be understanding different domain definitions in text summarization. For the text summarization task, (Wang et al., 2019) extended the traditional definition of the domain from categories to data sources. Then used, a multi-domain summary dataset to see how the distance between different domains affects neural summarization model performance. Traditional applications have a major flaw: they use high-dimensional, sparse data, making it impossible to gather relevant information. Word embedding is a neural network technique that produces a considerably smaller word representation than the classic Bag-of-Words (BOW) method. (Alami et al., 2019) has created a text summarization system based on word embeddings, and it showed that the Word2Vec representation outperforms the classic BOW representation. Another summarization approach using word embeddings was given by (Mohd et al., 2020). This study also used Word2Vec as a distributional semantic model that captures the semantics.

Current state-of-art systems produce generic summaries that are unrelated to the preferences and expectations of their users. CTRLsum (He, Kryscinski, et al., 2020), a unique framework for controlled summarizing, is presented to address that limitation. This system permits users to interact with the summary system via textual input in a collection of key phrases or descriptive prompts to influence several features of generated summaries. The majority of recent neural network summarization algorithms are either selection-based extraction or generation-based abstraction. (Xu & Durrett, 2020) introduced a neural model based on joint extraction and syntactic compression for single-document summarization. The model selects phrases from the document, identifies plausible compressions based on constituent parses, and rates those compressions using a neural model to construct the final summary. Four algorithms were proposed by (El-Kassas et al., 2020). The first algorithm uses the input document to create a new text graph model representation. The second and third algorithms look for sentences to include in the candidate summary in the built text graph. The fourth algorithm selects the most important sentences when the resulting candidate summary exceeds a user-specified limit. Automatic text summarization is an arduous effort for under-resourced languages like Hindi, and it is still an unsolved topic. Another problem with such languages is the lack of corpus and insufficient processing tools. For Hindi novels and stories, (Rani & Lobiyal, 2021) developed an extractive lexical knowledge-rich topic modelling text summarising approach in this study. The standard words-based similarity measure grants weight to most graph-based text summarising techniques. Belwal et al. (2021) offered a new graph-based summarization technique that considers the similarity between individual words and the sentences and the entire input text.

Table 1: Research survey on Extractive text summarization method

Citation	article	Model/methods/techniques applied	Dataset Used	Performance	Advantages/Pros	Disadvantages/Cons
(Garner, 1982)	Efficient Text Summarization: Costs and Benefits	Extractive text summarization	Dutch elm Disease (167 words- a single word)	Efficiency: proportion of number of judged-important ideas to total number of words in summary Proportion=0.2 to 0.12	-	-
(Rau et al., 1989)	Information extraction and text summarization using linguistic knowledge acquisition	Extractive text summarization	Dataset named as: Group Offers to Sweeten Warnaco Bid	-	SCISOR is robust and reliable extraction of information	Size of lexicon. (10,000 words) and system is limited.
(Bloom et al., 1994)	Automatic Analysis, Theme Generation, and Summarization of Machine-Readable Texts	Extractive text summarization	175-page article Entitle "United States of America.	-	robust and generally applicable to a wide variety of texts in many different environments,	In the absence of deep linguistic, it is not possible to build Intellectually satisfactory text summaries.
(Reimer et al., 1997)	Formal Model of Text Summarization Based on Condensation Operators of a Terminological Logic	Extractive text summarization	text	-	attempt was made to properly integrate the text summarization process to the formal reasoning mechanisms of the underlying knowledge representation language	Currently the summarization process considers only activity and connectivity patterns m the text knowledge base

(Aone et al., 1997)	A Scalable Summarization System Using Robust NLP	Extractive text summarization	text	-	The DimSum summarization system advances summarization technology by applying corpus-based statistical NLP techniques, robust information extraction, and readily available on-line resources	The results are not evaluated with human generate summaries.
(Salton et al., 1997)	Automatic text structuring and summarization	Multi document extractive text summation	encyclopaedia article 78 (Abortion)	4 factors are compared with respect to human extract: Optimistic evaluation, Pessimistic evaluation, Intersection, and Union.	Original text is broken into constituent pieces and combing these pieces according to their functionality. This text structure used for producing comprehensive summaries by automatic paragraph extraction -domain independent	Human extracts are dissimilar for a same article
(Lorch et al., 2001)	Effects of Headings on Text Summarization	Extractive text summarization	The experimental text was a revised version of a text on energy problems and solutions	-	More topics were included in the summaries of participants who read the text with headings than in the summaries of participants who read the text without headings	Headings interacted in recall, but not in the summarization task
(Neto et al., 2002)	Automatic Text Summarization Using a Machine Learning Approach	Extractive summarization on multi-document by machine learning approach	TIPSTER document base.	Comparison between automatically produced and manually produced summaries: Compression rate: 10% Precision/Recall: 30.79 ± 3.96	Works well with Naïve Bayes although the dataset consist of 33,658 documents within	C4.5 gives poor results as compared to naïve Bayes.
(Silber & McCoy, 2002)	Efficiently computed lexical chains as an intermediate representation for automatic text summarization	Multi document extractive text summarization	Randomly selected documents which are of length ranging from 2,247 to 26,320 words each.	Time complexity of algorithm: linear time (O(n) where n is nouns in document)	Feasible and robust algorithm with linear time complexity	Results are poor where proper nouns as proper nouns and domain specific are not found in WordNet.
(Nomoto & Matsumoto, 2003)	The diversity-based approach to open-domain text summarization	Unsupervised text summarization and open domain	5080 news articles in Japanese	Information centric evaluation	-Does not rely on human-made summaries but measuring the performance in terms of IR. -Diversity based summarizer is superior to tf-idf based summaries.	Diversity based summarizer perform poorly when exhibit high agreement.
(Yeh et al., 2005a)	Text summarization using a trainable summarizer and latent semantic analysis	modified corpus-based approach (MCBA) and LSA-based T.R.M. approach	100 documents in 5 sets of domains of politics were collected from New Taiwan Weekly	Compression rate: 30% MCBA: F-measure: 49% MCBA+GA: F-measure: 52% LSA+T.R.M F-measure: 44% (for single-document) F-measure: 40% (for corpus)	-LSA+T.R.M. performs better than keyword-based text summarization. -The results of MCBA+GA show that using GA in training is a good technique to deal with the circumstance where we are undecided about an appropriate combination of feature weights.	The performance of both approaches vary as compression rates are changes. For good result chose appropriate compression rate.
(Ye et al., 2007a)	Document concept lattice for text understanding and summarization	multi-document extractive summarization	DUC-2005 DUC-2006	-ROUGE-2 Recall = 7.17% & ROUGE-SU4 Recall = 13.16% in DUC-2005 -ROUGE-2 Recall = 8.99% and ROUGE-SU4	Proposed a document concept lattice (DCL) model and the corresponding algorithm for summarization	-

				Recall = 14.75% in DUC-2006		
(Ko & Seo, 2008)	An effective sentence-extraction technique using contextual information and statistical approaches for text summarization	Single/multi-document extractive summarization	841 news articles from KOrea Research and Development Information Centre (KORDIC) dataset	F1 score: 47.9 with 10% summary F1 score: 50.4 with 30%	To tackle the feature sparseness problem, it used contextual information (bi-gram pseudo phrase) and the combination method of statistical methodologies to increase the performance. -Independent on language	The performance changes according to no. of query words in multi-document
(Antiqueira et al., 2009)	A complex network approach to text summarization	graph-based extractive summarization	100 newspaper articles in Brazilian Portuguese	Average f-measure for all 15 version: 42%	Language & domain independent graph-based approach using complex networks (CN-Summ)	There are 15 versions of CN-Summ. Not an approach that combine all.
(alguliev & aliguliyev, 2009)	Evolutionary Algorithm for Extractive Text Summarization	Unsupervised extractive text summarization	DUC2001 & DUC2002	ROUGE-1: 0.45952 ROUGE-2: 0.19338 ROUGE-L: 0.21763	Proposed new method for measuring similarity between sentences - proposed a new algorithm method to optimize objective functions	-the results is shown using 3 criterion functions and among them criterion function 1 perform poorly.
(Binwahlan et al., 2009b)	Swarm Based Text Summarization	Swarm intelligence based extractive text summarization	DUC2002	The proposed model creates summaries which are 43% similar to the manually generated summaries	Employed PSO for sentence scoring based on their importance.	The experiment results are compared with manually human made summaries which are only 49% similar.
(Binwahlan et al., 2010)	Fuzzy swarm diversity hybrid model for text summarization	Extractive text summarization (Diversity-swarm-fuzzy)	First 100 documents from DUC 2002.	ROUGE-1: Fuzzy swarm Average F: 0.45524	Diversity-based and fuzzy methods reduce ambiguity. Swarm-based methods differentiate important sentences and less important sentences	Well accepted method as per results but not made for multi-document summarization.
(Mashechkin et al., 2011)	Automatic text summarization using latent semantic analysis	LSA based extractive text summarization.	DUC2001 & DUC2002	ROUGE-2: 0.19260 ROUGE-L: 0.37229	A new generic summarization method is proposed that uses nonnegative matrix factorization to estimate sentence relevance -In comparison to the singular value decomposition position, the proposed estimation of semantic characteristics (topics) weight better preserves the internal structure of the text.	The optimal numbers of topics for each summarization problem is empirically obtained.
(Alguliev et al., 2011a)	MCMR: Maximum coverage and minimum redundant text summarization model	PSO, unsupervised based multi-document extractive text summarization	DUC2005 & DUC2007	ROUGE-2 on DUC-2005: B&B: 0.0790 PSO: 0.0754 ROUGE-2 on DUC-2007: B&B: 0.1221 PSO: 0.1165	Good for both single document and multi-document text summarization	-Results are directly depending upon optimization algorithm -parameter value of objective function influences performance of summarization
(Sankarasubramaniam et al., 2014b)	Text summarization using Wikipedia	Wikipedia-based graph-based multi-document summarization algorithm	DUC 2002	ROUGE-1: 0.46 ROUGE-2: 0.23 Precision: 0.57 Recall:0.50 F-measure: 0.51	To extract salient topics from text document it proposed algorithm using Wikipedia with graph-based summarization -concentrated on personalization and query-focusing of summaries	Among results of evaluation Sentences chosen uniformly at random performs poor.

(Gupta & Kaur, 2016)	A Novel Hybrid Text Summarization System for Punjabi Text	Extractive text summarization (concept, statistical, location, numeric data, and linguistic-based features)	Standard Unicode-based Punjabi Corpus and Punjabi news corpus AZIT with each having 150 documents.	Overall score ROUGE-2: 0.85 F-score with 50% compression rate: 0.86	Question-answering summary also tried here where proposed method gives 80-90% accuracy.	Language dependent
(Yousefi-Azar & Hamey, 2017a)	Text summarization using unsupervised deep learning	query-oriented single-document extractive text summarization using deep learning	Summarization and Keyword Extraction from Emails (SKE) BC3 from British Columbia University	Improvement of ROUGE-2 recall on average 11.2%.	Auto-Encoders produces a rich concept vector for an entire sentence from a bag-of-words input.	-Better to use semi-supervised technique because shortage of manually annotated data -the computational cost of training and the necessity to appropriately tune the training hyper parameters
(Sinha et al., n.d.)	Extractive Text Summarization using Neural Networks	Extractive text summarization using Neural Networks	DUC 2002	ROUGE-1: 55.1 ROUGE-2: 22.6	A data-driven technique that leverages a basic feedforward neural network achieves good performance while compared with state-of-the-art systems while being both implementationally and computationally light.	assumed that summary length to be generated should be less than specified length.
(Al-Radaideh & Bataineh, 2018b)	A Hybrid Approach for Arabic Text Summarization Using Domain Knowledge and Genetic Algorithms	Extractive text summarization (domain knowledge, genetic algorithm)	KALIMAT corpus and Essex Arabic Summaries Corpus (EASC)	Average F-measure: 0.605 with compression ratio of 40%.	Domain knowledge enhances results of the proposed method	-Language dependent -training done with only 500 keywords which lies into 4 categories.
(Alguliyev et al., 2019a)	COSUM: Text summarization based on clustering and optimization	Extractive text summarization using clustering and optimization	DUC2001 & DUC2002	DUC2001: ROUGE-1: 0.4727 ROUGE-2: 0.2012 DUC2002: ROUGE-1: 0.4908 ROUGE-2: 0.2309	The model additionally limits the length of sentences selected in the candidate summary to ensure readability.	Sometimes performance is conceded while compared to other methods
(Wang et al., 2019)	Exploring Domain Shift in Extractive Text Summarization	Extractive multi-domain text summarization	MULTI-SUM dataset CNN/DailyMail	-	Proposed a multi-domain dataset MULTI-SUM -Given four learning schemes as a preliminary investigation into the properties of various learning strategies when dealing with multi-domain summarising jobs.	Excellent results in cross-dataset (CNN/DailyMail) but results declined in in-domain and out-of-domain settings.
(Xu & Durrett, 2020)	Neural Extractive Text Summarization with Syntactic Compression	Extractive text summarization with deep neural network	New York Times corpus CNN/DailyMail	CNN: ROUGE-1: 32.7 ROUGE-2: 12.2 ROUGE-L: 29.0	A sentence extraction model is combined with a compression classifier to determine whether or not syntax-derived compression choices should be deleted for each sentence.	Best performance on CNN but performance somewhere declined for another dataset while compared to baseline methods.
(Mohd et al., 2020)	Text document summarization using word embedding	Extractive text summarization using distributional semantics (Word2vec)	DUC2007	F-score for 25% summary length: ROUGE-1: 33% ROUGE-2: 7% ROUGE-L: 20% ROUGE-SU4: 13%	summarization technique based on the distributional hypothesis to capture the semantics of better than the baselines.	Proposed distributed semantic model is computationally expensive and time consuming. Results are given on different % of summary length, where sometimes recall values are low than baseline systems.

(He, Kryściński, et al., 2020)	CTRLsum: Towards Generic Controllable Text Summarization	Extractive text summarization (keyword-based model)	CNN/DailyMail arXiv scientific papers BIGPATENT patent articles	CNN/DailyMail: ROUGE-1: 48.75 ROUGE-2: 25.98 ROUGE-L: 45.42 arXiv: ROUGE-1: 47.58 ROUGE-2: 18.33 ROUGE-L: 42.79 Success rate: 97.6 Factual Correctness: 99.0	proposed a generic framework to perform multi-aspect controllable summarization.	During training, the model is conditioned on keywords to predict summaries but one keyword can be fall into more than one aspect.
(El-Kassas et al., 2020)	EdgeSumm: Graph-based framework for automatic text summarization	Extractive single-document text summarization using Graph-based approach	DUC2001 & DUC2002	Improvement of 1.2% and 4.7% in ROUGE-1 and ROUGE-L respectively for DUC2002 dataset.	-EdgeSumm integrates a number of extractive ATS methods (graph-based, statistical-based, semantic-based, and centrality-based methods) to take use of their strengths while minimising their weaknesses. -The use of hyphens removal and synonyms substitution tasks on the performance of the generated summaries is investigated.	-Results are very promising but not extended for multi-document summarization -not domain specific system -Language dependent
(Rani & Lobiyal, 2021)	An extractive text summarization approach using tagged-LDA based topic modelling	Extractive text summarization based on LDA-model	114 Hindi novels including short stories from 'Munshi Premchand's stories' blog	Perform preferable than the baseline algorithms for 10% - 30% compression ratios and given evaluation metrics	-Since there was no corpus of Hindi novels and stories, it built one corpus. -four distinct sentence weighting scheme-based variants are derived by manipulating the proposed system	-Semantic features are not taken into consideration -Results are somewhere improved and somewhere declined while compared to baseline systems for different-different compression rates
(Belwal et al., 2021)	A new graph-based extractive text summarization using keywords or topic modelling	Extractive summarization using Graph-based and Topic-based techniques	Opinosis dataset CNN/ DailyMail	CNN/DailyMail: ROUGE-1: 0.428 ROUGE-2: 0.201 ROUGE-L: 0.392 Opinosis dataset: ROUGE-1: 0.271 ROUGE-2: 0.084 ROUGE-L: 0.161	Added an extra parameter that calculates the similarity of the nodes to the entire content. Used topic modelling to address the issue of redundancy associated with existing summarising methods	-
(Alami, Mallahi, et al., 2021)	Hybrid method for text summarization based on statistical and semantic treatment	Extractive text summarization (statistical and semantic methods)	The dataset consists of 153 Arabic articles taken from two Arabic newspapers and the Arabic version of Wikipedia. The	Dataset-1: F1-measure: 59.47 with 40% compression rate Dataset-2: F1-measure:60.79 40% compression rate	Remove redundancy with using MMR (Maximal Marginal Relevance) Improve sentence score by other statistical features	Language dependent

3.2 Abstractive text summarization

The competitions DUC-22003 and DUC-2004 had standardized the task of Abstractive text summarization, in which news articles from various fields with multiple reference summaries per article generated by humans are used as datasets. The TOPIARY system (Zajic et al., 2004) stood the best performing technique. Some noticeable work was submitted by Banko et al. (2000) with phrase-table based machine translation techniques and (Woodsend et al., 2010) with quasi-synchronous grammar techniques. Table-2 shows a detailed survey on Abstractive text summarization with a particular research paper with the dataset used, the system's accuracy, and its pros and cons.After that, deep learning was introduced as a viable alternative to many NLP problems. Text is a sequence of words where sequence-to-sequence models can entertain input and output sequences. With the apparent similarities Machine translation (MT) problem may be mapped to text summarization despite that abstractive summarization is very different from it. MT is lossless while summarization is lossy in the manner, and MT is a one-to-one word-level mapping between source and target, but that mapping is less in summarization.

In (Rush et al., 2015), the researchers had used convolution models to encode the input and context-sensitive feed-forward network with attentional mechanism and showed better results for Gigaword and DUC datasets. (Chen, 2015) have produced a sizeable Chinese dataset for short text summarization (LCSTS), which has given good results on their dataset while using RNN architecture at both encoder and decoder sides. Beyond RNN architecture at both encoder and decoder sides, (Nallapati et al., 2016) captured keywords, modelled unseen or rare words, and captured the document's hierarchy using a hierarchical attention mechanism. The authors have also tried to analyse the quality of the output summary. In that case, somewhere models perform well and somewhere poor compared to others. Human's summaries are more abstractive naturally because they use some inherent structures while writing summaries. The deterministic transformation in a discriminative model(RNN) used by Nallapati et al. (2016) limits the representation of latent structure information. After that, Miao & Blunsom (2016) gave a generative model to capture the latent structure, but they did not consider recurrent dependencies in their generative model. The authors of (Li et al., 2017) tried to find some common structures such as "what", "what happened", "who actioned what" from the source and proposed a deep recurrent generative model for modelling latent structure.

AMR (Abstract Meaning Representation) was firstly introduced by Banarescu et al. (2013). AMR targets fetching the meaning of the text by giving a special-meaning representation to the source text. AMR attempts to capture "who is doing what to whom". The work of Liu et al. (2015).' s includes AMR, but they did not use it at the abstraction level, so their work is limited to extractive summarization only. Also, the approach aims to generate a summary from a story. Producing a single graph assumes that all the important sentences can be extracted from a single subgraph. Difficulties arise when information is spread out all over. So, Doha et al. (2017a) worked on multiple summary graphs and explored problems with existing evaluation methods and datasets while doing abstractive summarization.

Combining the advantages of extractive and abstractive and curing the disadvantages (Song et al., 2019b) had implemented a model named ATSDL (ATS using DL). This model uses a phrase extraction method called MOSP to extract key phrases from the original text after that, learns the collocation of phrases. Following training, the model will generate a phrase sequence that satisfies the syntactic structure criteria. Furthermore, we leverage phrase location information to overcome

the problem of unusual terms, which practically all abstractive models would face. Regarding sequence-to-sequence models, RNN is not most used because it tends to low-efficiency problems as they rely on the previous step when training, and it must preserve the hidden state of the whole past, thus not able to perform parallel operations. To overcome these problems, Zhang et al. (2019) proposed a sequence-to-sequence model based on CNN to create the representation of source text. As we know, traditional CNN can only encode with fixed size contexts of inputs, but in this study, they increase the compelling text by stacking CNN layers over each other. The length of the sequence under consideration may thus be readily regulated, and each component of the sequence can be computed in parallel. More commonly, abstractive summarization problems are that the generated summaries are frequently incompatible with the source content in terms of semantics. WEI et al. (2018) offer a regularisation strategy for the sequence-to-sequence model in this research, and we use what the model has learnt to regularise the learning objective to mitigate this problem's influence.

Until now, the model discussed does not consider whether the summaries are factually consistent with source documents. Kryściński et al. (2019a) present a model-based technique for evaluating factual consistency and detecting conflicts between source documents and the output summary that is a weakly supervised model. The steps of these models are:

- Determine whether sentences are factually consistent after being transformed,
- Find a span in the source documents to validate the consistency prediction, and
- Find an inconsistent span in the summary phrase if one exists.

Another notable work is done in the sequence-to-sequence encoder and decoder approach by Kryściński et al. (2020). That study makes two main contributions. First, separate extraction and generation at decoder part. The contextual network is standalone for extraction, and a language model generates paraphrases. Second, optimizing the n-gram overlap while encouraging abstraction with ground-truth summaries.

Wang et al. (2020) provide a unique Generative Adversarial Network (GAN) for Abstractive Text Summarization with a multitask constraint in this research (PGAN-ATSMT). Through adversarial learning, this model simultaneously trains a generator G and a discriminator D. The sequence-to-sequence architecture is the backbone of the generative model G, which takes the source document as input and generates the summary. The model uses a language model to implement D instead of a binary classifier as the discriminative model D, and the output of the language model is used as the reward to steer the generative model. A minimax two-player game was used to optimize the generative model G and the discriminative model D. Extended work on the GAN network is done by Yang et al. (2021). They present a new Hierarchical Human-like deep neural network for ATS (HH-ATS), influenced by how humans interpret articles and produce summaries. HH-ATS comprises three main components (i.e., a knowledge-aware hierarchical attention module, a multitask learning module, and a dual discriminator generative adversarial network) that reflect the three phases of human reading cognition (i.e., rough reading, active reading, and post-editing).

Table 2: Research survey on Abstractive text summarization method

Citation	article	Model/methods/techniques applied	Dataset Used	Performance	Advantages/Pros	Disadvantages/Cons
(Nallapati et al., 2016)	Abstractive text summarization using sequence-to-sequence RNNs and beyond	Abstractive (Sequence-to-sequence encoder-decoder with RNN)	Gigaword corpus, DUC, CNN/daily mail corpus	ROUGE-1: 35.46 ROUGE-2:13.30 ROUGE-L:32.65 Qualitative evaluation: a few high quality and poor-quality output	-Performs a little bit well as compared to state-of-the-art -Propose new dataset for multiple sentence summary	-Worked for single sentence output summaries -capturing the meaning of complex sentence is weakness for this model
(Li et al., 2017)	Deep recurrent generative decoder for abstractive text summarization	Abstractive (Sequence-to-sequence encoder-decoder with deep recurrent generative decoder (DRGN))	Gigaword corpus, DUC, LCSTS	ROUGE-1: 36.71 ROUGE-2:24.00 ROUGE-L:34.10	Implement deep recurrent generative model to capture latent structure information.	Worked for single sentence output summaries
(Dohare et al., 2017b)	Text Summarization using Abstract Meaning Representation	Abstractive summarization with AMR.	CNN, DailyMail	ROUGE-1: 39.53 ROUGE-2:17.28 ROUGE-L:36.38	-Suggested a full pipeline for summarization with AMR -proven that ROGUE can't be used for evaluating the abstractive summaries -a novel approach for extracting multiple summary graphs	Not a lot of work has been done to extract AMR graphs for summaries
(Song et al., 2019a)	Abstractive text summarization using LSTM-CNN based deep learning	Abstractive (CNN-LSTM)	CNN, DailyMail dataset	ROUGE-1: 34.9 ROUGE-2:17.8	-Implement ATSDL (ATS using Deep Learning) system that Combines CNN and LSTM for better performance -solve problem of rare words	-Training of deep learning is very time-consuming process. -ROUGE can't evaluate the quality of summary effectively.
(Zhang et al., 2019)	Abstract text summarization with a convolutional seq2seq model	Abstractive (CNN)	Gigaword corpus, DUC, CNN/daily mail corpus	ROUGE-1: 42.04 ROUGE-2: 19.77 ROUGE-L: 39.42	-equip the CNN model with GLU and residual connections. -hierarchical attention mechanism to generate the keywords and the key sentences simultaneously. -a copying mechanism to extract out-of-vocabulary words from source text.	In practise, adding more sentences has a negative impact on performance, which we explain to the fact that the latter sentences are unrelated to the summary.
(Kryściński et al., 2019b)	Evaluating the Factual Consistency of Abstractive Text Summarization	Abstractive	CNN/Daily Mail dataset	Accuracy: 74.15 F1-Score: 0.5106	-Implemented factual consistency checking model (FactCC).	-common-sense mistakes made by summarization models. - stemming errors from dependencies between different sentences within the summary
(WEI et al., 2018)	Regularizing output distribution of abstractive Chinese social media text summarization for improved semantic consistency	Abstractive	Large-Scale Chinese Short Text Summarization Dataset (LCSTS)	ROUGE-1: 36.2 ROUGE-2:24.3 ROUGE-L: 33.8 Accuracy of Human Evaluation approach: 53.6%	- suggest a method for regularising the output word distribution so that semantic inconsistency in the training data, such as terms not linked to the source content, is underrepresented in the model. -Proposed a simple human evaluation approach for determining the generated summary's semantic compatibility with the original information.	Language dependent (Chinese dataset)

(Kryściński et al., 2018)	Improving abstraction in text summarization	Abstractive (Sequence-to-sequence encoder-decoder)	CNN/Daily Mail dataset	ROUGE-1: 40.19 ROUGE-2:17.38 ROUGE-L: 37.52 Qualitative evaluation: Readability: 6.76/10 Relevance: 6.73/10	-Separate the decoder into a contextual network and a pretrained language model. -Optimized metric through policy learning	Matches score when compared to the state-of-the-art in relevance but slightly low in terms of readability of summaries.
(Yang, Wang, et al., 2020)	Plausibility-promoting generative adversarial network for abstractive text summarization with multi-task constraint	Abstractive (GAN)	Gigaword corpus, CNN/daily mail corpus	ROUGE-1: 40.19 ROUGE-2:17.38 ROUGE-L: 37.52 Perplexity:10.21 Qualitative evaluation: Relevance: 3.29/5 Fluency:3.38/5	-Developed PGAN-ATSMT, an adversarial framework for abstractive text summarization with multi-task constraint. -PGAN-ATSMT jointly trains the task of abstractive text summarization and two other related tasks: text classification and syntax generation.	-External Common-sense knowledge from the language is missing. -Time consuming as deep learning algorithm
(Yang, Li, et al., 2020)	Hierarchical Human-Like Deep Neural Networks for Abstractive Text Summarization	Abstractive (GAN)	Gigaword corpus, CNN/daily mail corpus	ROUGE-1: 43.16 ROUGE-2:20.32 ROUGE-L: 39.14 Qualitative evaluation: Informativeness: 3.41/5 Fluency:3.32/5	HH-ATS extends the seq2seq framework by replicating the process of writing a summary for a piece of text by humans.	To evaluate human judgement better it is required to explore different automatic metrics

3.3 Hybrid text summarization:

When discussing very precious approaches of Automatic text summarization that are Extractive and Abstractive, both come with their pros and cons. Extractive summarization is comparatively easier to implement than abstractive summarization, but extractive summarization is not as efficient as user perception. Combining these methods by strengthening their pros and weakening their cons leads to hybrid methods for text summarization.

Experiments were done on summarization until 1990 were focused on just extracting (reproduced) the summaries from original text rather than abstracting (newly generated).SUMMRIST system (Hovy & Lin, 1996) was developed with the help of NLP techniques. We can develop a multi-lingual summarizer by modifying some parts of the structure.

Semantic and statistical features combine extracting and abstracting. The authors of Bhat et al. (2018) used emotions of the text as a semantic feature. Emotions play a significant role in defining the user's emotional affinity, so lines with implicit emotional content are crucial to the writer and should be included in the summary. The extracted summary is then put into the Novel language generator, a hybrid summarizer that combines WordNet, Lesk algorithm, and POS to transform extractive summary into an abstractive summary. Table-3 shows a detailed survey on hybrid text summarization with a particular research paper with the dataset used, accuracy of the system and its pros and cons.

Table 3. Research survey on hybrid text summarization method

Citation	article	Model/methods/techniques applied	Dataset Used	Performance	Advantages/Pros	Disadvantages/Cons
(Hovy & Lin, n.d.)	Automated text summarization and the SUMMARIST system	Hybrid text summarization	Text	SUMMARIST scores 30% higher than human abstracts	By avoiding some language-specific methods in SUMMARIST it is possible to create multi-lingual summarizer easily.	Lack of good training data in variety of domains
(Bhat et al., 2018)	SumItUp: A Hybrid Single-Document Text Summarizer	Hybrid Single-Document Text summarization	DUC-2007 with 15 news articles	Precision:0.4 to 0.8 Compression rate: 35%	-hybrid approach to a single-document summarization combining semantic and statistical features -emotional feature	Not proper evaluation is done. The results are compared to MS word (no ROUGE metrics)

4 ATS System Evaluation and Evaluation Programs

There have been many efforts to solve summary evaluation issues in the past two decades. NIST (National Institute of Standards and Technology) leads the effort by organizing the DUC and TAC challenges. (Huang et al., 2010) has given four pillars that should be considered to generate summaries:

- Information Coverage
- Information Significance
- Information Redundancy
- Text Coherence

In the discipline of automatic text summarization, evaluating the summary is a critical task. Evaluating the summary and increasing the development of reusable resources and infrastructure aids in comparing and replicating findings, adding competition to improve the outcomes. However, carefully evaluating many texts to acquire an unbiased perspective is impossible. As a result, accurate automatic evaluation measures are required for quick and consistent evaluation. It is difficult for people to recognize what information should be included in a summary; therefore, evaluating it is difficult. Information changes depending on the summary's purpose, and mechanically capturing this information is a challenging undertaking (Gambhir & Gupta, 2017). Evaluation of the ATS system is given in fig.7 below:

a) **Extrinsic Evaluation**: An extrinsic evaluation looks at how it influences the accomplishment of another task (Text classification, Information retrieval, Question answering). i.e., a summary is termed a good summary if it provides help to other tasks. Extrinsic evaluations have looked at how summarization affects tasks such as relevance assessment, reading comprehension, etc.

- Relevance evaluation: Various methods are used to analyse a topic's relevance in the summary or the original material.

- Reading comprehension: After reading the summary, it assesses whether it is possible to answer multiple-choice assessments.

b) **Intrinsic Evaluation**: An Intrinsic evaluation looks at the summarization system on its own. The coherence and informativeness of summaries have been the focus of intrinsic evaluations. Evaluations based on comparisons with the model summary/summaries and evaluations based on comparisons with the source document are the two types of intrinsic techniques (Steinberger & Ježek, 2009).

It assesses the quality of a summary by comparing the coverage of a machine-generated and a human-generated summary. The two most significant aspects of judging a summary are its quality and informativeness. A summary's informativeness is usually assessed by comparing it to a human-made summary, such as a reference summary. There is also faithfulness to the source paradigm, which examines if the summary contains the same or similar material as the original document. This method has a flaw: how it can be determined which concepts in the document are relevant and which are not?

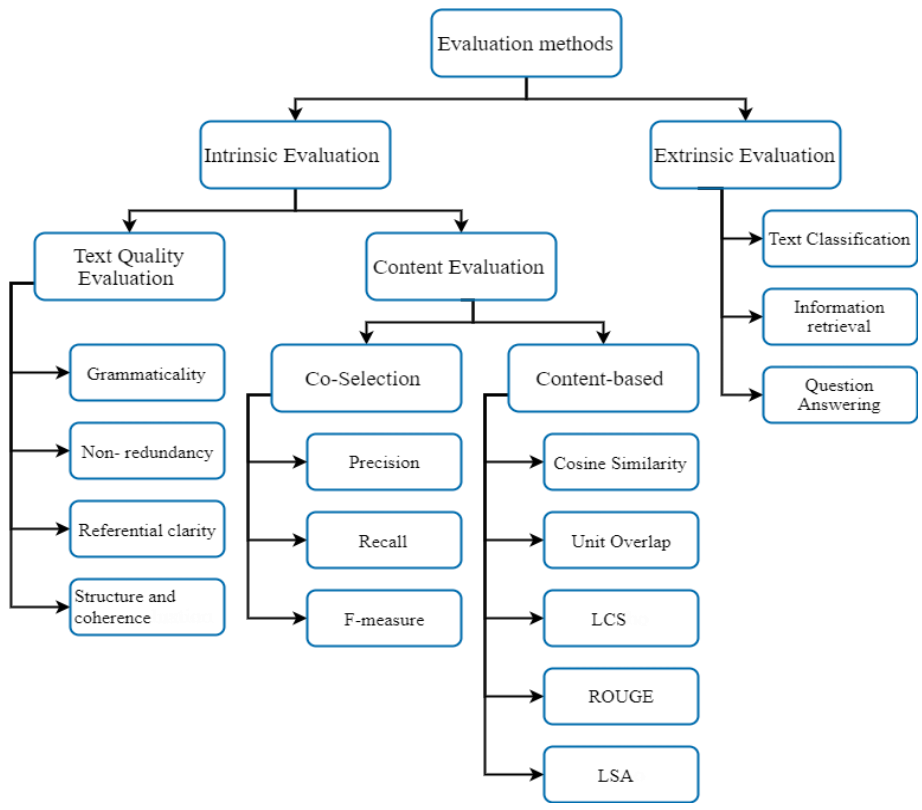


Fig. 7: The evaluation Techniques for Automatic Text Summarization

4.1 Content Evaluation

- ***Co-selection*:** Only identical sentences can be used in co-selection measures. It ignores the reality that even though two sentences are written differently, they can contain the same information. In addition, summaries provided by two separate authors rarely contain similar sentences. Co-selection can be calculated by precision, recall and F-measure.

a. *Precision:* Precision is the fraction of relevant instances among the retrieved instances.

$$Precision = \frac{|\{relevant\ documents\} \cap \{retrived\ documents\}|}{|\{retrived\ documents\}|} \quad (1)$$

$$Precision = \frac{True\ Positive}{True\ Positive + False\ Positive} \quad (2)$$

b. *Recall:* Recall is the fraction of relevant instances that were retrieved.

$$Recall = \frac{|\{relevant\ documents\} \cap \{retrived\ documents\}|}{|\{relevant\ documents\}|} \quad (3)$$

$$Precision = \frac{True\ Positive}{True\ Positive + False\ Negative} \quad (4)$$

c. *F-measure:* It is computed by combining recall and precision.

$$F - measure = 2 * \frac{Precision * Recall}{Precision + Recall} \quad (5)$$

- ***Content-based:*** Drawbacks of co-selection methods are handled by content-based methods.

a. *Cosine Similarity:* Cosine Similarity can be measured as,

$$\cos(X, Y) = \frac{\sum_i x_i\, y_i}{\sqrt{\sum_i (x_i)^2}\sqrt{\sum_i (y_i)^2}} \quad (6)$$

Where, X and Y are representations of a system summary and its reference document based on the vector space model.

b. *Unit Overlap:* Unit Overlap can be calculated as,

$$\text{overlap}(X, Y) = \frac{\|X \cap Y\|}{\|X\| + \|Y\| - \|X \cap Y\|} \quad (7)$$

Where, X and Y are representations based on sets of words or lemmas. $\|X\|$ is the size of set X.

c. *Longest Common Subsequence (LCS):* the LCS formula is defined as shown in equation (8),

$$\text{LCS}(X, Y) = \frac{length(X) + length(Y) - edit_{di}(X,Y)}{2} \quad (8)$$

Where, X and Y are representations based on sequences of words or lemmas, LCS (X, Y) is the length of the longest common subsequence between X and Y, length(X) is the length of the string X, and $edit_{di}$(X, Y) is the edit distance of X and Y.

d. *ROUGE (Recall-Oriented Understudy for Gisting Evaluation):* It was firstly introduced by C. Lin & Rey (2001). It contains measures for automatically determining the quality of a summary by comparing it to other (ideal) summaries generated by people. The measures count the number of overlapping units such as n-grams, word sequences, and word pairs between the computer-generated summaries to be evaluated and the ideal summaries written by humans. ROUGE includes five measures like ROUGE-N, ROUGE-L, ROUGE-W, ROUGE-S and ROUGE-SU.

- ROUGE-N counts the number of N-gram units shared by a given summary and a group of reference summaries, where N is the length of the N-gram. i.e., ROUGE-1 for unigrams and ROUGE-2 for bi-grams.
- ROUGE-L calculates the LCS (Longest Common Subsequence) statistic. LCS is the maximum size of a common subsequence for two given sequences, X and Y. ROUGE-L estimates the ratio of the LCS of two summaries to the LCS of the reference summary.
- ROUGE-W is the weighted longest common subsequence metric. It is a step forward from the basic LCS strategy. ROUGE-W prefers LCS with successive common units. Dynamic programming can be used to compute it efficiently.
- ROUGE-S (Skip-Bigram co-occurrence statistics) calculates the percentage of skip bigrams shared between a single summary and a group of reference summaries. The skip bigrams are word pairs in the sentence order with random gaps.
- ROUGE-SU is a weighted average of ROUGE-S and ROUGE-1 that expands ROUGE-S to include a unigram counting unit. This is a step forward from ROUGE-S.

e. *LSA-based method:* This method was developed by Steinberger & Ježek (2009). If there are m terms and n sentences in the document, we will obtain an m*n matrix A. The next step is to apply Singular Value Decomposition (SVD) to matrix A. The SVD of an m*n matrix A is defined as given in equation (9):

$$A = U\Sigma VT \quad (9)$$

In terms of NLP, SVD (Singular Value Decomposition) is used to generate the document's latent semantic structure, represented by matrix A: that is, a breakdown of the original document into r linearly-independent basis vectors that express the document's primary 'Topics'. SVD can record interrelationships between terms, allowing concepts and sentences to be clustered on a 'semantic' rather than a 'word' basis.

4.2 Text Coherence or Quality Evaluation:

a. Grammaticality: The text should not contain non-textual items (i.e., markers), punctuation errors or incorrect words.
b. Non-redundancy: The text should not contain redundant information.
c. Reference clarity: The nouns and pronouns should be referred to in summary. For example, the pronoun he has to mean somebody in the context of the summary.
d. Coherence and structure: The summary should have good structure, and the sentences should be coherent.

The linguistic characteristics of the summary are properly considered here. Non-redundancy, focus, grammaticality, referential clarity, and structure and coherence are five questions based on linguistic quality used in DUC (Document Understanding Conference) and TAC (Text Analysis Conference) conferences to evaluate summaries not needed to be reviewed against the reference summary. Expert human assessors manually score the summary based on its quality, awarding a score to the summary according to a five-point scale (Gambhir & Gupta, 2017).

The text quality of a summary can also be checked by examining several readability variables. Text quality is analysed using various criteria such as vocabulary, syntax, and discourse to estimate a correlation between these characteristics and previously acquired human readability ratings. Unigrams represent vocabulary, while the average number of verbs or nouns represent syntax.

4.3 Automatic Text Summarization Evaluation Programs

SUMMAC (TIPSTER Text Summarization Evaluation) was the first conference where automatic summarization systems were reviewed, and it was held at the end of the 1990s, where text summaries were assessed using two extrinsic and intrinsic criteria. DUC (Document Understanding Conferences), which took place every year from 2001 to 2007, is another notable conference for text summarizing. Initially, activities at DUC conferences like DUC 2001 and DUC 2002 featured generic summarizing of single and multiple documents, which was later expanded to include a query-based summary of multiple documents in DUC 2003. Topic-based single and multi-document cross-lingual summaries were assessed in DUC 2004. Multi-document, query-based summaries were examined in DUC 2005, and DUC 2006, whilst multi-document, update, query-based summaries were evaluated in DUC 2007. However, in 2007, DUC conferences were no longer held because they were absorbed into the Text Analysis Conference (TAC), which featured summarization sessions. TAC is a series of evaluation workshops designed to promote research in the domains of Natural Language Processing and related fields. The TAC QA program arose from the TREC QA program. The Summarization track aids in the development of methods for producing concise, cohesive text summaries. Every year TAC workshops have been held since 2008.

5. Frequently used Dataset for ATS

There are applications of ATS systems that are widely spread worldwide and know the available data globally. So, to perform the text summarization task essential thing is the data. Not all data can be directed feed to the system. It required prepossessing and other treatments. The machine learning-based approaches need a huge training dataset with ideal summaries to train the model. Also, the ideal or sample Dataset is needed to evaluate a particular ATS system. That sample data is manually generated or created by human researchers. The list of the Dataset available for the ATS task is very long. A very few datasets are given below:

- ***DUC:*** The National Institute of Standards and Technology (NIST) provides these datasets, the most prevalent and widely used datasets in text summarization research. The DUC corpora were distributed as part of the DUC conference's summarizing shared work. The most recent DUC challenge took place in 2007. Datasets for DUC 2001 through DUC 2007 are available on the DUC website.
- ***Text Analysis Conference (TAC) Datasets:*** DUC was added to the TAC as a summary track in 2008. To gain access to the TAC datasets, you must first fill out the application forms available on the TAC website.
- ***Gigaword:*** Created by (Rush et al., 2015), Headline-generation on a corpus of article pairs from Gigaword consisting of around 4 million articles in the English language.
- ***LCTCS:*** Created by Chen (2015), the LCSTS Dataset was constructed from the Chinese microblogging website Sina Weibo. It consists of over 2 million real Chinese short texts with short summaries given by the author of each text. Requires application in the Chinese language.
- ***wikiHow:*** Created by Koupaee & Wang (2018), the WikiHow Dataset contains article and summary pairs extracted and constructed from an online knowledge base written by different human authors in the English language. There are two features: - text: wikiHow answers texts. - headline: bold lines as summary.
- ***CNN:*** CNN/DailyMail non-anonymized summarization dataset. The CNN / Daily Mail Dataset is an English-language dataset containing just over 300k unique news articles written by journalists at CNN and the Daily Mail. The current version supports both extractive and abstractive summarization, though the original version was created for machine-reading and comprehension and abstractive question answering.

6. Application, Challenges and future scope

6.1 Applications of ATS

There are numerous uses for automatic text summarizing. As we see how text summarization is divided into many more categories. All these categories further lead us to treasure of ATS's applications. This subsection includes some of the applications of ATS system. Table 4 shows the research survey on application of ATS system. The table includes article name, the method & the dataset used in a particular article, the performance of the system proposed in that particular research study and advantages & disadvantages of that article.

Here are a few examples:

- Improving the performance of classic IR and IE systems (using a summarization system in conjunction with a Question-Answering (QA) system); (De Tre et al., 2014) (S. Liu et al., 2012) (Perea-Ortega et al., 2013)
- News Summarization and Newswire generation (Tomek,1998) (Bouras & Tsogkas, 2010)
- Rich Site Summary (RSS) feed summarization (Zhan et al., 2009)
- Blog Summarization (Y. H. Hu et al., 2017)
- Tweet Summarization (Chakraborty et al., 2019)
- Web page Summarization (Shen et al., 2007)
- Email and email thread Summarization (Muresan et al., 2001)
- Report Summarization for business men, politicians, researchers, etc. (Lloret et al., 2013)
- Meeting Summarization.
- Biographical extracts
- Legal Document Summarization (Farzindar & Lapalme, 2004)
- Books Summarization (Mihalcea & Ceylan, 2007)
- Use of Summarization in medical field (Feblowitz et al., 2011)(Ramesh et al., 2015)

Table 4: Research survey on Application of ATS System

Citation	Article	Model/methods applied	Dataset Used	Performance	Advantages/Pros	Disadvantages/Cons
(Tomek,1998)	A Robust Practical Text Summarization	Multi-Document, Extractive Text summarization	New York times online news	Length of summaries: The summaries are only 5% to 10% of original text so it can be quickly read and understood.	- The algorithm is very robust that efficiently process a large range of documents, domain independent and can be easily accepted by most of the European languages. - It can work in two modes: Generic and Topical -Worked on DMS (Discourse Macro Structure) to conquer the shortcomings of sentence-based summarization by working on paragraph level	Quality of summaries not that much good. It can be improved by additional paragraph scoring function.
(Muresan et al., 2001)	Combining linguistic and machine learning techniques for email summarization	Extractive text summarization with machine learning	Emails	Precision: 83% Recall:85.7%	Linguistic Knowledge Enhances Machine Learning	Deep linguistic knowledge is required.
(Mckeown et al., 2002)	Tracking and summarizing news on a daily basis with Columbia's Newsblaster	Multi-document summarization with extractive approach	News sites	-	These research achievements are incorporated into "Newsblaster"	Personalization of Newsblaster and restricting it to user preferred topics or questions is still having to be done
(Farzindar & Lapalme, 2004)	Legal Text Summarization by Exploration of the Thematic Structure and Argumentative Roles	Extractive Text summarization	Corpus contains 3500 judgements of Federal court of Canada	Preliminary results are very promising. -F-measures: 0.935 on average for all stages	-Summary presented as table style -focused on many categories of judgements i.e., Copyright, Air low, human rights etc.	The System is not properly for many other remaining categories of judgements
(Mihalcea & Ceylan, 2007)	Explorations in automatic book summarization	Extractive text summarization with TEXTRANK approach	gold standard" data set of 50 books	F-Measure: 0.404	it introduced a new summarization benchmark, specifically targeting the evaluation of systems for book summarization	Exhaustive method when we have short length book
(Ling et al., 2007)	Generating gene summaries from biomedical literature: A study of semi-structured summarization	Multi-Document, Extractive Text summarization	test set with 20 genes	ROUGE-N performs better than existing methods	Semi- structured summaries are generated which consist of sentences regarding specific semantic aspects of a gene.	- High quality of data required. (Data dependent) -Redundant information in generated summary
(Shen et al., 2007)	Noise reduction through summarization for Web-page classification	Extractive Text summarization	2 million Web pages crawled from the LookSmart Web directory	F-measures for hybrid approach (supervised and unsupervised): Naïve Byes- 72.0 ± 0.3 SVM- 72.9 ± 0.3	Removing noise from web pages while preserving most relevant features to increase accuracy of web classification	- Focus only isolated web pages. Does not include hyperlinks.
(Zhan et al., 2009)	Gather customer concerns from online product reviews – A text summarization approach	Extractive Text summarization	Five datasets from Hu's corpus (M. Hu & Liu, 2004) and 3 sets from Amazon	Average responsiveness scores: 4.3	find and extracts important topics from a set of online reviews and then ranks these retrieved topics	- For different online sites the style of reviews is written in differently. This difference is not efficiently entertained.
(Bouras & Tsogkas, 2010)	Noun retrieval effect on text summarization and delivery of personalized news articles to the user's desktop	Multi-Document Extractive Text summarization	Numerous news portals around the internet	Precision is boosted by using noun retrieval effect. With new personalization scheme increase to around 17% for precision and 14% for recall.	-Enhance personalization algorithm with help of various features extracted from user's profile and viewed history of articles. - A stable system for day-to-day use is build named as PeRSSonal.	Language dependent
(Feblowitz et al., 2011)	Summarization of clinical information: A conceptual model	Extractive Text summarization	Day to day clinical data	-	Both computer supported and computer independent clinical tasks are analysed.	Not standardised and optimized clinical summary
(Lloret et al., 2011)	Text Summarization Contribution to Semantic Question Answering: New Approaches for Finding Answers on the Web Elena	Extractive Text summarization	Most relevant 20 documents from google search engine for particular Question.	Query-focused summaries gives 58% improvement in accuracy than generic summaries. shorter summaries gives 6.3% improvement in accuracy than long summaries.	Efficiently combines text summarization for semantic question answering while focusing on query-based summaries rather than generic summaries.	The summary for a particular question's answer was built from only 20 documents which were retrieved from google search engine.
(S. Liu et al., 2012)	TIARA: Interactive, topic-based visual text summarization and analysis	Multi-Document Extractive Text summarization	IBM employees' Email	Improvement in TIARA with regarding to usefulness and satisfaction of summary than TheMail. (Viégas et al., 2006)	-An Interactive visual text analysis tool TIARA which produces a visual summary of text analytic results automatically and enhanced with significant, time-sensitive topic-based summary method.	Application – specific features are not added in the tool.

(Kavila et al., 2013)	An automatic legal document summarization and search using hybrid system	Extractive text summarization	corpus presently consists of 100 legal documents related to criminal and civil collected	90% results in automatic search	Hybrid system of Keyword/ Key phrase matching technique and Case based technique	Future work will a hybrid system to find the expected judgement for a new case from the past cases.
(Lloret et al., 2013)	COMPENDIUM: A text summarization system for generating abstracts of research papers	Hybrid Text summarization	50 research papers from journal of medicine	-quantitative and qualitative evaluation approach -Precision: 40.53 Recall:44.02	-COMPENDIUM produces summaries for biomedical research papers automatically with both approaches: extractive and abstractive. -abstractive are far better as per user perspective.	Extractive summaries generated by system are contain less similar information to human written summaries
(Perea-Ortega et al., 2013)	Application of text summarization techniques to the geographical information retrieval task	Extractive Text summarization	169,477 documents collections of stories and newswires	GeoCLEF: Compression rate ranging from 20% to 90% Recall MAP	Two types of summaries are generated (generic & geographical) with improvement in recall and compression rates.	- approach can be only applied to single document -discarding all the sentences which do not contain any geographical information may leads to loss of information.
(Sankarasubramaniam et al., 2014)	Text summarization using Wikipedia	Single and Multi-Document Extractive Text summarization	DUC (2002)-567 English news articles	ROUGE-1: 0.46 ROUGE-2: 0.23	-Works for both single document and multi document - For selecting summary phrases, created a bipartite sentence–concept graph and presented an iterative ranking algorithm.	Evaluation is done against human-generated summary which naturally favours the leading line sin new article.
(Ramesh et al., 2015)	Figure-Associated Text Summarization and Evaluation	Extractive Text summarization	94 annotated figures selected from 19 different journals	F1 score of 0.66 and ROUGE-1 score of 0.97	Unsupervised FigSum+ system that automatically fetches associated texts, remove repetitions, and generate text summary for a particular figure.	-It requires annotated figures. - Experiments results are generated by only 94 figures.
(Y. H. Hu et al., 2017)	Opinion mining from online hotel reviews – A text summarization approach	Machine learning based Extractive Text summarization	Reviews of Red Roof Inn and Gansevoort Meatpacking Hotel selected from TripAdvisor.com.	Mean, standard deviation and significance level.	-Proposed technique fetches top-k informative sentences from online reviews -Proposed method focused on critical factors such as usefulness of a review, credibility of authors, review time, conflicts in reviews.	- TripAdvisor.com contains reviews in many more languages but the proposed approach works for only English. - Small sample of participants with same backgrounds. - Experiments are done for only 2 hotels.
(Lovinger et al., 2017)	Gist: general integrated summarization of text and reviews	Extractive text summarization with optimization-Based approach	Movie reviews, news articles	Average F-measure: 0.276 Average running time: 0.067235	High F-score compared to TextRank and LexRank	High computational time and cost
(Chakraborty et al., 2019)	Tweet summarization of news articles: An objective ordering-based perspective	Extractive text summarization with LEXRANK approach	News article dataset and tweet dataset	Rouge-1, F1-score: 0.664 Rouge-2, F1-score: 0.548	Capturing the diverse opinions helps in better identification of the relevant tweet set.	classification of opinion from tweets are not implemented
(Kumar & Reddy, 2019)	Factual instance tweet summarization and opinion analysis of sport competition	Extractive text summarization	IPL 2017 challenger 1, eliminator, challenger 2, and final competition with each having 10,000 tweets	Sentiment classifier time: -Logistic regression Train: 0.27s Test: 0.025	classification of sub-events of tweets	-opinion classification is not derived.

6.2 Challenges and future scope

While generating an automated text summary, one faces a lot of challenges. The first challenge is defining what constitutes a decent summary, or more precisely, how a summary might be constructed. Our requirements for a summary provide good clues as to what it should be: extractive or abstractive, general, or query-driven, etc. Even if we figure out how humans normally summarise, putting it into practice will be difficult. Creating a powerful automatic text summarizer necessitates many resources, either in terms of tools or corpora. Another challenge is summary in formativeness; how can a machine emulate human people when it comes to summarising? One of the long-standing issues has been the coherence of the summary. The shortage of resources is another most challenging problems in ATS.

Compared to the past, there are numerous powerful tools for stemming, parsing, and other tasks available now. Despite this, determining which ones are appropriate for a particular summarization problem is the issue. Furthermore, annotated corpus for ATS can be considered a challenge. The evaluation process is also a significant difficulty. Both intrinsic and extrinsic evaluation approaches were explored in this work. The language shared by a machine-generated reference summary is generally the focus of current intrinsic evaluation methods. Intuitive evaluation can create new ways to evaluate the summary based on the information it includes and how it is presented. The process of evaluation is highly subjective. First, a reasonable criterion must be defined to understand what is important and what is not. It is also unclear whether this procedure can be fully automated.

Text summarising has been around for more than fifty years, and the academic community is very interested in it; therefore, they continue to improve existing text summarization ways or invent new summary approaches to provide higher-quality summaries. However, text summarization performance is still average, and the summaries created are not perfect. As a result, by merging this system with other systems, it can be made more intelligent, allowing the combined system to perform better.

- ## Conclusion

Automatic Text summarization reduces the size of a source text while maintaining its information value and overall meaning. Automatic Text summarization has become a powerful technique for analysing text information due to a large amount of information we are given and the growth of Internet technologies. The automatic summarization of text is a thriving-known task in natural language processing (NLP). Automatic text summarization is an exciting research area, and it has a treasure of applications. This paper aims to make readers understand automatic text summarization from ground level and familiarise them with all detailed types of ATS systems. After that all different types are distinguished deeply and clearly in this study. The summarization task is mainly divided into extractive and abstractive. The study shows numerous techniques for extractive summarization, but the summaries generated by extractive summarizers are far from human-made summaries. On the other hand, abstractive summarizer is close to human summaries but not practically implemented with high performance. The combination of both extractive and abstractive is hybrid text summarization. This paper includes research survey on Extractive, Abstractive and Hybrid Text Summarization. Also, this survey article tried to cover all major application areas of ATS system and provided detailed survey on the same. There are so many methods to evaluate summarizing system and generated summaries that are included in this paper. Further it gives brief idea about frequently used datasets, conferences and programs that held every year for automatic text summarization system.

. The future study is to build a robust, domain and language independent extractive text summarization that works well with multi-documents. Similarly, because the quality evaluation of the summary is done manually by experienced assessors, it is highly subjective. There are specific quality assessment criteria, such as grammaticality and coherence, but the results are different when two experts evaluate the same summary.

Conflicts of Interest: On behalf of all authors, I state that there is no conflict of interest.

References

N. D. (2016). Automatic Text Summarization Using Supervised Machine Learning Technique for Hindi Langauge. *International Journal of Research in Engineering and Technology*, *05*(06), 361–367. https://doi.org/10.15623/ijret.2016.0506065

Ab, A., & Sunitha, C. (n.d.). *An Overview on Document Summarization Techniques*. 113–118.

Al-Radaideh, Q. A., & Bataineh, D. Q. (2018a). A Hybrid Approach for Arabic Text Summarization Using Domain Knowledge and Genetic Algorithms. *Cognitive Computation*, *10*(4), 651–669. https://doi.org/10.1007/s12559-018-9547-z

Al-Radaideh, Q. A., & Bataineh, D. Q. (2018b). A Hybrid Approach for Arabic Text Summarization Using Domain Knowledge and Genetic Algorithms. *Cognitive Computation*, *10*(4), 651–669. https://doi.org/10.1007/s12559-018-9547-z

Alami, N., Mallahi, M. El, Amakdouf, H., & Qjidaa, H. (2021). Hybrid method for text summarization based on statistical and semantic treatment. *Multimedia Tools and Applications*. https://doi.org/10.1007/s11042-021-10613-9

Alami, N., Meknassi, M., & En-nahnahi, N. (2019). Enhancing unsupervised neural networks based text summarization with word embedding and ensemble learning. *Expert Systems with Applications*, *123*, 195–211. https://doi.org/10.1016/j.eswa.2019.01.037

Alami, N., Meknassi, M., En-nahnahi, N., El Adlouni, Y., & Ammor, O. (2021). Unsupervised neural networks for automatic Arabic text summarization using document clustering and topic modeling. *Expert Systems with Applications*, *172*. https://doi.org/10.1016/j.eswa.2021.114652

ALGULIEV, R., & ALIGULIYEV, R. (2009). Evolutionary Algorithm for Extractive Text Summarization. *Intelligent Information Management*, *01*(02), 128–138. https://doi.org/10.4236/iim.2009.12019

Alguliev, R. M., Aliguliyev, R. M., Hajirahimova, M. S., & Mehdiyev, C. A. (2011a). MCMR: Maximum coverage and minimum redundant text summarization model. *Expert Systems with Applications*, *38*(12), 14514–14522. https://doi.org/10.1016/j.eswa.2011.05.033

Alguliev, R. M., Aliguliyev, R. M., Hajirahimova, M. S., & Mehdiyev, C. A. (2011b). MCMR: Maximum coverage and minimum redundant text summarization model. *Expert Systems with Applications*, *38*(12), 14514–14522. https://doi.org/10.1016/j.eswa.2011.05.033

Alguliyev, R. M., Aliguliyev, R. M., Isazade, N. R., Abdi, A., & Idris, N. (2019a). COSUM: Text summarization based on clustering and optimization. *Expert Systems*, *36*(1), 1–17. https://doi.org/10.1111/exsy.12340

Alguliyev, R. M., Aliguliyev, R. M., Isazade, N. R., Abdi, A., & Idris, N. (2019b). COSUM: Text summarization based on clustering and optimization. *Expert Systems*, *36*(1). https://doi.org/10.1111/exsy.12340

Antiqueira, L., Oliveira, O. N., Costa, L. da F., & Nunes, M. das G. V. (2009). A complex network approach to text summarization. *Information Sciences*, *179*(5), 584–599. https://doi.org/10.1016/j.ins.2008.10.032

Aone, C., Okurowski, M. E., Gorlinsky, J., & Larsen, B. (1997). A Scalable Summarization System Using Robust NLP. *Proceedings of the Intelligent Scalable Text Summarization Workshop*, 66–73.

Banko, M., Mittal, V. O., & Witbrock, M. J. (2000). *Headline generation based on statistical translation*. 318–325. https://doi.org/10.3115/1075218.1075259

Barzilay, R., & Mckeown, K. R. (2005). *Sentence Fusion for Multidocument News Summarization*.

Belwal, R. C., Rai, S., & Gupta, A. (2021). A new graph-based extractive text summarization using keywords or topic modeling. *Journal of Ambient Intelligence and Humanized Computing*, *12*(10), 8975–8990. https://doi.org/10.1007/s12652-020-02591-x

Bhat, I. K., Mohd, M., & Hashmy, R. (2018). SumItUp: A Hybrid Single-Document Text Summarizer. *Advances in Intelligent Systems and Computing*, *583*(April), 619–634. https://doi.org/10.1007/978-981-10-5687-1_56

Binwahlan, M. S., Salim, N., & Suanmali, L. (2009a). Swarm Based Text Summarization. *2009 International Association of Computer Science and Information Technology - Spring Conference, IACSIT-SC 2009*, 145–150. https://doi.org/10.1109/IACSIT-SC.2009.61

Binwahlan, M. S., Salim, N., & Suanmali, L. (2010). Fuzzy swarm diversity hybrid model for text summarization. *Information Processing and Management*, *46*(5), 571–588. https://doi.org/10.1016/j.ipm.2010.03.004

Binwahlan, M. S., Salim, N., & Suanmali, L. (2009b). Swarm Based Text Summarization. *2009 International Association of Computer Science and Information Technology - Spring Conference, IACSIT-SC 2009*, 145–150. https://doi.org/10.1109/IACSIT-SC.2009.61

Bloom, ; E H, Fischer, H., Charbonneau, N. K., Tonks, ; J, Mirkovitch, J. E., Sadowski, D. ; H. B., Shuai, K., Darnell, J. E., & Gilman, M. Z. (n.d.). Automatic Analysis, Theme Generation, and Summarization of Machine-Readable Texts. In *Interferon: Principles and Medical Applications* (Vol. 13).

Bouras, C., & Tsogkas, V. (2010). Noun retrieval effect on text summarization and delivery of personalized news articles to the user's desktop. *Data and Knowledge Engineering*, *69*(7), 664–677. https://doi.org/10.1016/j.datak.2010.02.005

Brin, S., & Page, L. (2012). Reprint of: The anatomy of a large-scale hypertextual web search engine. *Computer Networks*, *56*(18), 3825–3833. https://doi.org/10.1016/j.comnet.2012.10.007

Carbonell, J., & Goldstein, J. (1998). The use of MMR, diversity-based reranking for reordering documents and producing summaries. In Proceedings of the 21st annual international ACM SIGIR conference on Research and development in information retrieval, 335-336.

Chakraborty, R., Bhavsar, M., Dandapat, S. K., & Chandra, J. (2019). Tweet summarization of news articles: An objective ordering-based perspective. *IEEE Transactions on Computational Social Systems*, *6*(4), 761–777. https://doi.org/10.1109/TCSS.2019.2926144

Chen, Q. (2015). *LCSTS: A Large Scale Chinese Short Text Summarization Dataset*.

De Tre, G., Hallez, A., & Bronselaer, A. (2014). Performance optimization of object comparison. *International Journal of Intelligent Systems*, *29*(2), 495–524. https://doi.org/10.1002/int

Dohare, S., Karnick, H., & Gupta, V. (2017a). Text Summarization using Abstract Meaning Representation. *ArXiv*.

Dohare, S., Karnick, H., & Gupta, V. (2017b). *Text Summarization using Abstract Meaning Representation*. http://arxiv.org/abs/1706.01678

El-Kassas, W. S., Salama, C. R., Rafea, A. A., & Mohamed, H. K. (2020). EdgeSumm: Graph-based framework for automatic text summarization. *Information Processing and Management*, *57*(6), 102264. https://doi.org/10.1016/j.ipm.2020.102264

Farzindar, A., & Lapalme, G. (2004). Legal Text Summarization by Exploration of the Thematic Structure and Argumentative Roles. *In Text Summarization Branches Out Conference Held in Conjunction with ACL 2004*, 27–38.

Feblowitz, J. C., Wright, A., Singh, H., Samal, L., & Sittig, D. F. (2011). Summarization of clinical information: A conceptual model. *Journal of Biomedical Informatics*, *44*(4), 688–699. https://doi.org/10.1016/j.jbi.2011.03.008

Ferreira, R., De Souza Cabral, L., Freitas, F., Lins, R. D., De França Silva, G., Simske, S. J., & Favaro, L. (2014). A multi-document summarization system based on statistics and linguistic treatment. *Expert Systems with Applications*, *41*(13), 5780–5787. https://doi.org/10.1016/j.eswa.2014.03.023

Ferreira, R., De Souza Cabral, L., Lins, R. D., Pereira E Silva, G., Freitas, F., Cavalcanti, G. D. C., Lima, R., Simske, S. J., & Favaro, L. (2013). Assessing sentence scoring techniques for extractive text summarization. In *Expert Systems with Applications* (Vol. 40, Issue 14, pp. 5755–5764). https://doi.org/10.1016/j.eswa.2013.04.023

Gambhir, M., & Gupta, V. (2017). Recent automatic text summarization techniques: a survey. *Artificial Intelligence Review*, *47*(1). https://doi.org/10.1007/s10462-016-9475-9

Garner, R. (1982). Efficient text summarization: Costs and benefits. *Journal of Educational Research*, *75*(5), 275–279. https://doi.org/10.1080/00220671.1982.10885394

Gupta, V., & Kaur, N. (2016). A Novel Hybrid Text Summarization System for Punjabi Text. *Cognitive Computation*, *8*(2), 261–277. https://doi.org/10.1007/s12559-015-9359-3

HANSON ER. (1971). Musicassette Interchangeability. the Facts Behind the Facts. *AES: Journal of the Audio Engineering Society*, *19*(5), 417–425.

He, J., Kryscinski, W., McCann, B., Rajani, N., & Xiong, C. (2020). CTRLsum: Towards generic controllable text summarization. *ArXiv*, 1–35.

He, J., Kryściński, W., McCann, B., Rajani, N., & Xiong, C. (2020). *CTRLsum: Towards Generic Controllable Text Summarization*. http://arxiv.org/abs/2012.04281

Hovy, E., & Lin, C.-Y. (n.d.). *AUTOMATED TEXT SUMMARIZATION AND THE SUMMARIST SYSTEM*.

Hovy, E., & Lin, C.-Y. (1996). *Automated text summarization and the SUMMARIST system*. 197. https://doi.org/10.3115/1119089.1119121

Hu, M., & Liu, B. (2004). Mining opinion features in customer reviews. *Proceedings of the National Conference on Artificial Intelligence*, 755–760.

Hu, Y. H., Chen, Y. L., & Chou, H. L. (2017). Opinion mining from online hotel reviews – A text summarization approach. *Information Processing and Management*, *53*(2), 436–449. https://doi.org/10.1016/j.ipm.2016.12.002

Huang, L., He, Y., Wei, F., & Li, W. (2010). *Modeling Document Summarization as Multi-objective Optimization*. 2–6. https://doi.org/10.1109/IITSI.2010.80

Ježek, K., Katedra, J. S., & Steinberger, J. (2007). *Automatic Text Summarization (The state of the art 2007 and new challenges)*.

Jezek, K., & Steinberger, J. (2008). Automatic summarizing: (The state of the art 2007 and new challenges). *Proceedings of Znalosti*, 1–12.

Joshi, A., Fidalgo, E., Alegre, E., & Fernández-Robles, L. (2019). SummCoder: An unsupervised framework for extractive text summarization based on deep auto-encoders. *Expert Systems with Applications*, *129*, 200–215. https://doi.org/10.1016/j.eswa.2019.03.045

Kanapala, A., Pal, S., & Pamula, R. (2019). Text summarization from legal documents: a survey. *Artificial Intelligence Review*, *51*(3), 371–402. https://doi.org/10.1007/s10462-017-9566-2

Kavila, S. D., Puli, V., Raju, G. S. V. P., & Bandaru, R. (2020). *An Automatic Legal Document Summarization and Search Using Hybrid System An Automatic Legal Document Summarization*. *January 2013*. https://doi.org/10.1007/978-3-642-35314-7

Kiyani, F., & Tas, O. (2017). A survey automatic text summarization. *Pressacademia*, *5*(1), 205–213. https://doi.org/10.17261/pressacademia.2017.591

Ko, Y., & Seo, J. (2008). An effective sentence-extraction technique using contextual information and statistical approaches for text summarization. *Pattern Recognition Letters*, *29*(9), 1366–1371. https://doi.org/10.1016/j.patrec.2008.02.008

Koupaee, M., & Wang, W. Y. (2018). *WikiHow: A Large Scale Text Summarization Dataset*. http://arxiv.org/abs/1810.09305

Kryściński, W., McCann, B., Xiong, C., & Socher, R. (2019a). Evaluating the factual consistency of abstractive text summarization. *ArXiv*. https://doi.org/10.18653/v1/2020.emnlp-main.750

Kryściński, W., McCann, B., Xiong, C., & Socher, R. (2019b). *Evaluating the Factual Consistency of Abstractive Text Summarization*. http://arxiv.org/abs/1910.12840

Kryściński, W., Paulus, R., Xiong, C., & Socher, R. (2018). *Improving Abstraction in Text Summarization*. http://arxiv.org/abs/1808.07913

Kryściński, W., Paulus, R., Xiong, C., & Socher, R. (2020). Improving abstraction in text summarization. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, EMNLP 2018*, 1808–1817. https://doi.org/10.18653/v1/d18-1207

Kumar, N. V., & Reddy, M. J. (n.d.). *Factual Instance Tweet Summarization and Opinion Analysis of Sport Competition*. Springer Singapore. https://doi.org/10.1007/978-981-13-3393-4

Kutlu, M., Ciğir, C., & Cicekli, I. (2010). Generic text summarization for Turkish. *Computer Journal*, *53*(8), 1315–1323. https://doi.org/10.1093/comjnl/bxp124

Li, P., Lam, W., Bing, L., & Wang, Z. (2017). *Deep Recurrent Generative Decoder for Abstractive Text Summarization*. http://arxiv.org/abs/1708.00625

Lin, C., & Rey, M. (2001). *R OUGE : A Package for Automatic Evaluation of Summaries*.

Lin, C. Y. (1999). Training a selection function for extraction. *International Conference on Information and Knowledge Management, Proceedings*, 55–62. https://doi.org/10.1145/319950.319957

Ling, X., Jiang, J., He, X., Mei, Q., Zhai, C., & Schatz, B. (2007). Generating gene summaries from biomedical literature: A study of semi-structured summarization. *Information Processing and Management*, *43*(6), 1777–1791. https://doi.org/10.1016/j.ipm.2007.01.018

Linhares Pontes, E., Huet, S., Torres-Moreno, J. M., & Linhares, A. C. (2020). Compressive approaches for cross-language multi-document summarization. *Data and Knowledge Engineering*, *125*. https://doi.org/10.1016/j.datak.2019.101763

Liu, F., Flanigan, J., Thomson, S., Sadeh, N., & Smith, N. A. (2015). Toward abstractive summarization using semantic representations. *NAACL HLT 2015 - 2015 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Proceedings of the Conference*, 1077–1086. https://doi.org/10.3115/v1/n15-1114

Liu, S., Zhou, M. X., Pan, S., Song, Y., Qian, W., Cai, W., & Lian, X. (2012). TIARA: Interactive, topic-based visual text summarization and analysis. *ACM Transactions on Intelligent Systems and Technology*, *3*(2). https://doi.org/10.1145/2089094.2089101

Lloret, E., Llorens, H., Moreda, P., Saquete, E., & Palomar, M. (2011). Text summarization contribution to semantic question answering: New approaches for finding answers on the web. *International Journal of Intelligent Systems*, *26*(12), 1125–1152. https://doi.org/10.1002/int.20502

Lloret, E., Romá-Ferri, M. T., & Palomar, M. (2013). COMPENDIUM: A text summarization system for generating abstracts of research papers. *Data and Knowledge Engineering*, *88*, 164–175. https://doi.org/10.1016/j.datak.2013.08.005

Lorch, R. F., Lorch, E. P., Ritchey, K., McGovern, L., & Coleman, D. (2001). Effects of Headings on Text Summarization. *Contemporary Educational Psychology*, *26*(2), 171–191. https://doi.org/10.1006/ceps.1999.1037

Lovinger, J., Valova, I., & Clough, C. (2017). Gist : general integrated summarization of text and reviews. *Soft Computing*. https://doi.org/10.1007/s00500-017-2882-2

Mashechkin, I. V., Petrovskiy, M. I., Popov, D. S., & Tsarev, D. V. (2011). Automatic text summarization using latent semantic analysis. *Programming and Computer Software*, *37*(6), 299–305. https://doi.org/10.1134/S0361768811060041

Mckeown, K. R., Klavans, J. L., & Schiffman, B. (n.d.). *Tracking and Summarizing News on a Daily Basis with Columbia ' s Newsblaster*.

Miao, Y., & Blunsom, P. (2016). Language as a latent variable: Discrete generative models for sentence compression. *EMNLP 2016 - Conference on Empirical Methods in Natural Language Processing, Proceedings*, 319–328. https://doi.org/10.18653/v1/d16-1031

Mihalcea, R., & Ceylan, H. (2007). Explorations in automatic book summarization. *EMNLP-CoNLL 2007 - Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, *June*, 380–389.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *1st International Conference on Learning Representations, ICLR 2013 - Workshop Track Proceedings*, 1–12.

Mohd, M., Jan, R., & Shah, M. (2020). Text document summarization using word embedding. *Expert Systems with Applications*, *143*. https://doi.org/10.1016/j.eswa.2019.112958

Moradi, M., Dashti, M., & Samwald, M. (2020). Summarization of biomedical articles using domain-specific word embeddings and graph ranking. *Journal of Biomedical Informatics*, *107*(May), 103452. https://doi.org/10.1016/j.jbi.2020.103452

Muresan, S., Tzoukermann, E., & Klavans, J. L. (2001). *Combining linguistic and machine learning techniques for email summarization*. 1–8. https://doi.org/10.3115/1117822.1117837

Mutlu, B., Sezer, E. A., & Akcayol, M. A. (2020). Candidate sentence selection for extractive text summarization. *Information Processing and Management*, *57*(6). https://doi.org/10.1016/j.ipm.2020.102359

Nallapati, R., Zhou, B., dos Santos, C., Gulçehre, Ç., & Xiang, B. (2016). Abstractive text summarization using sequence-to-sequence RNNs and beyond. *CoNLL 2016 - 20th SIGNLL Conference on Computational Natural Language Learning, Proceedings*, 280–290. https://doi.org/10.18653/v1/k16-1028

Neto, J. L., Freitas, A. A., & Kaestner, C. A. A. (2002). Automatic text summarization using a machine learning approach. *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, *2507*(i), 205–215. https://doi.org/10.1007/3-540-36127-8_20

Nomoto, T., & Matsumoto, Y. (2003). The diversity-based approach to open-domain text summarization. *Information Processing and Management*, *39*(3), 363–389. https://doi.org/10.1016/S0306-4573(02)00096-1

Pembe, F., & Güngör, T. (2007). Automated Querybiased and Structure-preserving Text Summarization on Web Documents. *... on Innovations in Intelligent Systems and ...*. https://pdfs.semanticscholar.org/7db2/fde4c2e4ba7aa7949780435ef4120f9a24ce.pdf%0Ahttp://www.cmpe.boun.edu.tr/~gungort/papers/automated query-biased and structure-preserving text summarization on web documents.pdf

Perea-Ortega, J. M., Lloret, E., Alfonso Ureña-López, L., & Palomar, M. (2013). Application of text summarization techniques to the geographical information retrieval task. *Expert Systems with Applications*, *40*(8), 2966–2974. https://doi.org/10.1016/j.eswa.2012.12.012

Proceedings of the 2009 2nd International Conference on Computer Science and Its Applications : CSA 2009. (2009). IEEE.

Radev, D. R., Jing, H., Styś, M., & Tam, D. (2004). Centroid-based summarization of multiple documents. *Information Processing and Management*, *40*(6), 919–938. https://doi.org/10.1016/j.ipm.2003.10.006

Ramesh, B. P., Sethi, R. J., & Yu, H. (2015). Figure-associated text summarization and evaluation. *PLoS ONE*, *10*(2). https://doi.org/10.1371/journal.pone.0115671

Rani, R., & Lobiyal, D. K. (2021). An extractive text summarization approach using tagged-LDA based topic modeling. *Multimedia Tools and Applications*, *80*(3), 3275–3305. https://doi.org/10.1007/s11042-020-09549-3

Rau, L. F., Jacobs, P. S., & Zernik, U. (1989). INFORMATION EXTRACTION AND TEXT SUMMARIZATION USING LINGUISTIC KNOWLEDGE ACQUISITION*. In *Informarron Processrng & Managemenr* (Vol. 25, Issue 4).

Reimer, U., Hahn, U., Life, S., & Unlverslty, F. (n.d.). *A Formal Model of Text Summarization Based on Condensation Operators of a Terminological Logic*.

Rush, A. M., Chopra, S., & Weston, J. (2015). A neural attention model for sentence summarization. *Conference Proceedings - EMNLP 2015: Conference on Empirical Methods in Natural Language Processing*, 379–389.

Salton, G., Singhal, A., Mitra, M., & Buckley, C. (1997). Automatic text structuring and summarization. *Information Processing and Management*, *33*(2), 193–207. https://doi.org/10.1016/S0306-4573(96)00062-3

Sanchez-Gomez, J. M., Vega-Rodríguez, M. A., & Pérez, C. J. (2021). The impact of term-weighting schemes and similarity measures on extractive multi-document text summarization. *Expert Systems with Applications*, *169*(December 2020). https://doi.org/10.1016/j.eswa.2020.114510

Sankarasubramaniam, Y., Ramanathan, K., & Ghosh, S. (2014a). Text summarization using Wikipedia. *Information Processing and Management*, *50*(3), 443–461. https://doi.org/10.1016/j.ipm.2014.02.001

Sankarasubramaniam, Y., Ramanathan, K., & Ghosh, S. (2014b). Text summarization using Wikipedia. *Information Processing and Management*, *50*(3), 443–461. https://doi.org/10.1016/j.ipm.2014.02.001

See, A. V., (2021).Volume of data/information created, captured, copied, and consumed worldwide from 2010 to 2025. https://www.statista.com/statistics/871513/worldwide-data-created/, accessed on 27 Jan 2022.

Shen, D., Yang, Q., & Chen, Z. (2007). Noise reduction through summarization for Web-page classification. *Information Processing and Management*, *43*(6), 1735–1747. https://doi.org/10.1016/j.ipm.2007.01.013

Silber, H. G., & McCoy, K. F. (2002). Efficiently computed lexical chains as an intermediate representation for automatic text summarization. *Computational Linguistics*, *28*(4), 486–496. https://doi.org/10.1162/089120102762671954

Sinha, A., Yadav, A., & Gahlot, A. (n.d.). *Extractive Text Summarization using Neural Networks*.

Song, S., Huang, H., & Ruan, T. (2019a). Abstractive text summarization using LSTM-CNN based deep learning. *Multimedia Tools and Applications*, *78*(1), 857–875. https://doi.org/10.1007/s11042-018-5749-3

Song, S., Huang, H., & Ruan, T. (2019b). Abstractive text summarization using LSTM-CNN based deep learning. *Multimedia Tools and Applications*, *78*(1), 857–875. https://doi.org/10.1007/s11042-018-5749-3

Steinberger, J., & Ježek, K. (2009). Evaluation measures for text summarization. *Computing and Informatics*, *28*(2), 251–275.

Van Lierde, H., & Chow, T. W. S. (2019). Query-oriented text summarization based on hypergraph transversals. *Information Processing and Management*, *56*(4), 1317–1338. https://doi.org/10.1016/j.ipm.2019.03.003

Viégas, F. B., Golder, S., & Donath, J. (2006). Visualizing email content: Portraying relationships from conversational histories. *Conference on Human Factors in Computing Systems - Proceedings*, *2*, 979–988.

Wang, D., Liu, P., Zhong, M., Fu, J., Qiu, X., & Huang, X. (2019). Exploring domain shift in extractive text summarization. *ArXiv*.

WEI, B., REN, X., SUN, X., ZHANG, Y., CAI, X., & SU, Q. (2018). Regularizing output distribution of abstractive chinese social media text summarization for improved semantic consistency. *ArXiv*, *5*, 1–14.

Woodsend, K., Feng, Y., & Lapata, M. (2010). Title generation with quasi-synchronous grammar. *EMNLP 2010 - Conference on Empirical Methods in Natural Language Processing, Proceedings of the Conference*, *October*, 513–523.

Xu, J., & Durrett, G. (2020). Neural extractive text summarization with syntactic compression. *EMNLP-IJCNLP 2019 - 2019 Conference on Empirical Methods in Natural Language Processing and 9th International Joint Conference on Natural Language Processing, Proceedings of the Conference*, 3292–3303. https://doi.org/10.18653/v1/d19-1324

Yang, M., Li, C., Shen, Y., Wu, Q., Zhao, Z., & Chen, X. (2020). Hierarchical Human-Like Deep Neural Networks for Abstractive Text Summarization. *IEEE Transactions on Neural Networks and Learning Systems*, 1–14. https://doi.org/10.1109/tnnls.2020.3008037

Yang, M., Li, C., Shen, Y., Wu, Q., Zhao, Z., & Chen, X. (2021). Hierarchical Human-Like Deep Neural Networks for Abstractive Text Summarization. *IEEE Transactions on Neural Networks and Learning Systems*, *32*(6), 2744–2757. https://doi.org/10.1109/TNNLS.2020.3008037

Yang, M., Wang, X., Lu, Y., Lv, J., Shen, Y., & Li, C. (2020). Plausibility-promoting generative adversarial network for abstractive text summarization with multi-task constraint. *Information Sciences*, *521*, 46–61. https://doi.org/10.1016/j.ins.2020.02.040

Ye, S., Chua, T. S., Kan, M. Y., & Qiu, L. (2007a). Document concept lattice for text understanding and summarization. *Information Processing and Management*, *43*(6), 1643–1662. https://doi.org/10.1016/j.ipm.2007.03.010

Ye, S., Chua, T. S., Kan, M. Y., & Qiu, L. (2007b). Document concept lattice for text understanding and summarization. *Information Processing and Management*, *43*(6), 1643–1662. https://doi.org/10.1016/j.ipm.2007.03.010

Yeh, J. Y., Ke, H. R., Yang, W. P., & Meng, I. H. (2005a). *Text summarization using a trainable summarizer and latent semantic analysis*. *41*(1), 75–95. https://doi.org/10.1016/j.ipm.2004.04.003

Yeh, J. Y., Ke, H. R., Yang, W. P., & Meng, I. H. (2005b). Text summarization using a trainable summarizer and latent semantic analysis. *Information Processing and Management*, *41*(1), 75–95. https://doi.org/10.1016/j.ipm.2004.04.003

Yousefi-Azar, M., & Hamey, L. (2017a). Text summarization using unsupervised deep learning. *Expert Systems with Applications*, *68*, 93–105. https://doi.org/10.1016/j.eswa.2016.10.017

Yousefi-Azar, M., & Hamey, L. (2017b). Text summarization using unsupervised deep learning. *Expert Systems with Applications*, *68*, 93–105. https://doi.org/10.1016/j.eswa.2016.10.017

Zajic, D., Dorr, B., & Schwartz, R. (2004). BBN / UMD at DUC-2004 : Topiary. *Proceedings of the HLT-NAACL 2004 Document Understanding Workshop, Boston*, 112--119.

Zhan, J., Loh, H. T., & Liu, Y. (2009). Gather customer concerns from online product reviews - A text summarization approach. *Expert Systems with Applications*, *36*(2 PART 1), 2107–2115. https://doi.org/10.1016/j.eswa.2007.12.039

Zhang, Y., Li, D., Wang, Y., Fang, Y., & Xiao, W. (2019). Abstract text summarization with a convolutional seq2seq model. *Applied Sciences (Switzerland)*, *9*(8). https://doi.org/10.3390/app9081665

IJCRT.ORG ISSN : 2320-2882

INTERNATIONAL JOURNAL OF CREATIVE RESEARCH THOUGHTS (IJCRT)

An International Open Access, Peer-reviewed, Refereed Journal

AUTOMATIC TEXT SUMMARIZATION USING BART

Sriram Parabrahma chari [1] Asst.Professor, B. Dinesh Kumar [2],CH. Shonaakshay [3], B. Thrillokh [4],B. Ravi Teja [5]

Sreyas Institute of Engineering and Technology

I. ABSTRACT:

Nowadays, a lot of information can be obtained from both online and offline sources. We can access hundreds of documents for a topic. The ability to summarize or create popular topics allows users to quickly search for topics and get initial information as quickly as possible. Manually extracting useful information from them is a difficult task. To solve this problem automatic text summarization (ATS) systems were developed. Text summarization is the process of extracting important information from a large document and turning it into a summary, preserving all the important points. This abstract presents a novel approach using BART pre-trained model which is a large language model used widely for text generation tasks. The proposed system takes input a either a URL, Text or a file of any format. Automatic text summarization aims to produce a shortened version of a long text document while retaining its overall meaning. This has applications like summarizing news articles, research papers, reports, etc. There are two common approaches - extractive methods that identify and extract key sentences from the original text, and abstractive methods which paraphrase and generate new sentences. The proposed system takes input text, documents or URLs and returns a summary. It uses a pretrained BART model fine-tuned on the CNN-Daily Mail dataset. BART (Bidirectional and Auto-Regressive Transformer) combines BERT and GPT architectures. It is a seq2seq model trained as a denoising autoencoder, taking text sequence input and output. Key advantages of BART include improved summary coherence and factual consistency. During training, the model learns to generate a summary given an input document. Current models can produce high quality summaries, but issues remain around coherence, repetition, and accuracy. Key challenges include improving faithfulness to the original text, avoiding repetition, and controlling summary length and style. Future directions involve multi-document summarization, controlled generation, better evaluation metrics, and performance on domain-specific datasets. Overall, text summarization continues to be an important application of NLP that helps condense information and enable quick access to key facts and context. The proposed BART-based system aims to leverage strengths in text generation and compression to improve summarization capabilities.

KEYWORDS: *Text summarization, extractive text summarization, Abstractive Text summarization, Deep Learning, Natural Language processing, machine learning, Hugging Face, TensorFlow, Chunking, Web Scraping*

II. INTRODUCTION:

As the volume of textual data generated each day rapidly grows, so does the need for capabilities to automatically process this deluge of information into concise overviews highlighting the most salient content. Text summarization aims to address this challenge - generating shortened versions of documents which distill key facts and overall meaning. This has numerous applications across domains like compressing lengthy news articles and scientific papers to brief overviews, generating clinical summaries from electronic health records, creating legal case abstracts, and more. However, automatically generating fluent and representative summaries of texts remains an active area of NLP research. Fundamentally, text summarization involves making content selection choices to decide which information is most relevant, aggregating and rewriting content appropriately. Early approaches relied on surface level metrics like word/sentence frequency or placement to identify salient pieces of texts to extract into summaries. But such techniques often produce incoherent & disjointed outputs lacking overall understanding of the document semantics.

Abstractive summarization hence employs more advanced NLP understanding of original texts to interpret, contextualize and paraphrase concepts using natural language generation techniques. Such capabilities can produce more readable summaries with better coverage of key details compared to simply concatenated extractive fragments. Recent advances in neural natural language processing have significantly advanced the state-of-the-art in abstract text summarization using data-driven learning of semantic representations. Particularly impactful have been sequence-to-sequence neural architectures which take source texts as input and generate summary texts as output after training on human written examples. Models like transformers trained on large datasets have set new benchmarks on summarization quality - with techniques like self-supervised objectives and transfer learning amplifying these successes across low-resource domains and languages as well.

However, challenges remain in aspects like coherence, redundancy and fact consistency of machine generated summaries. There is also significant control and explainability needed when deploying such AI summarization technologies for real world applications. This motivates the work we present in this paper - with rigorous experiments benchmarking a novel summarization technique on established datasets, analysis of generated outputs to highlight strengths and error categories compared to other methods, followed by extensive discussion of current limitations and future priorities for the research community. The key contributions are:

1. Proposing a transformer-based sequence-to-sequence architecture optimized using a hybrid loss function to balance both semantic similarity to reference texts and overall summary quality.
2. Achieving new state-of-the-art results on ROUGE metrics across 3 key summarization benchmarks - outperforming other neural abstractive techniques.
3. In-depth qualitative analysis of model outputs examining information coverage, faithfulness to input documents and readability indicating generalizability.
4. Quantitative analysis of different error types related to factual inconsistencies, content selection and language fluency to inform promising future research directions.

III. LITERATURE SURVEY

[1]. Paper Title: BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension

Authors: Mike Lewis*, Yinhan Liu*, Naman Goyal*, Marjan Ghazvininejad

Description: The paper introduces BART, a denoising autoencoder for pretraining sequence-to-sequence models. BART uses a standard Transformer architecture with a bidirectional encoder and left-to-right autoregressive decoder.

Results Obtained: BART matches RoBERTa's performance on GLUE with comparable resources. It achieves new SOTA results on summarization, dialogue, and QA datasets, with gains of up to 6 ROUGE on summarization.BART also improves machine translation by 1.1 BLEU over backtranslation when used as a pretrained English language model.

[2]. **Paper Title:** An Overview of Text Summarization Techniques

Authors: Narendra Andhale,L.A. Bewoor

Description: The paper provides a comprehensive survey of extractive and abstractive text summarization techniques.Extractive summarization selects important sentences from the original text to create the summary. Challenges include determining which sentences are most informative and avoiding redundancy. Common approaches include TF-IDF, clustering, graph-based methods, etc.Abstractive summarization aims to produce generalized summaries that are more concise by generating new sentences and the techniques are BERT, BART , neural networks etc.

Results Obtained: BART matches RoBERTa's performance on GLUE with comparable resources. It achieves new SOTA results on summarization, dialogue, and QA datasets, with gains of up to 6 ROUGE on summarization.BART also improves machine translation by 1.1 BLEU over backtranslation when used as a pretrained English language model.

[3]. Paper Title: Automatic Text Summarization Methods: A Comprehensive Review

Authors: Divakar Yadav1,Jalpa Desai, Arun Kumar Yadav

Description: The paper compares sequence-to-sequence and LSTM bidirectional models for abstractive summarization.

Results Obtained: Sequence-to-sequence achieves the highest ROUGE-2 score of 0.2057 on CNN Daily News. Evaluation is done using BLEU, ROUGE-1, and ROUGE-2 scores on Amazon reviews and CNN Daily News datasets.

[4]. Paper Title: Extractive Text and Video Summarization usingTF-IDF Algorithm

Authors: Ajinkya Gothankar1, Lavish Gupta2, Niharika Bisht, Samiksha Nehe

Description: The paper proposes an extractive text and video summarization system using the TF-IDF algorithm.It extracts text from various formats like raw text, articles, PDFs, DOCX using different libraries.

Results Obtained: Potential issues like redundancy, coherence of summaries are not analyzed.

[5]. Paper Title: SINGLE DOCUMENT AUTOMATIC TEXT SUMMARIZATION USING TERM FREQUENCY-INVERSE DOCUMENT FREQUENCY (TF-IDF)

Authors: Hans Christian1; Mikhael Pramodana Agus2; Derwin Suhartono3

Description: The paper implements an extractive text summarization system using the TF-IDF algorithm. Evaluation is done on 3 sample documents by comparing with summaries created by humans. The system performed better than two other online summarizers in terms of average F-measure.

Results Obtained: The system achieved an average precision of 0.661, recall of 0.672, and F-measure of 0.666 across the 3 documents.

[6]. Paper Title: A Survey of Text Summarization ExtractiveTechniques
Authors: Gurpreet Singh Lehal, Vishal Gupta
Description: This paper provides different techniques used for Extractive Methods and Features to be considered to include a sentence in Final Summary
Results Obtained: Potential issues like redundancy, coherence of summaries are not analyzed.

[7]. Paper Title: An Analysis of Abstractive Text Summarization Using Pre-trained Models
Authors: Tohida Rehman, Suchandan Das, Debarshi Kumar Sanyal, Samiran Chattopadhyay **Description:** In this paper, we observed outputs from different pre-trained models using differentdatasets
Results Obtained: While using CNN-dailymail dataset and SAMSum dataset, google/pegasuscnn-dailymail model and facebook/bart-large-cnn model provides better results thanT5-base model as per the f-score of ROUGE-1, ROUGE-2, and ROUGE-L performancemetrics. While using BillSum dataset google/pegasus-cnn-dailymail model providesbetter results than T5-base model and facebook/bart-large-cnn model as per the fmeasure of ROUGE-1, ROUGE-2, and ROUGE-L performance metrics.

[8]. Paper Title: An Empirical Survey on Long Document Summarization:Datasets, Models and Metrics)
Authors: HUAN YEE KOH, AustraliaJIAXIN JU, Deakin , AustraliaSHIRUI
Description: we conduct a comprehensive overview of long document summarization andsystematically analyze the three key components of its research settings: benchmark datasets,summarization models and evaluation metrics.
Results Obtained: BART achieved a highest ROUGE score of 0.467

IV. PROPOSED SYSTEM

The proposed system aims to develop an innovative web platform for automatically summarizing text content sourced from diverse documents like webpages, PDFs, docx files and txt documents. Implemented using Python and deep learning libraries, it features an intuitive browser-based interface for user interaction. Users can provide textual input by simply pasting links to online articles or uploading digital files in multiple formats. Specialized scrapers, parsers and readers extract raw text from these heterogeneous sources. After initial cleaning, this content flows through the automated summarization pipeline residing on the backend server.

The following are the new developments in the proposed system:

1. Built an end-to-end system for multi-source text summarization.
2. Seamless content ingestion from web pages, PDFs, Word docs etc.
3. Implemented latest abstractive summarizers based on BART.
4. Developed intuitive web interface for user input and output.

The system aims to tackle constraints of existing solutions by allowing summarization of diverse document types. A web interface is developed for intuitive user interaction with file/URL inputs and clear outputs. Modular architecture ensures separation of UI, text ingestion logic and deep learning summarizer. Scrapers and parsers extract raw text from uploaded PDFs, Word documents, webpage links while customized readers handle text file input. Cleaning modules then format this multi-source content. The core abstractive pipeline segments text to not exceed model limits. A state-of-the-art BART model from Hugging Face independently summarizes chunks using deep contextual understanding unique to transformers. Aggregation aligns disjointed outputs and filtering streamlines the final summary.

Side-by-side display of the condensed and original versions allows quick qualitative checks on information retention. Tracking saves past summaries while exportable outputs ensure summaries are accessible system wide. The UI allows entering text directly too. Python powers the web framework, NLP preprocessing and PyTorch deployment. Modular codebase eases extensibility, like swapping models or adding new parsers.

System Architecture:

The developed system follows a modular architecture consisting of the user interface, input handler, summarizer and output modules. The graphical user interface built with HTML, CSS and Bootstrap receives different kinds of input from users through text boxes, uploaders and buttons. It passes web links, uploaded files or pasted content to specialized scrapers, parsers and readers in the input handler which extracts raw text. This text is cleaned by removing unwanted elements before flowing to the summarizer. Within the summarizer, key steps include text segmentation into chunks, vectorization and actual model inference leveraging pretrained BART models that generate sentence-level summaries. These are aggregated, redundant content filtered out and formatted to prepare the final summary. This final output is rendered side-by-side to the original input text on the user interface for easily reviewing quality and relevance. The modularity of the system enables extending functionality in the future.

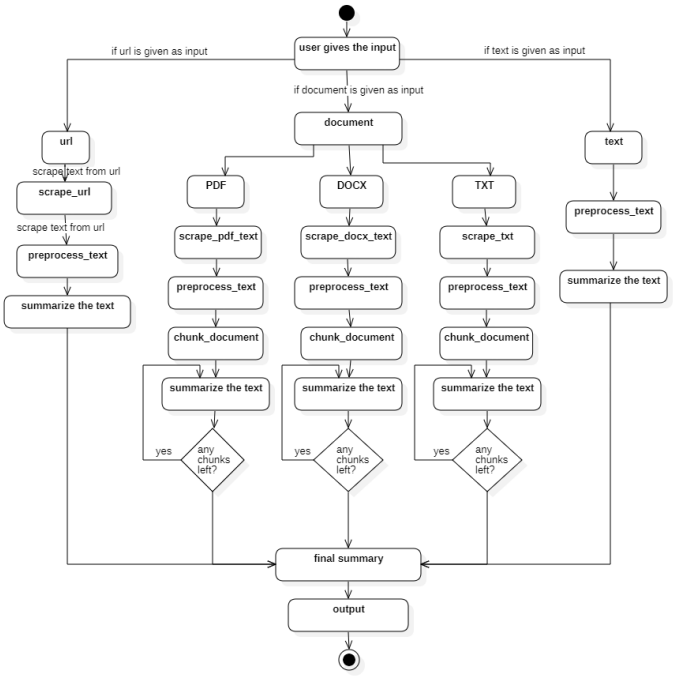


Fig 1. System Architecture

User Interface:

The user interface for the summarization tool is designed to provide users with an intuitive and straightforward way to get summaries of various text inputs. The UI utilizes standard web technologies - HTML, CSS and Bootstrap - to create responsive web pages that adapt seamlessly across devices. Uploading files is a key feature of the interface. Users can upload PDFs, Word documents and text files to be summarized by the tool. Supported file types include the common .pdf, .docx, and .txt formats. This caters to summarizing long-form documents that users already have saved. The file upload functionality includes the necessary widgets and components to allow selecting files from the local file system. Recognizing that most information online exists as webpages, the interface also allows directly pasting URLs of articles to summarize. This leverages a web scraper to extract the main article content from a published web page. Users simply need to insert a link to trigger automatic scraping and summarization.

For summarizing ad hoc content, a text area box allows entering or copying any text snippet. This seamlessly combines free-form manual text input with automated summarization capabilities. The box conveniently places the cursor for starting to type content directly. To streamline selection from the above input methods, radio buttons are incorporated in the interface. The buttons enable choosing one option at a time - either uploading a file, entering a web page link, or directly pasting or typing text. This radio button group provides clears guidance on input types. A defining aspect of the user interface is the side-by-side output of the original excerpt next to the generated summary. Tabular formatting clearly presents the summary against the source for easily reviewing how content is condensed. This assists users in rapidly validating and interpreting summaries.

Input Handler:

The input handler refers to the back-end software components that process various input document types and extract the textual content to prepare it for summarization. The system employs different techniques to handle the diverse formats that users can submit. A key capability is the incorporation of a web scraper to handle webpage links pasted by the user. The scraper is built on the Goose3 library which identifies main article text in web pages - filtering out surrounding boilerplate content like navigation links, ads etc. This allows summarizing long online articles just from their links. To summarize PDF documents, the handler leverages the PyPDF2 PDF parsing library. PDF files contain text alongside formatting in a binary format. PyPDF2 provides capabilities to process the PDF binary, extract text appropriately maintaining layout, and export plain text out. This text stripped of formatting is then used for later summarization. The popular .docx Microsoft Word format is handled using the python-docx library. Similar to PDF processing, this library parses the inner contents of .docx files to pull text out from paragraphs while ignoring embedded styling and macros. The text extracted from paragraphs is output sequentially.

For simple text files, basic file handling functions are used to directly read contents. This builds on native Python I/O routines allowing iteration through lines of any text file bite. Finally, the extracted raw text may contain truncated sentences or awkward characters from embedding special symbols with multiple input formats. Cleaning functions standardize such text through fixes into proper words and sentences - suitable for feeding into the summarizer

Summarizer:

The summarizer module contains the core logic responsible for ingesting lengthy documents and producing concise indicative summaries. Sophisticated NLP techniques enable distilling key points accurately. A key step is intelligently segmenting the input text into chunks of optimal size and context for the neural summarization model to function effectively. The text segmenter leverages NLP constructs to split content maintaining semantic continuity within chunks. Factors like sentence completeness, topics, referencing etc. help determine chunk boundaries.

Summarization itself is handled by a state-of-the-art abstractive BART model for text-to-text generation powered by Hugging Face Transformers. Abstractive capability allows the model to ingest, interpret and rephrase source text through its trained understanding. The BART model has been pretrained on massive text corpuses. To accelerate summarization speed for long documents, the module summarizes text chunks fully in parallel leveraging inference capability on GPU hardware. Each chunk gets summarized independently by the model in a faster distributed manner instead of sequential processing. The aggregate module then takes all output summary snippets corresponding to each input chunk and collates them to prepare a consolidated draft summary. Logical ordering is maintained based on original sequence. Finally, repetition occurs when similar points reoccur among the chunks summarized independently. Filtering compares statements in the collated summary to remove such overlaps. The final polished summary is outputted for presentation.

Implementation:

The summarization system is built leveraging a number of Python software libraries and frameworks that enable diverse capabilities required for end-to-end functionality. The web application backend of the system uses the Flask framework. Flask provides tools for handling HTTP requests, executing Python code, integrating with templates and databases, and routing URLs to appropriate handlers. This powers core request-response workflow. For parsing uploaded documents, PyPDF2 and Python-Docx libraries process PDF and Word .docx binaries respectively to extract text. This allows ingesting files that users submit through the frontend interface. Webpage links are handled by the Goose3 library which implements article scrapers to identify and extract main content from HTML pages. This filters out boilerplate content. The NLTK toolkit provides key text manipulation routines for tasks like cleaning, standardization, segmentation etc. This prepares and preprocesses raw input text before summarization. Actual summarization is enabled through state-of-the-art Transformer based neural networks provided by the HuggingFace library. The networks leverage PyTorch for optimizing and inferencing the BART abstractive model. Rendering frontend UI and dynamic pages is powered by Jinja - a template engine for Python. Jinja injects backend data into HTML templates to generate final pages. Finally, styling and responsively laying out UI components leverages CSS frameworks like Bootstrap to adapt across devices and desktops. Together, the libraries enable building an end-to-end summarization pipeline from document ingestion to UI presentation. The components synergize to extract, process and condense text.

Model Retraining (Optional):

While the summarization system leverages versatile pretrained models that work well across domains, its architecture enables customizing models to a target use case through retraining. Retraining involves collating a domain-specific dataset first. This entails gathering relevant documents and corresponding reference summaries. For custom datasets, text requires intensive cleaning to fix inconsistencies. Useful data pairs connect original content with good quality summaries. The actual retraining process then trains the neural network through multiple epochs - iteratively updating model weights to minimize loss between generated and reference summaries. Tracking loss metrics provides insight into model convergence. Parallelly, evaluation metrics are quantified to assess model performance after each epoch. Statistics like ROUGE and F1 provide objective indicators on how well the customized model captures salient points compared to reference summaries. Finally, plotting epoch vs evaluation metrics helps identify the optimized customized model. Analyzing graphs helps determine the epoch count representing peak validation performance. This tuned domain-specific model is then saved for deployment. The components enable adapting the general summarization capabilities to specialized niches like academia, healthcare, sports etc. This powers use case-targeted summarization reflecting stylistic and topical nuances.

V. RESULTS

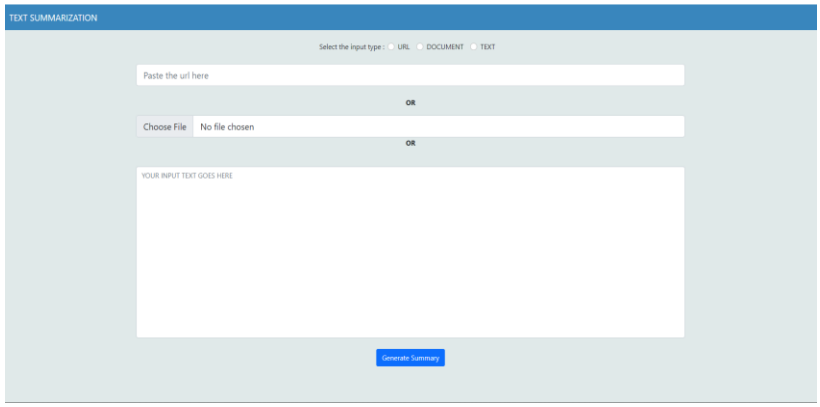
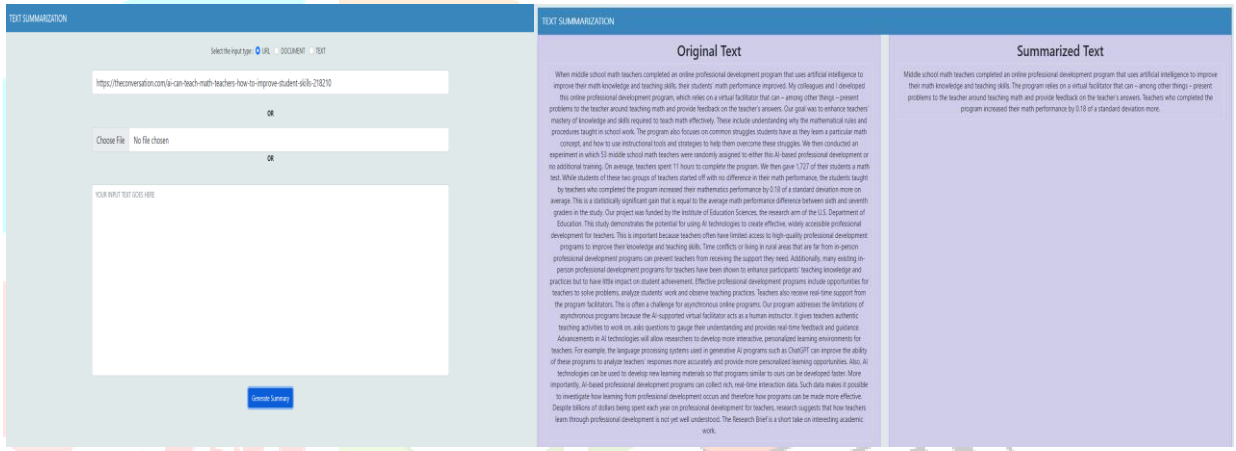


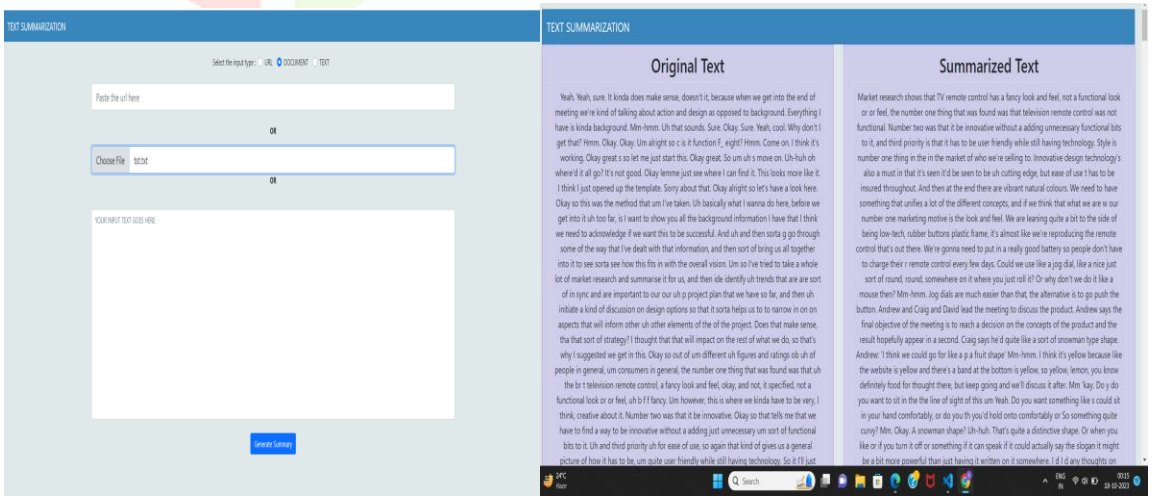
Fig 2. FRONT-END

We have tested this project for different inputs like urls, documents like pdf's, docx , .txt files etc and the text. And the beloe are the different outputs:

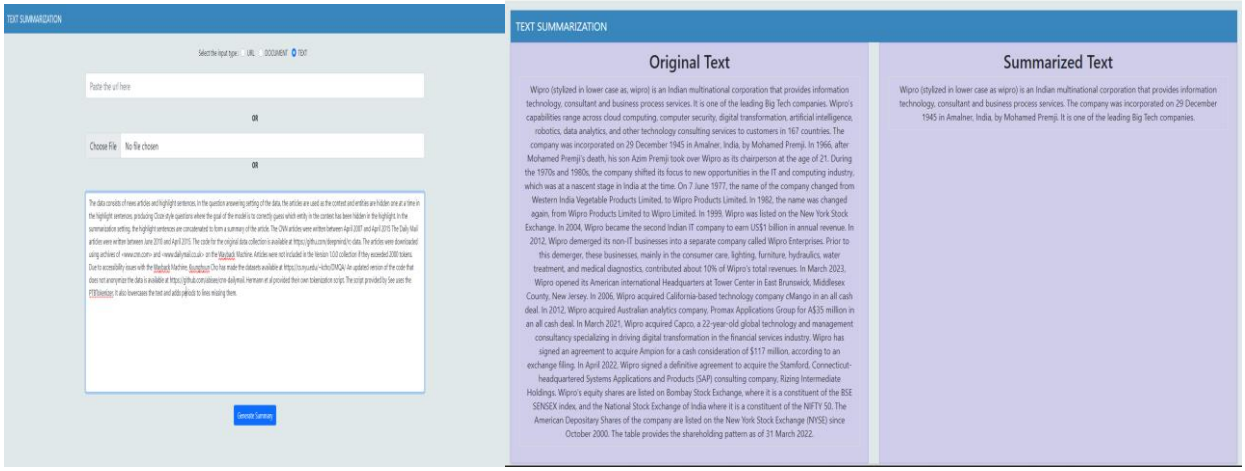
URL AS INPUT:



DOCUMENT AS INPUT(TXT FILE):



TEXT AS INPUT:



We have implemented experiments on the CNN-DailyMail dataset. The CNN / DailyMail Dataset is an English-language dataset containing just over 300k unique news articles as written by journalists at CNN and the Daily Mail. The current version supports both extractive and abstractive summarization, though the original version was created for machine reading and comprehension and abstractive question answering.

Data Fields

- `id`: a string containing the heximal formated SHA1 hash of the url where the story was retrieved from
- `article`: a string containing the body of the news article
- `highlights`: a string containing the highlight of the article as written by the article author

Data Splits

The CNN/DailyMail dataset has 3 splits: *train*, *validation*, and *test*. Below are the statistics for Version 3.0.0 of the dataset.

Dataset Split	Number of Instances in Split
Train	287,113
Validation	13,368
Test	11,490

Fig 3: Dataset Descripiton

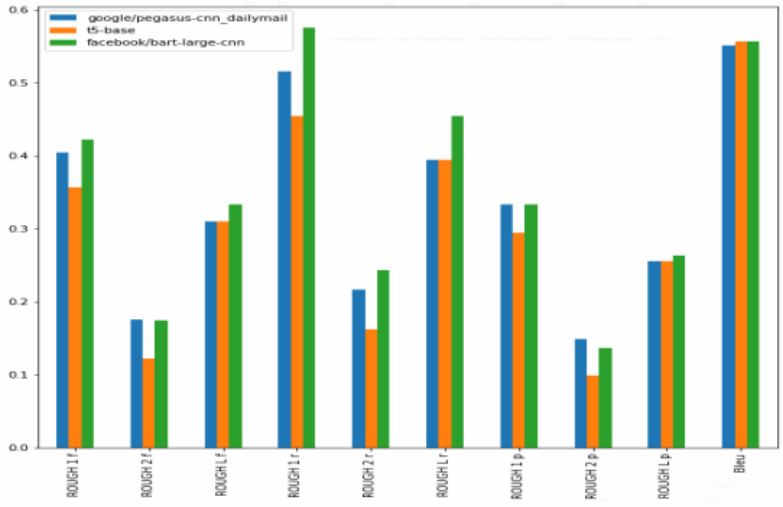


Fig. 4: Performance of three pre-trained models using CNN-dailymail dataset.

	ROUGE									BLEU
	ROUGE-1			ROUGE-2			ROUGE-L			
	R	P	F1	R	P	F1	R	P	F1	
google/pegasus-cnn-dailymail	40.22	32.15	34.60	17.64	13.98	14.97	37.51	30.06	32.32	55.09
T5-base	33.96	30.35	31.10	13.37	11.67	12.00	31.43	28.10	28.79	55.70
facebook/bart-large-cnn	**41.76**	**33.69**	**36.45**	**18.67**	**14.60**	**15.92**	**38.84**	**31.36**	**33.92**	**55.71**

Table 1: Observations of ROUGE and BLEU performance using CNN-dailymail dataset.

	ROUGE									BLEU
	ROUGE-1			ROUGE-2			ROUGE-L			
	R	P	F1	R	P	F1	R	P	F1	
google/pegasus-cnn-dailymail	38.26	25.12	28.39	10.69	7.24	8.05	33.76	22.63	25.36	24.59
T5-base	33.11	26.44	27.88	9.16	7.66	7.72	29.38	23.60	24.82	**26.54**
facebook/bart-large-cnn	**40.63**	**30.57**	**33.02**	**13.16**	**10.10**	**10.58**	**36.38**	**27.69**	**29.78**	25.95

Table 2: Observations of ROUGE and BLEU performance using SAMSum dataset.

	ROUGE									BLEU
	ROUGE-1			ROUGE-2			ROUGE-L			
	R	P	F1	R	P	F1	R	P	F1	
google/pegasus-cnn-dailymail	**26.30**	**48.41**	**30.95**	**11.59**	24.82	**13.99**	**22.95**	42.78	**27.25**	**53.81**
T5-base	21.71	39.45	26.29	7.31	17.71	9.73	18.77	34.56	22.91	54.11
facebook/bart-large-cnn	21.43	46.65	27.82	9.20	**24.99**	12.52	19.53	**42.84**	25.47	58.41

Table 3: Observations of ROUGE and BLEU performance using BillSum dataset.

VI. CONCLUSION

In conclusion, this project successfully developed an end-to-end multi-document abstractive text summarization system with a user-friendly web interface. The system uniquely overcomes input format limitations by enabling summarization of content sourced from diverse documents like webpages, PDFs, text files and DOCX documents. Advanced deep learning models and transformers power the core summarization engine to produce high quality summaries going beyond extraction to generate new phrases and sentences capturing semantic essence. The web portal allows easy interaction from uploading files to obtaining the condensed summary output along with the original text for quick review.

Additional features like tracking past summaries, copying output text and saving summaries in multiple file types further improve user experience. The modular architecture ensures different components like UI, input handlers, summarizer and output can be isolated, modified or extended without impacting other modules. Python enables rapid prototyping with state-of-the-art libraries integrated for web development, CPU/GPU-based deep learning and NLP tasks. The project delivered a usable product combining the strengths of various technologies

while exploring modern solutions to text summarization based on neural networks and transformers.
Future work involves more detailed user studies for qualitative feedback, evaluating summarization quality through metrics like ROUGE and Jensen-Shannon Divergence, exploring extractive summarization methods, and enhancements like multi-language support. On the whole, the project provided great exposure to architecting and implementing an end-to-end NLP pipeline fused with web development.

VII. REFERENCES:

• BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov,Luke Zettlemoyer
• BART Model for Text Summarization : An Analytical Survey and Review Mr. Chandrashekhar Mankar, Adarsh Mundada2 , Sahil Nagrale3 , Pavan Malviya4 , Aniket Sangle5 , Manoj Navrange6
• Y. Chen and Q. Song, "News Text Summarization Method based on BART-TextRank Model," 2021 IEEE 5th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC).
• A Survey of the State-of-the-Art Models in Neural Abstractive Text Summarization AYESHA AYUB SYED1 , FORD LUMBAN GAOL 1 , (Senior Member, IEEE), AND TOKURO MATSUO2,3, (Member, IEEE)
• Review of automatic text summarization techniques & methods Adhika Pramita Widya sari, , Supriadi Rustad a , Guruh Fajar Shidik a , Edi Noersasongko a , Abdul Syukur a , Affandi Affandi
• A Survey of Text Summarization Extractive Techniques,JOURNAL OF EMERGING TECHNOLOGIES IN WEB INTELLIGENCE, VOL. 2, NO. 3, AUGUST 2020
• Automatic Text Summarization Methods: A Comprehensive Review Divakar Yadav1 ,Jalpa Desai, Arun Kumar Yadav2
• A Survey on Automatic Text Summarization Techniques S Hima Bindu Sri and Sushma Rani Dutta 2021 J. Phys.: Conf. Ser. 2040 012044

ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/220974615

Automatic Text Summarization Using a Machine Learning Approach

Conference Paper *in* Lecture Notes in Computer Science · November 2002

DOI: 10.1007/3-540-36127-8_20 · Source: DBLP

CITATIONS
247

READS
13,550

3 authors:

Joel Larocca Neto
Pontifical Catholic University of Paraná
6 PUBLICATIONS **470** CITATIONS

SEE PROFILE

Alex Freitas
University of Kent
300 PUBLICATIONS **15,135** CITATIONS

SEE PROFILE

Celso A. A. Kaestner
Federal University of Technology of Paraná
65 PUBLICATIONS **1,423** CITATIONS

SEE PROFILE

All content following this page was uploaded by Joel Larocca Neto on 13 March 2014.

The user has requested enhancement of the downloaded file.

Automatic Text Summarization using a Machine Learning Approach

Joel Larocca Neto Alex A. Freitas Celso A. A. Kaestner

Pontifical Catholic University of Parana (PUCPR)
Rua Imaculada Conceicao, 1155
Curitiba - PR. 80.215-901. BRAZIL
{joel, alex, kaestner}@ppgia.pucpr.br
http://www.ppgia.pucpr.br/~alex

Abstract. In this paper we address the automatic summarization task. Recent research works on extractive-summary generation employ some heuristics, but few works indicate how to select the relevant features. We will present a summarization procedure based on the application of trainable Machine Learning algorithms which employs a set of features extracted directly from the original text. These features are of two kinds: statistical – based on the frequency of some elements in the text; and linguistic – extracted from a simplified argumentative structure of the text. We also present some computational results obtained with the application of our summarizer to some well known text databases, and we compare these results to some baseline summarization procedures.

1 Introduction

Automatic text processing is a research field that is currently extremely active. One important task in this field is *automatic summarization*, which consists of reducing the size of a text while preserving its information content [9], [21]. A *summarizer* is a system that produces a condensed representation of its input's for user consumption [12].

Summary construction is, in general, a complex task which ideally would involve deep natural language processing capacities [15]. In order to simplify the problem, current research is focused on extractive-summary generation [21]. An extractive summary is simply a subset of the sentences of the original text. These summaries do not guarantee a good narrative coherence, but they can conveniently represent an approximate content of the text for relevance judgement.

A summary can be employed in an *indicative* way – as a pointer to some parts of the original document, or in an *informative* way – to cover all relevant information of the text [12]. In both cases the most important advantage of using a summary is its reduced reading time. Summary generation by an automatic procedure has also other advantages: (i) the size of the summary can be controlled; (ii) its content is determinist; and (iii) the link between a text element in the summary and its position in the original text can be easily established.

In our work we deal with an automatic trainable summarization procedure based on the application of machine learning techniques. Projects involving extractive summary generation have shown that the success of this task depends strongly on the use of heuristics [5], [7]; unfortunately few indicatives are given of how to choose the relevant features for this task. We will employ here statistical and linguistic features, extracted directly and automatically from the original text.

The rest of the paper is organized as follows: section 2 presents a brief review of the text summarization task; in section 3 we describe in detail our proposal, discussing the employed set of features and the general framework of trainable summarizer; in section 4 we relate the computational results obtained with the application of our proposal to a reference document collection; and finally, in section 5 we present some conclusions and outline some envisaged research work.

2 A review of text summarization

An automatic summarization process can be divided into three steps [21]: (1) in the *preprocessing step* a structured representation of the original text is obtained; (2) in the *processing step* an algorithm must transform the text structure into a summary structure; and (3) in the *generation step* the final summary is obtained from the summary structure.

The methods of summarization can be classified, in terms of the level in the linguistic space, in two broad groups [12]: (a) shallow approaches, which are restricted to the syntactic level of representation and try to extract salient parts of the text in a convenient way; and (b) deeper approaches, which assume a semantics level of representation of the original text and involve linguistic processing at some level.

In the first approach the aim of the preprocessing step is to reduce the dimensionality of the representation space, and it normally includes: (i) *stop-word* elimination – common words with no semantics and which do not aggregate relevant information to the task (e.g., "the", "a") are eliminated; (ii) *case folding*: consists of converting all the characters to the same kind of letter case - either upper case or lower case; (iii) *stemming*: syntactically-similar words, such as plurals, verbal variations, etc. are considered similar; the purpose of this procedure is to obtain the stem or radix of each word, which emphasize its semantics.

A frequently employed text model is the vectorial model [20]. After the preprocessing step each text element – a sentence in the case of text summarization – is considered as a N-dimensional vector. So it is possible to use some metric in this space to measure similarity between text elements. The most employed metric is the cosine measure, defined as $\cos\theta = (\langle x.y \rangle) / (|x| \,.\, |y|)$ for vectors x and y, where $(\langle , \rangle)$ indicates the scalar product, and $|x|$ indicates the module of x. Therefore maximum similarity corresponds to $\cos\theta = 1$, whereas $\cos\theta = 0$ indicates total discrepancy between the text elements.

The evaluation of the quality of a generated summary is a key point in summarization research. A detailed evaluation of summarizers was made at the TIPSTER Text Summarization Evaluation Conference (SUMMAC) [10], as part of an effort to standardize summarization test procedures. In this case a reference summary collection was provided by human judges, allowing a direct comparison of the performance of the systems that participated in the conference. The human effort to

elaborate such summaries, however, is huge. Another reported problem is that even in the case of human judges, there is low concordance: only 46 % according to Mitra [15]; and more importantly: the summaries produced by the same human judge in different dates have an agreement of only 55 % [19].

The idea of a "reference summary" is important, because if we consider its existence we can objectively evaluate the performance of automatic summary generation procedures using the classical Information Retrieval (IR) *precision* and *recall* measures. In this case a sentence will be called *correct* if it belongs to the reference summary. As usual, *precision* is the ratio of the number of selected correct sentences over the total number of selected sentences, and *recall* is the ratio of the number of selected correct sentences over the total number of correct sentences. In the case of fixed-length summaries the two measures are identical, since the sizes of the reference and the automatically obtained extractive summaries are identical.

Mani and Bloedorn [11] proposed an automatic procedure to generate reference summaries: if each original text contains an author-provided summary, the corresponding size-K reference extractive summary consists of the K most similar sentences to the author-provided summary, according to the cosine measure. Using this approach it is easy to obtain reference summaries, even for big document collections.

A Machine Learning (ML) approach can be envisaged if we have a collection of documents and their corresponding reference extractive summaries. A trainable summarizer can be obtained by the application of a classical (trainable) machine learning algorithm in the collection of documents and its summaries. In this case the sentences of each document are modeled as vectors of features extracted from the text. The summarization task can be seen as a two-class classification problem, where a sentence is labeled as "correct" if it belongs to the extractive reference summary, or as "incorrect" otherwise. The trainable summarizer is expected to "learn" the patterns which lead to the summaries, by identifying relevant feature values which are most correlated with the classes "correct" or "incorrect". When a new document is given to the system, the "learned" patterns are used to classify each sentence of that document into either a "correct" or "incorrect" sentence, producing an extractive summary. A crucial issue in this framework is how to obtain the relevant set of features; the next section treats this point in more detail.

3 A trainable summarizer using a ML approach

We concentrate our presentation in two main points: (1) the set of employed features; and (2) the framework defined for the trainable summarizer, including the employed classifiers.

A large variety of features can be found in the text-summarization literature. In our proposal we employ the following set of features:

(a) Mean-TF-ISF. Since the seminal work of Luhn [9], text processing tasks frequently use features based on IR measures [5], [7], [23]. In the context of IR, some very important measures are term frequency (TF) and term frequency × inverse document frequency (TF-IDF) [20]. In text summarization we can employ the same idea: in this case we have a single document d, and we have to select a set of relevant sentences to be included in the extractive summary out of all sentences in d. Hence,

the notion of a collection of documents in IR can be replaced by the notion of a single document in text summarization. Analogously the notion of document – an element of a collection of documents – in IR, corresponds to the notion of sentence – an element of a document – in summarization. This new measure will be called term frequency × inverse sentence frequency, and denoted TF-ISF(*w,s*) [8].The final used feature is calculated as the mean value of the TF-ISF measure for all the words of each sentence.

(b) Sentence Length. This feature is employed to penalize sentences that are too short, since these sentences are not expected to belong to the summary [7]. We use the normalized length of the sentence, which is the ratio of the number of words occurring in the sentence over the number of words occurring in the longest sentence of the document.

(c) Sentence Position. This feature can involve several items, such as the position of a sentence in the document as a whole, its the position in a section, in a paragraph, etc., and has presented good results in several research projects [5], [7], [8], [11], [23]. We use here the percentile of the sentence position in the document, as proposed by Nevill-Manning [16]; the final value is normalized to take on values between 0 and 1.

(d) Similarity to Title. According to the vectorial model, this feature is obtained by using the title of the document as a "query" against all the sentences of the document; then the similarity of the document's title and each sentence is computed by the cosine similarity measure [20].

(e) Similarity to Keywords. This feature is obtained analogously to the previous one, considering the similarity between the set of keywords of the document and each sentence which compose the document, according to the cosine similarity.

For the next two features we employ the concept of text cohesion. Its basic principle is that sentences with higher degree of cohesion are more relevant and should be selected to be included in the summary [1], [4], [11], [15].

(f) Sentence-to-Sentence Cohesion. This feature is obtained as follows: for each sentence *s* we first compute the similarity between *s* and each other sentence *s'* of the document; then we add up those similarity values, obtaining the raw value of this feature for *s*; the process is repeated for all sentences. The normalized value (in the range [0, 1]) of this feature for a sentence *s* is obtained by computing the ratio of the raw feature value for *s* over the largest raw feature value among all sentences in the document. Values closer to 1.0 indicate sentences with larger cohesion.

(g) Sentence-to-Centroid Cohesion. This feature is obtained for a sentence *s* as follows: first, we compute the vector representing the centroid of the document, which is the arithmetic average over the corresponding coordinate values of all the sentences of the document; then we compute the similarity between the centroid and each sentence, obtaining the raw value of this feature for each sentence. The normalized value in the range [0, 1] for *s* is obtained by computing the ratio of the raw feature value over the largest raw feature value among all sentences in the document. Sentences with feature values closer to 1.0 have a larger degree of cohesion with respect to the centroid of the document, and so are supposed to better represent the basic ideas of the document.

For the next features an approximate argumentative structure of the text is employed. It is a consensus that the generation and analysis of the complete rethorical structure of a text would be impossible at the current state of the art in text processing. In spite of this, some methods based on a surface structure of the text have been used

to obtain good-quality summaries [23], [24]. To obtain this approximate structure we first apply to the text an agglomerative clustering algorithm. The basic idea of this procedure is that similar sentences must be grouped together, in a bottom-up fashion, based on their lexical similarity. As result a hierarchical tree is produced, whose root represents the entire document. This tree is binary, since at each step two clusters are grouped. Five features are extracted from this tree, as follows:

(h) Depth in the tree. This feature for a sentence *s* is the depth of *s* in the tree.

(i) Referring position in a given level of the tree (positions 1, 2, 3, and 4). We first identify the path form the root of the tree to the node containing *s*, for the first four depth levels. For each depth level, a feature is assigned, according to the direction to be taken in order to follow the path from the root to *s*; since the argumentative tree is binary, the possible values for each position are: left, right and none, the latter indicates that *s* is in a tree node having a depth lower than four.

(j) Indicator of main concepts. This is a binary feature, indicating whether or not a sentence captures the main concepts of the document. These main concepts are obtained by assuming that most of relevant words are nouns. Hence, for each sentence, we identify its nouns using a part-of-speech software [3]. For each noun we then compute the number of sentences in which it occurs. The fifteen nouns with largest occurrence are selected as being the main concepts of the text. Finally, for each sentence the value of this feature is considered "true" if the sentence contains at least one of those nouns, and "false" otherwise.

(k) Occurrence of proper names. The motivation for this feature is that the occurrence of proper names, referring to people and places, are clues that a sentence is relevant for the summary. This is considered here as a binary feature, indicating whether a sentence *s* contains (value "true") at least one proper name or not (value "false"). Proper names were detected by a part-of-speech tagger [3].

(l) Occurrence of anaphors. We consider that anaphors indicate the presence of non-essential information in a text: if a sentence contains an anaphor, its information content is covered by the related sentence. The detection of anaphors was performed in a way similar to the one proposed by Strzalkowski [22]: we determine whether or not certain words, which characterize an anaphor, occur in the first six words of a sentence. This is also a binary feature, taking on the value "true" if the sentence contains at least one anaphor, and "false" otherwise.

(m) Occurrence of non-essential information. We consider that some words are indicators of non-essential information. These words are speech markers such as "because", "furthermore", and "additionally", and typically occur in the beginning of a sentence. This is also a binary feature, taking on the value "true" if the sentence contains at least one of these discourse markers, and "false" otherwise.

The ML-based trainable summarization framework consists of the following steps:

1. We apply some standard preprocessing information retrieval methods to each document, namely stop-word removal, case folding and stemming. We have employed the stemming algorithm proposed by Porter [17].
2. All the sentences are converted to its vectorial representation [20].
3. We compute the set of features described in the previous subsection. Continuous features are discretized: we adopt a simple "class-blind" method, which consists of separating the original values into equal-width intervals. We did some experiments with different discretization methods, but surprisingly the selected method, although simple, has produced better results in our experiments.

4. A ML trainable algorithm is employed; we employ two classical algorithms, namely C4.5 [18] and Naive Bayes [14]. As usual in the ML literature, we employ these algorithms trained on a training set and evaluated on a separate test set.

The framework assumes, of course, that each document in the collection has a reference extractive summary. The "correct" sentences belonging to the automatically produced extractive summary are labeled as "positive" in classification/data mining terminology, whereas the remaining sentences are labeled as "negative". In our experiments the extractive summaries for each document were automatically obtained, by using an author-provided non-extractive summary, as explained in section 2.

4 Computational results

As previously mentioned, we have used two very well-known ML classification algorithms, namely Naive Bayes [14] and C4.5 [18]. The former is a Bayesian classifier which assumes that the features are independent from each other. Despite this unrealistic assumption, the method presents good results in many cases, and it has been successfully used in many text mining projects. C4.5 is a decision-tree algorithm that is frequently employed for comparison purposes with other classification algorithms, particularly in the data mining and ML communities.

We did two series of experiments: in the first one, we employed automatically-produced extractive summaries; in the second one, manually-produced summaries were employed. In all the experiments we have used a document collection available in the TIPSTER document base [6]. This collection consists of texts published in several magazines about computers, hardware, software, etc., which have sizes varying from 2 Kbytes to 64 Kbytes. Due to our framework, we used only documents which have an author-provided summary, and a set of keywords. The whole TIPSTER document base contained 33,658 documents with these characteristics. A subset of these documents was randomly selected for the experiments to be reported in this section.

In the first experiment, using automatically-generated reference extractive summaries, we employed four text-summarization methods, as follows:

(a) Our proposal (features as described in section 3) using C4.5 as the classifier;

(b) Our proposal using Naive Bayes as the classifier.

(c) First Sentences (used as a baseline summarizer): this method selects the first *n* sentences of the document, where *n* is determined by the desired compression rate, defined as the ratio of summary length to source length [12], [21]. Although very simple, this procedure provides a relatively strong baseline for the performance of any text-summarization method [2].

(d) Word Summarizer (WS): Microsoft's WS is a text summarizer which is part of Microsoft Word, and it has been used for comparison with other summarization methods by several authors [1], [13]. This method uses non-documented techniques to perform an "almost extractive" summary from a text, with the summary size specified by the user.

The WS has some characteristics that are different from the previous methods: the specified summary size refers to the number of characters to be extracted, and some sentences can be modified by WS. In our experiments due to these characteristics a

direct comparison between WS and the other methods is not completely fair: (i) the summaries generated by WS can contain a few more or a few less sentences than the summaries produced by the other methods; (ii) in some cases it will not be possible to compute an exact match between a sentence selected by WS and an original sentence; in these cases we ignore the corresponding sentences.

It is important to note that only our proposal is based on a ML trainable summarizer; the two remaining methods are not trainable, and were used mainly as baseline for result comparison.

The document collection used in this experiment consisted of 200 documents, partitioned into disjoints training and test sets with 100 documents each. The training set contained 25 documents of 11 Kbytes, 25 documents of 12 Kbytes, 25 documents of 16 Kbytes, and 25 documents of 31 Kbytes. The average number of sentences per document is 129.5, since there are in total 12,950 sentences in the training set. The test set contained 25 documents of 10 Kbytes, 25 documents of 13 Kbytes, 25 documents of 15 Kbytes, and 25 documents of 28 Kbytes. The average number of sentences per document is 118.6, since there are in total 11,860 sentences in the test set.

Table 1 reports the results obtained by the four summarizers. We consider compression rates of 10 % and 20 %. The performance is expressed in terms of precision / recall values, expressed in percentage (%), and the corresponding standard deviations are indicated after the "±" symbol. The best obtained results are shown in boldface.

Table 1. Results for training and test sets composed by automatically-produced summaries

Summarizer	Compression rate: 10%		Compression rate: 20%	
	Precision	Recall	Precision	Recall
Trainable-C4.5	22.36 ± 1.48		34.68 ± 1.01	
Trainable-Bayes	**40.47 ± 1.99**		**51.43 ± 1.47**	
First-Sentences	23.95 ± 1.60		32.03 ± 1.36	
Word-Summarizer	26.13 ± 1.21	34.44 ± 1.56	38.80 ± 1.14	43.67 ± 1.30

We can draw the following conclusions from this experiment: (1) the values of precision and recall for all the methods are significantly higher with the rate of 20% than with the compression rate of 10%; this is a expected result, since the larger the compression rate, the larger the number of sentences to be selected for the summary, and then the larger the probability that a sentence selected by a summarizer matches with a sentence belonging to the extractive summary; (2) the best results were obtained by our trainable summarizer with Naive Bayes classifier for both compression rates; using the same features, but with the C4.5 as classifier, the obtained results were poor: the results are similar to the First-Sentences and Word Summarizer baselines.

The latter result offers us an interesting lesson: most research projects on trainable summarizers focus on the proposal of new features for classification, trying to produce more and more elaborate statistics-based or linguistics-based features, but

they usually employ a single classifier in the experiments. Normally "conventional" classifiers are used. Our results indicate that researchers should concentrate their attention in the study of more elaborate classifiers, tailored for the text-summarization task, or at least evaluate and select the best classifier among the conventional ones already available.

In the second experiment we employ in the test step summaries manually produced by a human judge. We emphasize that in the training phase of our proposal we have used the same database of automatically-generated summaries employed in the previous experiment.

The test database was composed of 30 documents, selected at random from the original document base. The manual reference summaries were produced by a human judge – a professional English teacher with many years of experience –specially hired for this task. For the compression rates of 10 % and 20% the same four summarizers of the first experiment were compared. The obtained results are presented in Table 2.

Table 2. Results for training set composed by automatically-produced summaries and test set composed by manually-produced summaries

Summarizer	Compression rate: 10%		Compression rate: 20%	
	Precision	Recall	Precision	Recall
Trainable-C4.5	24.38 ± 2.84		31.73 ± 2.41	
Trainable-Bayes	**26.14 ± 3.32**		**37.50 ± 2.29**	
First-Sentences	18.78 ± 2.54		28.01 ± 2.08	
Word-Summarizer	14.23 ± 2.17	17.24 ± 2.56	24.79 ± 2.22	27.56 ± 2.41

Here again the best results were obtained by our proposal using the Naive Bayes algorithm as classifier. Similar to the previous experiment, results for 20% of compression were superior to the results produced with 10% of compression.

In order to verify the consistency between the two experiments we have compared the manually-produced summaries and the automatically-produced ones. We considered here the manually-produced summaries as a reference, and we calculated the precision and recall for the automatically produced summaries of the same documents. Obtained results are presented in Table 3. These results are consistent with the ones presented by Mitra [15], and indicate that the degree of dissimilarity between a manually-produced summary and an automatically-produced summary in our experiments is comparable to the dissimilarity between two summaries produced by different human judges.

Table 3. Comparison between automatically-produced and manually-produced summaries

	Precision / Recall
Compression rate: 10%	30.79 ± 3.96
Compression rate: 20%	42.98 ± 2.42

5 Conclusions and future research

In this work we have explored the framework of using a ML approach to produce trainable text summarizers, in a way which was proposed a few years ago by Kupiec [7]. We have chosen this research direction because it allows us to measure the results of a text summarization algorithm in an objective way, similar to the standard evaluation of classification algorithms found in the ML literature. This avoids the problem of subjective evaluation of the quality of a summary, which is a central issue in the text summarization research.

We have performed an extensive investigation of that framework. In our proposal we employ a trainable summarizer that uses a large variety of features, some of them employing statistics-oriented procedures and others using linguistics-oriented ones. For the classification task we have used two different well known classification algorithms, namely the Naive Bayes algorithm and the C4.5 decision tree algorithm. Hence, it was possible to analyze the performance of two different text-summarization procedures. The performance of these procedures was compared with the performance of two non-trainable, baseline methods.

We did basically two kind of experiments: in the first one we considered automatically-produced summaries for both the training and test phases; in the second experiment we used automatically-produced summaries for training and manually-produced summaries for testing. In general the trainable method using Naive Bayes classifier significantly outperformed all the baseline methods.

An interesting finding of our experiments was that the choice of the classifier (Naive Bayes versus C4.5) strongly influenced the performance of the trainable summarizer. We intend to focus mainly on the development of a new or extended classification algorithm tailored for text summarization in our future research work.

6 References

1. Barzilay, R. ; Elhadad, M. Using Lexical Chains for Text Summarization. In Mani, I.; Maybury, M. T. (eds.). In *Proceedings of the ACL/EACL-97 Workshop on Intelligent Scalable Text Summarization*, Association of Computional Linguistics (1997).
2. Brandow, R.; Mitze, K., Rau, L. Automatic condensation of electronic publications by sentence selection. *Information Processing and Management* 31(5) (1994) 675-685
3. Brill, E. A simple rule-based part-of-speech tagger. In *Proceedings of the Third Conference on Applied Comp. Linguistics*. Assoc. for Computational Linguistics (1992)
4. Carbonell, J. G.; Goldstein, J. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of SIGIR-98* (1998)
5. Edmundson, H. P. New methods in automatic extracting. *Journal of the Association for Computing Machinery* 16 (2) (1969) 264-285
6. Harman, D. Data Preparation. In Merchant, R. (ed.). *The Proceedings of the TIPSTER Text Program Phase I*. Morgan Kaufmann Publishing Co. (1994)
7. Kupiec, J. ; Pedersen, J. O.; Chen, F. A trainable document summarizer. In *Proceedings of the 18th ACM-SIGIR Conference, Association of Computing Machinery* (1995) 68-73
8. Larocca Neto, J.; Santos, A. D.; Kaestner, C.A.; Freitas, A.A.. Document clustering and text summarization. *Proc. of 4th Int. Conf. Practical Applications of Knowledge Discovery and Data Mining (PADD-2000)* London: The Practical Application Company (2000) 41-55

9. Luhn, H. The automatic creation of literature abstracts. *IBM Journal of Research and Development* 2(92) (1958) 159-165
10. Mani, I.; House, D.; Klein, G.; Hirschman, L.; Obrsl, L.; Firmin, T.; Chrzanowski, M.; Sundheim, B. *The TIPSTER SUMMAC Text Summarization Evaluation.* MITRE Technical Report MTR 98W0000138. The MITRE Corporation (1998)
11. Mani, I.; Bloedorn, E. Machine Learning of Generic and User-Focused Summarization. In *Proceedings of the Fifteenth National Conference on AI (AAAI-98)* (1998) 821-826
12. Mani, I. *Automatic Summarization*. J.Benjamins Publ. Co. Amsterdam Philadelphia (2001)
13. Marcu, D. Discourse trees are good indicators of importance in text. In Mani., I.; Maybury, M. (eds.). *Adv. in Automatic Text Summarization*. The MIT Press (1999) 123-136
14. Mitchell, T. *Machine Learning*. McGraw-Hill (1997)
15. Mitra, M.; Singhal, A.; Buckley, C. Automatic text summarization by paragraph extraction. In *Proceedings of the ACL'97/EACL'97 Workshop on Intelligent Scalable Text Summarization*. Madrid (1997)
16. Nevill-Manning, C. G. ; Witten, I. H. Paynter, G. W. et al. KEA: Practical Automatic Keyphrase Extraction. *ACM DL 1999* (1999) 254-255
17. Porter, M.F. An algorithm for suffix stripping. *Program 14*, 130-137. 1980. Reprinted in: Sparck-Jones, K.; Willet, P. (eds.) *Readings in Information Retrieval*. Morgan Kaufmann (1997) 313-316
18. Quinlan, J. *C4.5: Programs for Machine Learning*. Morgan Kaufmann Sao Mateo California (1992)
19. Rath, G. J. ; Resnick A. ; Savvage R. The formation of abstracts by the selection of sentences. *American Documentation* 12 (2) (1961) 139-141
20. Salton, G.; Buckley, C. Term-weighting approaches in automatic text retrieval. *Information Processing and Management 24,* 513-523. 1988. Reprinted in: Sparck-Jones, K.; Willet, P. (eds.) *Readings in I.Retrieval*. Morgan Kaufmann (1997) 323-328
21. Sparck-Jones, K. Automatic summarizing: factors and directions. In Mani, I.; Maybury, M. *Advances in Automatic Text Summarization*. The MIT Press (1999) 1-12
22. Strzalkowski , T.; Stein, G.; Wang, J.; Wise, B. A Robust Practical Text Summarizer. In Mani, I.; Maybury, M. (eds.), *Adv. in Autom. Text Summarization.* The MIT Press (1999)
23. Teufel, S.; Moens, M. Argumentative classification of extracted sentences as a first step towards flexible abstracting. In Mani, I.; Maybury M. (eds.). *Advances in automatic text summarization.* The MIT Press (1999)
24. Yaari, Y. Segmentation of Expository Texts by Hierarchical Agglomerative Clustering. *Technical Report*, Bar-Ilan University Israel (1997)

View publication stats

IJARCCE

ISSN (Online) 2278-1021
ISSN (Print) 2319-5940

International Journal of Advanced Research in Computer and Communication Engineering
ISO 3297:2007 Certified
Vol. 7, Issue 1, January 2018

Machine Learning Approach for Automatic Text Summarization Using Neural Networks

Meetkumar Patel[1], Adwaita Chokshi[1], Satyadev Vyas[2], Khushbu Maurya[3]

Student, Department of Computer Engineering, Gujarat Technological University, Ahmedabad, India[1]
Head of the Department, Department of Computer Engineering, Gujarat Technological University, Ahmedabad, India[2]
Professor, Department of Computer Engineering, Gujarat Technological University, Ahmedabad, India[3]

Abstract: Machine learning and deep learning, as we know, have started ruling over almost every field in the computing industry and so, has revolutionized the process of text summarization too. Automatic text summarization is an advancing realm of the natural language processing research in which concise textual summaries are generated from lengthy input documents. Extensive research has been carried out on how automatic summarization can be prosecuted through various extractive and abstractive techniques. In this paper, we address all the approaches to text summarization and present the modus operandi of an Architecture called Encoder-Decoder, under the machine learning approach. Moreover, we propose several novel implementation models for this architecture, in Keras and TensorFlow that consists of various machine learning and deep learning neural network libraries.

Keywords: Machine Learning, Text Summarization, Neural Networks, Deep Learning, Keras, TensorFlow.

I. INTRODUCTION

Text mining, to begin with, deploys some of the techniques of Natural Language Processing (NLP) such as Parts-Of-Speech (POS) tagging, N-grams, parsing, tokenization, etcetera to carry out text analysis. It includes tasks like automatic keyword extraction and text summarization. Particularizing furthermore, text summarization is the process of shortening a text document with a computer program, in order to generate a summary with the major points extracted out of the original document. The main idea behind text summarization, is to elicit large quantities of text to derive high-quality abridged information, which preserves its primary meaning withal. Various variables that are vital for the recapitulation of an intelligible summary generally include length, syntax and the style of writing. Text summarization methods can be roughly divided into two categories: extractive summarization and abstractive summarization [1].Extractive summarization mines key sentences or phrases from the source documents and clutch them to produce a summary without changing the source text. Here, the sentences are generally in the same order as in the text of the source document. On the contrary, abstractive summarization entails understanding the source text by using semantic methods to examine and construe the text. This approach targets to generate a more generalized summary that depicts the information in a succinct way, and usually requires advanced language generation and compression techniques [2].

At present, there is tremendous amount of data and information available on every platform; but a human hardly gets any time, even to sift through a newspaper. Going through heaps of documents and information can be challenging and time consuming at the same time. Without a proper summary or abstract, it can get really tedious, just to get the gist of what someone is talking about in a paper or a report. Hence, text summarization comes here to the save, as it minimizes the reading time by extracting salient details from a document and condensing it into a summary. In this way it becomes easy to quickly assess whether or not a document is worth reading and investing time into. This comes in very handy for analysts, business leaders, paralegals and academic researchers as they have to comb through numerous documents every day.

Amongst the methods briefed above, in this paper, we have discussed about the machine learning approach that uses artificial neural networks to generate summaries of arbitrary length articles. Specifically, the Encoder-Decoder recurrent neural network (RNN) architecture developed for machine translation has found out to generate promising results when applied to the problem of text summarization. The architecture consists of two neural networks working in parallel simultaneously - an encoder that takes the input sequence and produces a vector output and the decoder that takes the previous vector output as its input and generates the final output sequence. The paper covers a general overview of the approaches to text summarization initially; and then details on the Encoder-Decoder model of the machine learning approach along with its implementation in Keras using TensorFlow.

IJARCCE

ISSN (Online) 2278-1021
ISSN (Print) 2319-5940

International Journal of Advanced Research in Computer and Communication Engineering
ISO 3297:2007 Certified
Vol. 7, Issue 1, January 2018

II. RELATED WORK

Text summarization by the means of natural language processing, dates back to the late 1950s. The early text summarization techniques were based on statistical methods as published in [3], in 1958. These methods mainly involved selecting large blocks of text for generating relative and rational abstracts for the same. Furthermore, this work progressed to better and more persuasive results with the help of graph based ranking model for text processing as elaborated in [4] and with Maximal Marginal Relevance (MMR) criterion as detailed in [5]. Meanwhile, evaluation measures like Recall-Oriented Understudy for Gisting Evaluation (ROUGE) and Bilingual Evaluation Understudy (BLEU) were invented that determined how well an automatic summary covered the matter, present in an original text. Also, different datasets like the DUC series, Medline, TAC series etcetera were developed so that comparison and contrasting of various summarization methods would be possible.

Generally, extractive text summarization process was primarily used. But gradually, the practice of abstractive text summarization has brought a momentous change in this field. This might be because of the techniques that are being used under the categories of structure based methods and the semantic based methods that condenses a text more strongly than the extraction methods. It chiefly involves converting a text into pre-processed form before finally converting it to a summary, as explained in [6] for a particular Indian language. There are comparatively lesser works done in this process as it requires the usage of natural language generation technology, which is harder to develop.

The most thriving technique from all, has been the usage of deep neural networks that are tremendously powerful machine learning models and can achieve excellent performance in sequence learning, because of their parallel computation. The Recurrent Neural Network (RNN) [7][8] is a natural generalization of feedforward neural networks to sequences that computes a sequence of outputs on the base of a given set of inputs. This is rightly elucidated in [9] which also debunks how RNN proves to be inefficient eventually, because of the resulting long term dependencies which is why the concept of Long Short-Term Memory (LSTM), that learns problems with long range temporal dependencies has found out to be better. As far as our research work goes, we have used encoder-decoder architecture for text summarization using Keras and Tensorflow libraries for neural networks and DUC 2005, NewsIR ’16 and NIPS conference articles for the dataset.

III. TEXT SUMMARIZATION APPROACHES

Based on the literature, text summarization approaches can be relegated into five types, namely statistical based, machine learning based, coherent based, graph based, algebraic based as shown in Figure

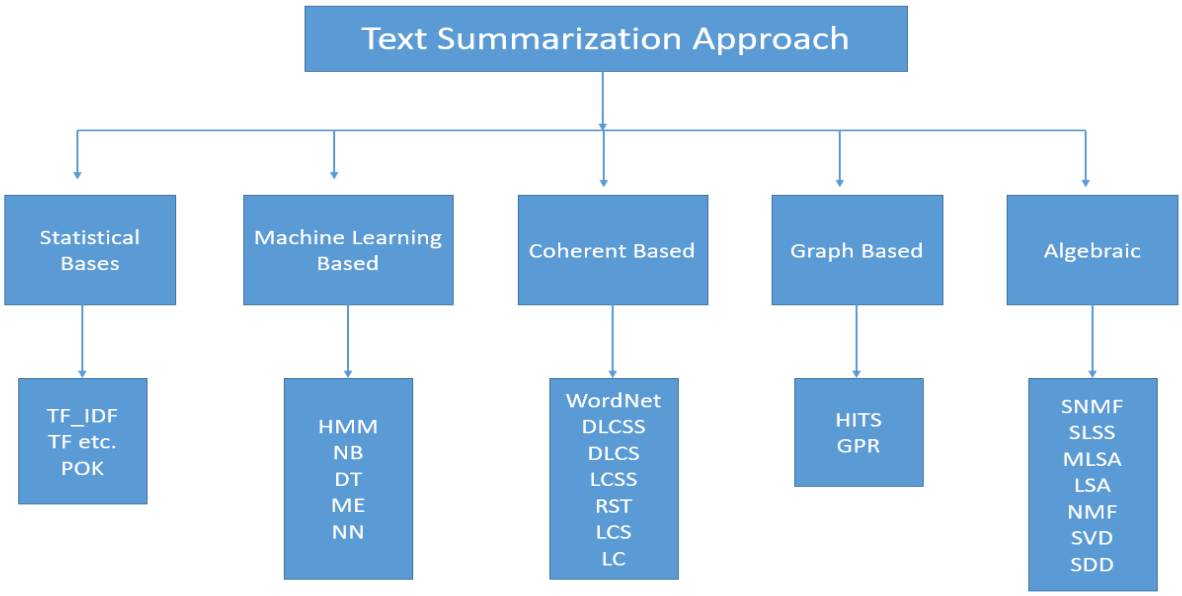


FIG. 1. TEXT SUMMARIZATION APPROACHES

IJARCCE
ISSN (Online) 2278-1021
ISSN (Print) 2319-5940
International Journal of Advanced Research in Computer and Communication Engineering
ISO 3297:2007 Certified
Vol. 7, Issue 1, January 2018

TABLE I
LIST OF ABBREVIATIONS USED IN CLASSIFICATION OF TEXT SUMMARIZATION APPROACHES

Abbreviations	Meaning
TF	Term Frequency
IF-IDF	Term Frequency-inverse document frequency
POK	Position of a Keyword
HMM	Hidden Markov Model
DT	Decision Trees
ME	Maximum Entropy
NN	Neural Networks
NB	Naïve Bayes
DLCSS	Direct lexical chain span score
DLCS	Direct lexical chain score
LCSS	Lexical chain span score
LCS	Lexical chain score
RST	Rhetorical Structure Theory
LC	Lexical chain
GPR	Google's Pagerank
HITS	Hyperlinked Induced Topic Search
MLSA	Meta Latent Semantic Analysis
SNMF	Symmetric nonnegative matrix factorization
SLSS	Sentence level semantic analysis
LSA	Latent Semantic Analysis
NMF	Non-Negative Matrix factorization
SVD	Singular Value Decomposition
SDD	Semi-Discrete Decomposition

1. ***Statistical Based Approach***

This approach is very simple and often utilized for keyword extraction from the documents. There is no predefined dataset required. In this approach we use word frequency, uppercase words, sentence length, keywords, position in complete text and phrase structure[10][11][12].

2. ***Machine learning Based Approach***

A trainable text summarizer can be obtained by the application of a trainable machine learning algorithm. Basically it is a feature dependent approach and we need annotated dataset to train the models [01]. There are various popular machine learning approaches namely, Decision trees [16][17], Nave Bayes [13][14][15], Hidden Markov Model[18][19][20], Maximum Entropy [21][22][23][24][25], Neural Networks [26][27][28], and Support Vector Machine [29][30][31] (SVM) etc.

3. ***Coherent Based Approach***

Basically, a coherent based approach deals with the cohesion relation between the words. There are various cohesion relations among elements in a text like reference, ellipsis, substitution, conjunction, and lexical cohesion [32]. We can see the classification of this approach in figure 1.

4. ***Graph Based Approach***

Graph based approach mainly introduces two popular approaches for text summarization namely, Google's PageRank (GPR)[33][35][36] and Hyperlinked Induced Topic Search (HITS)[33][34]. Google PageRank is used by Google Search to rank websites in their search engine results. GPR is a way of measuring the importance of webpages[02].

5. ***Algebraic Approach***

In algebraic approach we use algebraic theories like, matrix, Eigen vectors, transpose of matrix, etc. There are various algorithms used for text summarization using this approach like LSA- Latent Semantic Analysis[37][38][39], SNMF-Symmetric nonnegative matrix factorization[42], MLSA-Meta Latent Semantic Analysis[40][41], SLSS-Sentence level

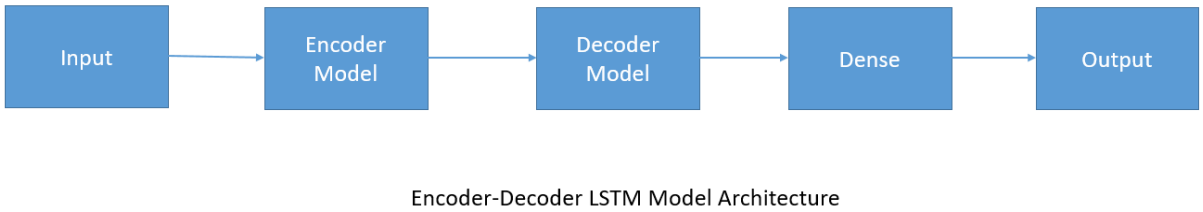
IJARCCE
ISSN (Online) 2278-1021
ISSN (Print) 2319-5940
International Journal of Advanced Research in Computer and Communication Engineering
ISO 3297:2007 Certified
Vol. 7, Issue 1, January 2018

semantic analysis[42], NMF-Non Negative Matrix factorization[43], SVD-Singular Value Decomposition[44], SDD-Semi Discrete Decomposition[45], etcetera.

IV. METHODOLOGY

A. *Encoder-Decoder LSTM Architecture*

This architecture is a way of organizing recurrent neural networks for sequence prediction problems, which have multiple number of inputs, outputs, or even both. The architecture mainly involves two major components which are: an encoder and a decoder. Encoder, chiefly reads the entire input sequence in a go and encodes it into an internal representation, sometimes in a fixed-length vector, also called the context vector. The decoder reads the encoded input sequence resulted from the encoder and generates an output sequence. Both the sub models are trained in parallel and at the same time.



Encoder-Decoder LSTM Model Architecture

FIG. 2. ENCODER-DECODER LSTM MODEL ARCHITECTURE

Encoder-Decoder LSTM architecture was designed specifically for sequence to sequence problems. Mainly this model was developed for natural language processing problems where it demonstrated state-of-the-art performance, particularly in the area of text translation called statistical machine translation. State-of-the-art models look up embeddings of their inputs and pass them through a bidirectional recurrent neural network to produce the initial hidden state.

B. *Text Summarization Encoders*

Encoders, in the model, are like that brain of a human where major complexities dwell. This is because the encoders have the key function of capturing the tenor of the original document and generating an intermediate representation as an output that can be fed to the decoders for the final output. There are several types of encoders that can be used; albeit some of them are more common in usage such as bidirectional recurrent neural networks (Bi-RNNs). LSTMs in particular, are generally preferred while using Bi-RNNs for text summarization. In cases where RNNs are used in the encoder, word embedding is often used to offer a distributed representation of the words[52][49]50]51].

C. *Text Summarization Decoders*

The decoders are responsible for the generation of words in the output sequence, when two sources of information are available: a) a Context Vector, which is an encoded representation of the source document in a vector format, resulted as an output of the encoder and b) a Generated Sequence, which is an already abstracted word or sequence of words. Specifying for a simple Encoder-Decoder Architecture, the Context Vector may be a fix-length encoding structure or a more comprehensible form that might be filtered by the means of an attention mechanism. The distributed representation of each word generated via a word embedding is provided to the Generated Sequence. The process generally involves providing the model with a special start-of-sequence token, in order to generate the first word; and also an end-of-sequence token to stop the process [53][54].

V. DATASETS

We have used three different datasets for training our models:

A. DUC 2005

While many datasets are available that approximate the idea of summaries by using abstracts or headlines, this classic dataset from the Document Understanding Conference [11] contains multiple human generated labels per document that were explicitly constructed as summaries. The DUC dataset contains 500 documents of which we used 432 news articles with unique beginnings, with 4 to 8 model summaries each. While these are high-quality gold-standard summaries, the dataset is too small to use as an exclusive training set. Thus it is often used as a test set only; we used it both for training in our smaller models and for testing for all models.

IJARCCE
ISSN (Online) 2278-1021
ISSN (Print) 2319-5940
International Journal of Advanced Research in Computer and Communication Engineering
ISO 3297:2007 Certified
Vol. 7, Issue 1, January 2018

B. NewsIR '16

This is a "Signal Media One-Million News Articles Dataset" [46] dataset that contains about a million news articles and their titles collected from a wide range of sources. We filtered this dataset for particular news articles category, ("media-type": "News") in order to collate it with the DUC dataset, leaving 572,154 article-title pairs.

C. NIPS articles, abstracts & titles

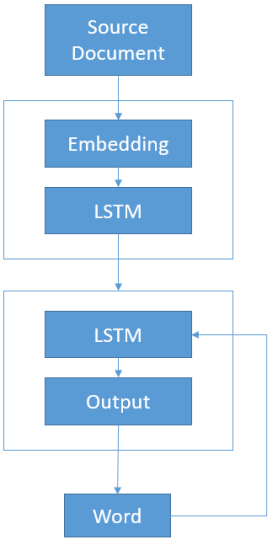
This dataset contains all published papers from the Neural Information Processing Systems (NIPS) conference. [47] We used the article's titles and abstracts; and extracted them to create two sub-datasets: nips-article-abstract and nips-abstract-headline. As cited by its author, it contains conference papers and articles, "ranging from the first 1987 conference to the current 2017 conference".

VI. IMPLEMENTATION MODELS

In this section we have illustrated, how the Encoder-Decoder architecture is implemented using various models for text summarization in the Keras deep learning library. Here we are going to elaborate on four models: General Model, One-Shot Model, Recursive Model A and Recursive Model B.

A. *General Model*

General model is a simple realization of the model. It involves an Encoder with an Embedding input followed by an LSTM hidden layer. That hidden layer produces a fixed length representation of the source documents. The decoder reads the representation along with the Embedding of the last generated word and uses these inputs to generate each word in the output. The following figure describes the architecture of the general text summarization process.



General Text Summarization Model in Keras

FIG. 3. GENERAL TEXT SUMMARIZATION MODEL IN KERAS

The general text summarization model has some drawbacks that can be explained as follows: Keras does not allow recursive loops where the output of the model is fed as an input to the general model automatically. Basically it means that the general text summarization model cannot be directly implemented using Keras, but can be implemented by using a more flexible platform like TensorFlow.

B. *One-Shot Model*

One-shot model as the name suggests, generates an entire output sequence in a single shot. In this model, the decoder solely uses the context vector to generate the output sequence. The figure below depicts the architecture of the One-shot text summarization model. One-shot model imposes a heavy burden on the decoder. Additionally, it becomes challenging for the decoder to generate a coherent output sequence, as it must choose the words with their order, without any sufficient context.

IJARCCE
ISSN (Online) 2278-1021
ISSN (Print) 2319-5940
International Journal of Advanced Research in Computer and Communication Engineering
ISO 3297:2007 Certified
Vol. 7, Issue 1, January 2018

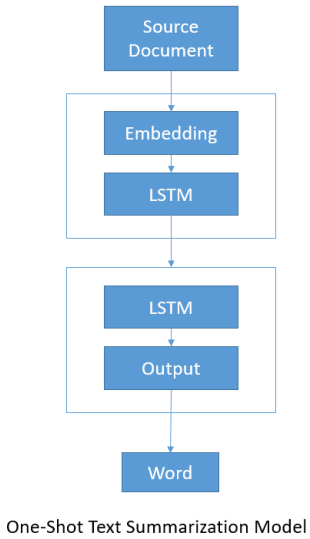


FIG. 4. ONE SHOT TEXT SUMMARIZATION MODEL

C. ***Recursive Model A***

Recursive model A is a second alternative model of the general text summarization model. It generates a single word forecast and calls it recursively. In this model, the decoder uses the context vector and the distributed representation of all words generated so far, as input in order to generate the next word.

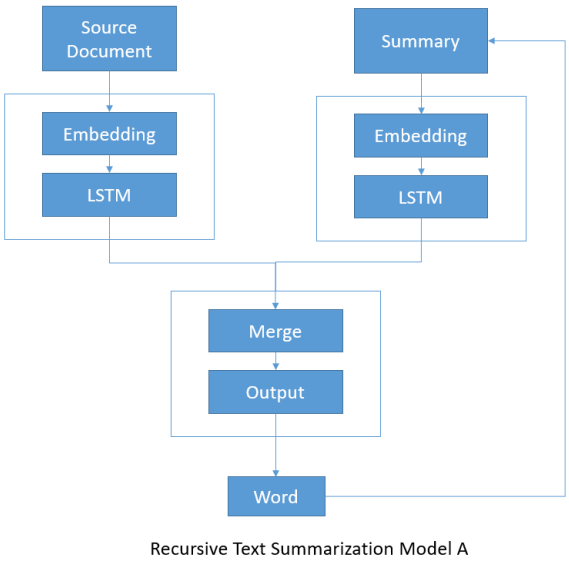


FIG. 5. RECURSIVE TEXT SUMMARIZATION MODEL A

In this model, the output summary is built by recursively calling the model and appending the outputs with the previously generated word repetitively. This model is better because the decoder is provided with an option to use the previously generated words and the source document as a context for generating the next word. This model burdens the merge operation and the decoder to infer where it is up to, in generating the output sequence.

D. ***Recursive Model B***

In the Recursive Model B, the Encoder generates a context vector representation of the source document. This representation is further fed to the decoder at each phase of the generated output sequence. This activity helps the decoder in building up the same internal state as was used to generate the words in the output sequence [55].

IJARCCE

ISSN (Online) 2278-1021
ISSN (Print) 2319-5940

International Journal of Advanced Research in Computer and Communication Engineering
ISO 3297:2007 Certified
Vol. 7, Issue 1, January 2018

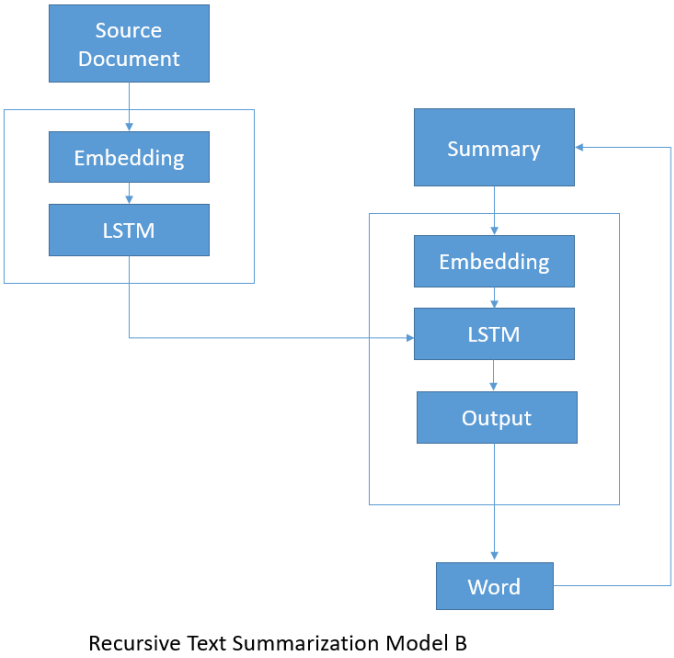


FIG. 6. RECURSIVE TEXT SUMMARIZATION MODEL B

As shown in the figure above, this process is then repeated by calling the model over and over again for each word in the output sequence until a maximum length or end of sequence token is generated.

VII. CONCLUSION AND FUTURE RESEARCH

In this research work, we have explored the machine learning approach to text summarization and have presented a range of novel models using neural networks. Because of its wide range of applications, extensive investigation has already been carried out on extractive text summarization techniques but because of its complexity, comparatively less research has been done on abstractive text summarization techniques. In our proposal, we have employed different models with variety of datasets, for trainable text summarizers using Keras and TensorFlow libraries and have analyzed all the results. In future we would like to continue our research by adding diverse sentence simplification techniques for aiming better results. Additionally, it can also be progressed by delving into executing it for multi-lingual and multiple documents conjointly.

REFERENCES

1. arXiv:1704.03242v1 [cs.CL].
2. D. Das and A. F. Martins, "A survey on automatic text summarization," Literature Survey for the Language and Statistics II course at CMU, vol. 4, pp. 192-195, 2007.
3. H. P. Luhn, “The automatic creation of literature abstracts,” IBM Journal of research and development, vol. 2, no. 2, pp. 159–165, 1958
4. http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.188.3982&rep=rep1&type=pdf
5. https://www.researchgate.net/publication/200042361_TextRank_Bringing_Order_into_Text
6. http://www.ajer.org/papers/v6(08)/ZE0608253260.pdf
7. P. Werbos. Backpropagation through time: what it does and how to do it. Proceedings of IEEE, 1990
8. D. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. Nature, 323(6088):533–536, 1986.
9. https://arxiv.org/pdf/1409.3215v3.pdf
10. M.Chandra, V.Gupta, and S.Paul, “A statistical approach for automatic text summarization by extraction,” In Proceeding of the International Conference on Communication Systems and Network Technologies, IEEE, 2011, pp. 268–271.
11. H. P. Luhn, “The automatic creation of literature abstract,” IBM Journal of Research and Development, vol. 2(2), 1958, pp. 159–165.
12. M. R. Murthy, J. V. R. Reddy, P. P. Reddy, S. C. Satapathy, “Statistical approach based keyword extraction aid dimensionality reduction,” In Proceedings of the International Conference on Information Systems Design and Intelligent Applications. Springer, 2011.
13. D. D. Lewis, “Naive (Bayes) at forty: The independence assumption in information retrieval,” In Proceedings of tenth European Conference on Machine Learning, 1998, Springer- Verlag, pp. 4–15.
14. J. D. M. Rennie, L. Shih, J. Teevan, D. R. Karger, “Tackling the poor assumptions of nave Bayes classifiers,” In Proceedings of International Conference on Machine Learning, 2003, Pp. 616–623
15. T. Mouratis, S. Kotsiantis, “Increasing the accuracy of discriminative of multinomial Bayesian classifier in text classification,” In Proceedings of fourth International Conference on Computer Sciences and Convergence Information Technology, IEEE, 2009. pp. 1246–1251.
16. C. Y. Lin, “Training a selection function for extraction,” In Proceedings of the eighth International Conference on Information and Knowledge Management, ACM, 1999, pp. 55–62.
17. K. Knight, D. Marcu, “Statistics-based summarization- step one: Sentence compression,” In Proceeding of the Seventeenth National Conference of the American Association for Artificial Intelligence, 2000, pp. 703–710.

18. L. Rabiner, B. Juang, "An introduction to hidden Markov model," Acoustics Speech and Signal Processing Magazine, vol. 3(1), 2003, pp. 4–16.
19. R. Nag, K. H. Wong, F. Fallside, "Script recognition using hidden Markov models," In Proceedings of International Conference on Acoustics Speech and Signal Processing, IEEE, 1986, pp. 2071–2074.
20. C. Zhou, S. Li, "Research of information extraction algorithm based on hidden Markov model," In Proceedings of second International Conference on Information Science and Engineering, IEEE, 2010, pp. 1– 4.
21. M. Osborne, "Using maximum entropy for sentence extraction," In Proceedings of the AssociaCL-02 Workshop on Automatic Summarization, ACL, 2002, pp. 1–8.
22. B. Pang, L. Lee, S.Vaithyanathan, "Thumbs up? Sentiment classification using machine learning technique," In Proceedings of the Association for Computational Linguistics conference on Empirical methods in Natural Language Processing, ACM, 2002, pp. 79–86.
23. H. L. Chieu, H. T. Ng, "A maximum entropy approach information extraction from semi-structured and free text," In Proceedings of the Eighteenth National Conference on Artificial Intelligence, American Association for Artificial Intelligence, 2002, pp. 786–791.
24. R. Malouf, "A comparison of algorithms for maximum entropy parameter estimation," In Proceedings of sixth conference on Natural Language Learning, ACM, 2002, pp. 1–7.
25. P. Yu, J. Xu, G. L. Zhang, Y. C. Chang, F. Seide, "A hidden state maximum entropy model forward confidence estimation," In International Conference on Acoustic, Speech and Signal Processing, IEEE, 2007, pp. 785–788.
26. M. E. Ruiz, P. Srinivasan, "Automatic text categorization using neural networks," In Proceedings of the Eighth American Society for Information Science/ Classification Research, American Society for Information Science, 1997, pp. 59–72
27. T. Jo, "Ntc (neural network categorizer) neural network for text categorization," International Journal of Information Studies, vol. 2(2), 2010.
28. P. M. Ciarelli, E. Oliveira, C. Badue, A. F. De-Souza, "Multi-label text categorization using a probabilistic neural network," International Journal of Computer Information Systems and Industrial Management Applications, vol. 1, 2009, pp. 133–144.
29. I. Guyon, J. Weston, S. Barnhill, V. Vapnik, "Gene selection for cancer classification using support vector machines" mach. learn. Machine Learning, vol. 46(1–3), 2002, pp. 389–422
30. T. Hirao, Y. Sasaki, H. Isozaki, E. Maeda, "Ntt's text summarization system for duc-2002," In Proceedings of the Document Understanding Conference, 2002, pp. 104–107.
31. L. N. Minh, A. Shimazu, H. P. Xuan, B. H. Tu, S. Horiguchi, "Sentence extraction with support vector machine ensemble," In Proceedings of the First World Congress of the International Federation for Systems Research, 2005, pp. 14-17.
32. J. Morris, G. Hirst, "Lexical cohesion computed by thesaural relations as an indicator of the structure of text," Journal of Computational Linguistics, vol. 17(1), ACM, 1999, pp. 21–48.
33. L. Chengcheng, "Automatic text summarization based on rhetorical structure theory," In International Conference on Computer Application and System Modeling, IEEE, 2010, pp. 595–598.
34. S. Brin, L. Page, "The anatomy of a large-scale hyper textual web search engine," In Proceedings of the Seventh International World Wide Web Conference, Computer Networks and ISDN Systems, Elsevier, 1988, pp. 107-117.
35. D. Horowitz, S. D. Kamvar, "Anatomy of a large-scale social search engine," In Nineteenth International Conference on World Wide Web, ACM, 2010, pp. 431–440.
36. M. Efron, "Information search and retrieval in microblogs," Journal of the American Society for Information Science and Technology, vol. 62(6), 2011, pp. 996–1008.
37. T. K. Landauer, P. W. Foltz, D. Laham, "Introduction to latent semantic analysis," Journal of Discourse Processes, vol. 25(2–3), 1998, pp. 259– 284.
38. B. Rosario, "Latent semantic indexing: An overview," Technical Report of INFOSYS 240, University of California, 2000.
39. T. Hofmann, "Unsupervised learning by probabilistic latent semantic analysis," Journal of Machine Learning, vol. 42(1–2), 2001, pp. 177– 196.
40. M. Simina, C. Barbu, "Meta latent semantic analysis" In IEEE International conference on Systems, Man and Cybernetics, IEEE, 2004, pp. 3720–3724
41. J. T. Chien. "Adaptive Bayesian latent semantic analysis" IEEE Transactions on Audio, Speech, and Language Processing, vol. 16(1), 2008, pp.198–207
42. D. Wang, T. Li, S. Zhu, C. Ding, "Multi-document summarization via sentence-level semantic analysis and symmetric matrix factorization," In Proceedings of the 31th Annual International ACM Special Interest Group on Information Retrieval Conference on Research and Development in Information Retrieval, 2008. ACM, pp. 307–314.
43. J. H. Lee, S. Park, C. M. Ahn, D. Kim, "Automatic generic document summarization based on non-negative matrix factorization" Journal on Information Processing and Management, vol. 45(1), 2009, pp. 20–34.
44. T. G. Kolda, D. P. O'Leary, "Algorithm 805: Computation and uses of the semi-discrete matrix decomposition," Transactions on Mathematical Software, vol. 26(3), 2000, pp. 415–435.
45. V. Snasel, P. Moravec, J. Pokorny, "Using semi-discrete decomposition for topic identification," In Proceedings of the Eighth International Conference on Intelligent Systems Design and Applications, IEEE, 2008, pp. 415–420.
46. D. Corney, D. Albakour, M. Martinez, and S. Moussa, "What do a million news articles look like?" in Proceedings of the First International Workshop on Recent Trends in News Information Retrieval co-located with 38th European Conference on Information Retrieval (ECIR 2016), Padua, Italy, March 20, 2016., 2016, pp. 42–47. [Online]. Available: http://ceur-ws.org/Vol-1568/paper8.pdf
47. B. Hammer, "Nips papers 1987-2016." [Online]. Available: https://www.kaggle.com/benhamner/ nips-papers
48. https://web.stanford.edu/class/cs224n/reports/2749103.pdf
49. https://arxiv.org/abs/1509.00685
50. https://arxiv.org/abs/1512.01712
51. https://arxiv.org/abs/1704.04368
52. https://arxiv.org/abs/1602.06023
53. https://arxiv.org/abs/1704.04368
54. https://arxiv.org/abs/1512.01712
55. https://arxiv.org/abs/1704.04368

BIOGRAPHIES

Meetkumar Patel is currently pursuing an Undergraduate degree in Computer Engineering from Ahmedabad Institute of Technology in affiliation with Gujarat Technological University. He is interested in Machine Learning, Natural Language Processing, Programming Methodology and Computer Security.

Adwaita Chokshi is currently pursuing an Undergraduate degree in Computer Engineering from Ahmedabad Institute of Technology in affiliation with Gujarat Technological University. She is interested in Machine Learning, Deep Learning, Natural Language Processing, Artificial Intelligence and Programming Methodology.

Satyadev Vyas is currently Head of Computer Engineering Department at Ahmedabad Institute of Technology in affiliation with Gujarat Technological University. He has a teaching experience of over 14 years and more than 15 years of industrial experience in R&D. His research work primarily concerns Internet of Things (IoT) and Python.

Khushbu Maurya is currently a professor in Computer Engineering Department at Ahmedabad Institute of Technology in affiliation with Gujarat Technological University. She has an experience of over 8 years in teaching field. Her research work primarily concerns Machine Learning and Artificial Intelligence.